

电 子 科 技 大 学

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

专业学位硕士学位论文

MASTER THESIS FOR PROFESSIONAL DEGREE



论文题目 基于服务的复杂装备数据分析技术研究

专业学位类别 工程硕士

学 号 201822010634

作 者 姓 名 于豪

指 导 教 师 杨鍊 副教授

分类号 _____ 密级 _____

UDC ^{注1} _____

学 位 论 文

基于服务的复杂装备数据分析技术研究

(题名和副题名)

于豪

(作者姓名)

指导教师

杨鍊

副教授

电子科技大学

成 都

(姓名、职称、单位名称)

申请学位级别 **硕士**

专业学位类别

工程硕士

工程领域名称

电子与通信工程

提交论文日期

2021.5

论文答辩日期

2021.5

学位授予单位和日期

电子科技大学

2021 年 6 月

答辩委员会主席

林水生

评阅人

李桓 潘晔

注 1：注明《国际十进分类法 UDC》的类号。

Research on Service-based Data Analysis Technologies of Complex Equipment

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Discipline: **Master of Engineering**

Author: **Hao Yu**

Supervisor: **A/Prof.Lian Yang**

School: **School of Information and Communication
Engineering**

摘要

在信息化战争的倒逼下,军用复杂装备逐渐向智能化方向发展。智能化的军用复杂装备涉及多个工程领域,导致领域间无法理解数据定义,出现“信息孤岛”现象。因此选用复杂装备作为场景,研究多领域间的资源管理以及异构数据集成技术。

本文主要有以下两个目的:第一,研究复杂装备数据的动态资源管理优化技术;第二,研究基于中间件服务架构的复杂装备数据集成优化技术。主要研究内容如下:

(1) 介绍复杂装备数据分析中涉及的相关背景和关键技术。

首先,介绍了复杂装备数据分析关键技术的基本概念和技术结构。然后,分析了几种主流的实现方法,并对比总结它们各自的优缺点。

(2) 提出基于异步消息队列的装备资源管理优化技术。

针对各军种、各部门复杂装备数据访问的特点,本文提出了一种基于异步消息队列的负载均衡框架,实现复杂装备资源动态管理优化。首先,本文分析了复杂装备数据权限分组访问和异步消息队列实现的可行性。然后,分别对以上技术进行实现。其中,权限分层访问控制包括分组权限控制和权限管理表设计;异步消息分发包括异步任务发布、任务调度器、任务执行单元三部分。最后,使用消息中间件存放消息数据以及任务结果。结果表明,本文方法降低了权限关系映射复杂度,减少权限响应时间;单机服务器响应速度对比其他主流方法平均提高 7.6%。

(3) 提出基于服务的复杂装备数据集成优化技术。

针对复杂装备异构数据的集成,本文提出了一种基于服务的复杂装备数据集成优化技术。首先,本文介绍了中间件集成架构以及轻量级的数据交换格式。然后,将复杂装备数据集成优化技术分为装备数据采集、数据结构转换、数据处理引擎、数据映射驱动模版和服务化发布五个部分。最后,针对耦合度、实时性以及查询性能方面,与其他集成技术进行对比。结果显示,本文提出的基于服务的复杂装备数据集成优化技术将平均响应时间降低至 1s,解决异构数据服务发现以及领域定义集成问题。

本研究获得的结果可用于解决复杂装备数据分析中资源动态管理、复杂装备数据集成等问题,且对复杂装备数据分析技术有重要延伸意义。

关键词: 多源异构数据, 负载均衡, 数据集成, 中间件

ABSTRACT

Under the pressure of information war, military complex equipment is gradually developing in the direction of intelligence. The intelligent military complex equipment involves many engineering fields, which makes it difficult to understand the data definition between fields and leads to the phenomenon of "information island". Therefore, the complex equipment is selected as the scene to study the resource management and heterogeneous data integration technology between multiple fields.

There are two purposes in this thesis: First, the dynamic resource management optimization technology in complex equipment data integration is discussed. Second, research the integration and optimization technology of complex equipment data based on middleware service architecture. The main research contents are as follows:

(1) The background concepts and key technologies of complex equipment data analysis are introduced.

Firstly, the basic concept and technical structure of the key technology of complex equipment data analysis are introduced. Then, several popular implementation methods are analyzed, and the advantages and disadvantages of these methods are compared and summarized.

(2) The optimization technology of equipment resource management based on asynchronous message queue is proposed.

According to the characteristics of complex equipment data access in various services and departments, this thesis studies a load balancing framework based on asynchronous message queue to realize the dynamic management and optimization of complex equipment resources. Firstly, this thesis analyzes the feasibility of hierarchical access to complex equipment data permissions and asynchronous message queuing. Then, the above technologies are implemented respectively. Hierarchical access control is divided into group access control and access management table design. asynchronous message distribution includes asynchronous task publishing, task scheduler and task execution unit. Finally, message queue is used to store message data and task results. The results show that this method reduces the mapping complexity of permission relation and the response time of permission; the response speed of single server is seven point six percent higher than that of other mainstream methods on average.

(3) The data integration and optimization technology of complex equipment based

on service is proposed.

Aiming at the heterogeneous data integration of complex equipment, this thesis studies a service-based optimization technology of complex equipment data integration. Firstly, this thesis introduces the middleware integration architecture and lightweight data exchange format. Then, the complex equipment data integration optimization technology is divided into five parts: equipment data acquisition, data structure transformation, data processing engine, data mapping driven template and service release. Finally, other integration technologies are compared in terms of coupling, real-time and query performance. The results show that the average response time is reduced to one second, which solves the problems of heterogeneous data service discovery and domain definition integration.

The results of this thesis can be used to solve the problems of resource dynamic management and complex equipment data integration in complex equipment data analysis. This thesis is of great significance to the data analysis technology of complex equipment as well.

Keywords: Multi-source Heterogeneous Data, Load Balancing, Data Integration, Middleware

目 录

第一章 绪 论	1
1.1 研究背景及意义	1
1.1.1 研究背景	1
1.1.2 研究意义	4
1.2 国内外研究现状	5
1.3 研究目的与内容	9
1.4 论文内容组织结构	9
第二章 复杂装备数据分析关键技术	11
2.1 引言	11
2.2 异构数据动态管理技术	11
2.2.1 基于轮询的动态管理技术	11
2.2.2 基于权重的动态管理技术	12
2.2.3 基于响应的动态管理技术	14
2.3 异构数据统一集成技术	16
2.3.1 基于 XML 的异构数据集成技术	16
2.3.2 基于联邦数据库的异构数据集成技术	18
2.3.3 基于中间件的异构数据集成技术	20
2.3.4 基于数据仓库的异构数据集成技术	22
2.4 异构数据清洗技术	24
2.4.1 基于插值法的异构数据清洗技术	24
2.4.2 基于最近插补法的异构数据清洗技术	26
2.4.3 基于固定值的异构数据清洗技术	26
2.5 异构数据挖掘技术	27
2.5.1 基于关联规则的异构数据挖掘技术	27
2.5.2 基于回归算法的异构数据挖掘技术	28
2.6 异构数据传输技术	29
2.6.1 基于 XML 的异构数据传输技术	29
2.6.2 基于 JSON 的异构数据传输技术	30
2.7 本章小结	30
第三章 复杂装备异构数据动态管理研究	31

3.1 引言.....	31
3.2 基于异步消息队列的装备资源管理技术.....	32
3.2.1 基于角色的访问控制	32
3.2.2 装备数据消息分发	38
3.3.3 装备数据消息存储	46
3.3 性能比较.....	46
3.3.1 装备数据动态管理部分	46
3.3.2 装备数据分发部分	47
3.4 本章小结.....	53
第四章 基于服务的复杂装备数据集成研究	54
4.1 引言.....	54
4.2 基于服务的装备数据集成技术.....	54
4.2.1 装备数据采集	56
4.2.2 数据结构转换	58
4.2.3 数据处理引擎	61
4.2.4 数据映射驱动模版	64
4.2.5 服务化发布	70
4.3 性能比较.....	73
4.4 本章小结.....	75
第五章 复杂装备异构数据作战场景的实现	76
5.1 引言.....	76
5.2 复杂装备异构数据作战场景实验.....	76
5.2.1 作战场景数据分析平台	77
5.2.2 复杂装备异构数据测试分析平台	81
5.3 本章小结.....	87
第六章 全文总结与展望	88
6.1 全文总结.....	88
6.2 后续工作展望.....	88
致 谢	90
参考文献	91
攻读硕士学位期间取得的成果	97

第一章 绪论

1.1 研究背景及意义

1.1.1 研究背景

近年来,随着复杂装备数据分析概念的深入理解和技术的高速发展,作为一种资源集成方法,统一资源接口访问技术已被相应部门高度关注。目前,复杂装备数字化的过程中,每个领域部门都构建可以满足该领域场景需求的数据接口系统。同时,各部门建立了相应异构数据源,将装备领域生成 TB 量级的数据进行存放。异构性的特点大多表现为不同硬件设备以及软件结构中,通常以不同形式的数据存储方式以及显示表达。复杂装备异构数据源之间数据结构以及系统有着巨大差异,海量不同版本的数据,导致这些异构数据归一处理和分析成为复杂装备信息化系统开发和全面异构数据集成需要面对的主要问题。为了提高装备资源利用效率,并为复杂装备信息化的全面集成、数据资源共享和系统服务供给高效的服务支持,因此需要设计数据集成系统来统一复杂装备相应领域中的异构数据源。该系统的设计是现代面临的重大挑战之一^[1]。

复杂装备的规模不断扩大,数据的异构程度也在增加。现代化装备具有技术先进性及结构复杂性的特点,如图 1-1。因此,复杂装备的生命周期往往会涉及到多个工程领域,典型的如需求分析、功能设计、行为描述及架构方案设计等过程。同时,复杂装备研发过程往往要对多领域、多维度的异构数据进行整合,对各领域的数据进行分析 and 挖掘,从抽象的装备数据中建立关联,得到装备体系的评估,从而形成最终决策。因此,在复杂装备生命周期过程中资源管理以及异构数据集成至关重要^[2]。

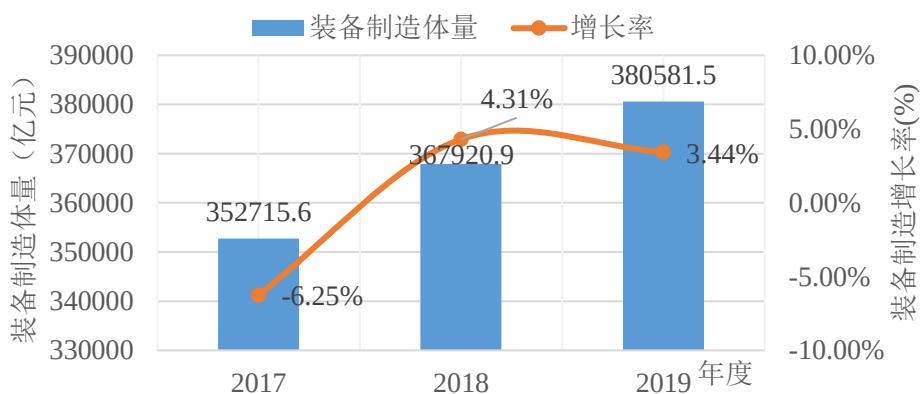


图 1-1 装备制造发展

当前,各种复杂装备的数据标准不同,使得不同装备研发部门的数据标准定义繁多以及交互接口不一致,各部门的异构数据传输有巨大阻碍,总体装备异构数据利用率较低。而传统的数据仓库等异构数据集成技术,极易导致数据冗余和不一致。装备系统越来越复杂,对装备的集成数据支持要求也越来越高。

同时,随着装备信息化的飞速发展,复杂装备领域不断升级,以及部门信息化进程的加速,装备部门间的数据交流需求越来越大。而服务领域、公共领域和其他行业已经采用了不同的独立应用程序系统来管理异构数据。随着信息化程度不断迭代,已经积累了许多不同的旧系统架构接口和旧数据。这些系统接口和数据类型处于不同的信息化阶段,由各部门的开发人员使用各类开发工具和不同的设计平台进行研发,因此存在异构性。

由于没有统一的异构数据标准,因此各装备部门数据管理系统无法达到异构数据交换和使用的目的。由此产生的“信息孤岛”广泛存在,这大幅度缩小了信息化装备建设的空间^[3]。

装备部门异构数据的集中表现为:(1) 计算机体系结构的异构性:成员数据库来自不同的领域平台,如大型服务使用的服务站、个人小型数据库或硬件系统等。

(2) 基本操作系统的结构不同:不同操作方式的系统,如:命令式、图形化界面、硬件系统等,底层指令执行结构不相同。(3) 数据库管理系统(Database Management System)自身的异构特性:虽然系统数据具有相同的解析方式,但数据管理系统可以不同,系统可以基于模板类型、基于数据关联、基于网络链接等。如图 1-2 所示。

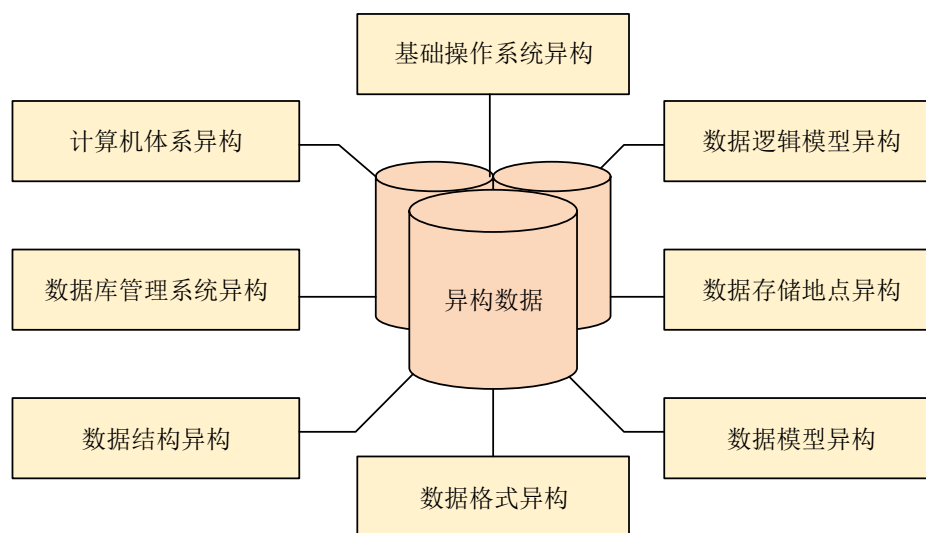


图 1-2 异构数据特征

数据源的异构性、系统产生的数据的多样性、不同子系统模型的不匹配会导致集成的错误和失败^[4]。为了彻底解决“信息孤岛”问题并实现装备异构数据资源利

用率的提高以及数据资源共享,信息集成研究逐渐崭露头角。数据集成的目标是通过应用全局数据模型以及通过检测和解决方案和数据冲突来融合各方面的数据,以便提供统一的统一视图^[5]。建立一个数据体系结构,在该体系结构中,来自不同企业的数据源将根据其含义和内容进行链接^[6]。复杂装备数据集成技术研究的关键是如何有效地融合各个装备领域的异构数据,隔离其分布特性和异构特性,并为用户提供高效的集成数据服务^[7]。复杂装备数据集成包括操作平台的集成、操作环境的集成、接口的集成、资源的集成、服务的集成等多个方面。其中,资源集成是其他方面集成的基石,也是复杂装备数据集成的关键环节^[8]。在装备数字化系统场景中,融合来自各领域异构数据源的信息是复杂装备数据集成的首要目的,以便各领域使用者可以开放式请求以上复杂装备异构数据源。对数字化系统进行集成,需要将各个交互集成过程中涉及到的数据保持完整,并尽可能多地采用多线程操作,增大数据流通吞吐量。同时,将数字化系统中的多源异构数据集成后,需要有统一的对外访问数据的接口服务,便于使用者对资源进行访问处理。

为了提高装备部门的数据分析处理能力,部门经常采用各种应用软件开发服务并依据自身领域范围来支持其部门的发展,这使得部门存放了大量的旧数据,同时部门所需的数据量也在不断增加^[9]。装备部门要实现异构数据共享,打通各个部门阻隔,提升数据交互效率,只能尽可能地利用现有装备数据资源,避开冗余的数据资源收集,减少相应的支出。由于各个领域部门使用的数据来源于各类系统中,要实现装备数据共享,各系统之间的数据格式类别以及体量都体现出了巨大的差距。因此,在一定程度上阻止了各装备领域之间的异构数据流通性^[10]。

如何有目的的集成异构数据是装备部门加强领域竞争力首要关注的问题。业务范围广的部门使用提高硬件性能和叠加服务器个数的方法,以此适应海量的用户请求。投入更多的成本费用来使得硬件提升性能以及服务器节点个数增加,但更关键的是硬件设备能否充分发挥其应有的性能。所以,巨额成本费用,让一些访问流量小的部门采用另一种方式,从软件方面对整体系统以及服务进行提高。

综上所述,从软件方向提高服务器性能是各大小领域装备部门都需要关注的核心问题。为了达到理想需求,集群(Cluster)系统的多服务器策略是最常用的解决方案,如图 1-3。集群系统拥有多组节点,通过一定的映射关系,将各个节点进行分发数据消息,但对外仅保留规范一致的接口进行访问。

对于服务器集群,需要将大量并发任务消息分发到各个处理单元中,通过各个处理单元降低系统并发处理压力,解决了一个处理单元无法处理大量请求的问题。长期以来,在多服务节点系统中,负载均衡(Load Balancing)是协调各个节点处理负载的关键技术。负载均衡的主要思想是将大量的任务分配到各个执行单元中处理,

降低单个处理单元的压力^[11]。集群中的执行单元的处理能力各有不同,接收到的任务消息复杂程度以及执行单元当前状态,都能够影响执行单元的响应时间。与此同时,也要避免持续地对某个执行单元分发任务,导致性能降低。巨大的数据处理能力可由集群系统提供,使得外部应用程序服务中具有统一的性能,集成网络使得系统具有系统统一的管理结构。集群中的节点数量可以根据部门领域请求量的需求进行调整。

与简单的搭建一个整体高性能的系统相比,该解决方案不仅降低了搭建高性能系统带来的技术难度,同时,降低了搭建平台的资金投入,提高了系统的鲁棒性以及拓展性。当集群中某节点出现了故障或者网络阻塞造成的节点僵死,集群模式下的系统将不会有太大的影响,而是会将故障节点停用,将任务分发到其他执行单元继续运行系统,实现系统的可靠性。利用集群的特点,可以将单个性能一般的执行单元整合起来,让系统整体发挥出高性能,降低技术门槛。

大量并发响应的环境是负载均衡技术首要的使用情况。采用特定的分发规则算法将获取到的请求,正确发送给后端节点进行处理,使得系统提高响应吞吐量,平均请求响应时间也相应减少。

按照请求响应分发的理论,可将负载均衡通过硬件设备和软件程序分发,前者常见的硬件型负载均衡器包括 Sangfor 系列等,适用于大型装备部门,因为其需要大量资金投入且需要高额成本维护;后者通常采用 Nginx 作为分发核心,适用于硬件设备有限的场合。

软件分发任务消息的形式只能是在算法上进行优化,对于集群性能要求不高的场景下,可以通过设计优秀的负载分发算法,对系统做优化。当场景需求严格时,则需要通过更换硬件分发设备进行系统升级,以彻底提高分发性能。但后者需要大量资金的支持,前者的方法更适合更多的场景需求变化。由于其维护成本低、配置简单并且使用灵活,所以受到了广泛的关注和应用^[12]。

1.1.2 研究意义

在复杂装备的信息化建设过程中,不同的武装力量根据自身需求开发了自己的复杂装备信息化系统^[13]。但是,由于缺乏统一的领导和总体规划,系统数据的存储模型和数据源是不同的。这些也导致复杂装备异构数据源在各个方面的异构性,并且信息互操作性和资源共享也很困难。如何整合这些异构数据,成为等待解决的问题^[14]。

复杂装备数据集成是军队数据工程中数据管理的基础。数据集成^[15]的目标是通过提供统一的对象模型接口来满足用户查询的需求,以查询各种不同的数据源。

集成源的信息资源可以作为一个新的单一源，即数据集成系统^[16]。

装备数据工程通常包含各种各样的数据类型，包括传统的结构化数据。例如存储在关系数据库中用于保存领域场景的数据，并且包括需求管理中的文本、视频测试采集的图像、转换后的音频信息和其他各类多媒体数据，甚至包括各种网络资源，这些数据结构不同给数据管理带来很多不便。

不能满足军队装备数据工程中数据管理的要求，尤其是当数据管理分布在多个节点中时，随着节点大小的增加，关系数据库中一致性和完整性的约束会大大降低数据库的性能，数据查询响应速度很慢并且可以满足实际使用数据的需求。生成和快速更新数据的频率和及时性很强。数据生成和更新的频率快，并且数据处理的及时性强。随着我军信息化水平的提高，军备比以往更快地生成各种类型的数据。此外，由于军事活动要求“士兵迅速”，数据处理通常有时间限制。

当前的信息社会已经进入大数据时代。2012年3月，美国政府根据当前数据信息化的大潮流，发布了“大数据研究与开发计划”^[17]。作为科技数字信息化推动力之一，美国政府启动了一项价值2亿美元的大数据开发计划。这项计划在信息领域中被美国政府视为一项关键性措施。一段时期内获取、管理和处理某一范围的资源后，传统软件工具不足以提供相应服务。支持判断性能、资源扫描，达到覆盖范围大、集成种类多的要求。这是美国大数据成为对新处理模型的需求的新方式^[18]。

面向复杂装备业务场景，使用基于服务的异构数据集成技术，对复杂装备数据生命周期涉及的工具进行集成，开发一站式的跨领域数据分析平台。解决现有复杂装备数据互通困难，数据流分析过程自动化低、结构复杂、工具集成程度低等问题。

此外，复杂装备的数据集成技术，目前在国内仍存在缺口。综上所述，复杂装备生命周期存在异构数据集成以及数据动态管理的问题。

1.2 国内外研究现状

二十世纪九十年代，在各个领域都渴望以一种廉价的方式提高计算性能的背景下，由美国 UIUC 大学的 NCSA 首先推出提供服务的节点集群的理念，以解决当前服务端的性能问题。此后，Cisco、IBM 等一些相关部门也开始对该技术进行高度关注。随着对该技术纵向研究，该方法取得了重大进展，国内外对负载均衡技术的改进也随着各个科技领先的团队研究在迅速推进。国内外一些大型装备部门已经制造了大量节点负载均衡器装备，较为主流的是 F5 生产的 F5Networks BIG-IP、Sangfor AD 系列、华为 L2800 等。使用负载均衡算法是将各个节点的任务消息进行正确地将发布到相应节点执行^[19]。国内外研究团队也发表一些新的负载均

衡研究方法，以推进节点负载均衡技术的进步。

2012 年，印度的 Jitendra Bhatia、Tirth Patel、Harshal Trivedi 出等人提出一种应用在服务器实例的动态负载均衡算法^[20]。该研究方法不间断检测各个节点的任务处理状态，在检测周期结束后，将替换并保存包含该执行单元的加权参数的集合。当接收到新的任务消息后，系统将通过加权参数集合，对接收到的信息进行分发。该算法完整结合了网络传输中的即时负载和响应时间，使系统可以达到较高的执行效率和较高的系统性能。

较为普遍的负载平衡算法包括直接遍历所有节点的轮询算法^[21]，通过引入权值的比例概率选中的加权轮询算法^[22]，随机选中服务节点的随机分配算法和动态反馈调度^[23]。但是，随着多节点集群方法的飞速发展，国内外对负载均衡技术的研究也在迅速推进。如参考文献^[24]所述，提出了一种基于负载权重的动态负载均衡算法。服务单元的性能以及权重的设定将影响该算法最后的分发效果。参考^[25]提出了一种基于临界加速度和减量的一致性哈希负载均衡算法。该算法使用虚拟节点执行真实的服务器节点分配，并使用加速递减的关键因素机制来控制节点的负载，从而避免了过多的负载响应。例如，在参考文献^[26]中，考虑了模拟退火算法在服务器集群负载均衡方面的优势。

在我国，对分布式计算技术的研发已经有很长时间的研究。2012 年，江西财经大学的熊乃学^[27]，华中师范大学的徐开华和陈立龙等人提出了一种基于网络的执行单元映射负载均衡算法^[28]。在服务器上构建了一组树状表结构，以保存高频次访问的服务器下标。接着将其他任务消息分发到相应的服务器中。

2012 年，北京邮电大学张新宇，赵永利等人提出了一种软件模拟的计算机的方式实现动态负载均衡算法^[28]。该方式通过使用软件模拟计算机的方式搭建实验环境，对系统环境的资源占用率进行采集。针对采集到的参数进行分析，修改当前负载配置状态，达到性能的优化。在实验环境下，实现数据资源低负载的分配效果，使得服务节点性能提高。

2013 年，武汉理工大学的罗树华等提出了一种基于服务提供节点的集群动态权重最小负载均衡算法^[29]。他们将节点内存使用量、节点 CPU 使用量和实时网络拥塞程度作为计算节点权重的参数，并达到了节点权重的动态调整的实验效果。由此算法可以使网络更加稳定运行，负载分配性能更加。

2017 年，电子科技大学的王永辉第一步是将集群中的节点分类进行组织分配，每个类都有一个中央节点，中央节点收集公共节点的负载信息^[30]。第二步，中央节点将采集到的负载数据发送至节点均衡器，节点均衡器获取负载信息给各个节点生成权重信息。对于每个请求，选取权重最大的前三个节点，接着根据权重比任意

选取组内一个节点给客户端提供服务^[31]。

2020 年, 北京邮电大学的常洁琳研究了面向数据中心网络的负载均衡算法, 通过建立数据中心, 搭建规模庞大的数据服务器以实现更优的负载均衡效果。

上世纪 70 年代中期异构数据集成的研究逐步开启, 随着众多团队以及研究人员的不断深入研究, 异构数据集成技术逐步变为研究异构数据集成。如: 基于中间件的集成^[32]、XML 的集成等^[33]、模式集成方法和数据复制方法^[34]。根据以上研究成果, 相关研究人员不断对此技术进行改进^[35]。

Hammer Mcleod(1979)提出一种将各类数据源通过一个统一的中心数据存储结构进行管理, 通过系统将各类异构数据源形成逻辑整体, 也就是联邦数据库的最早的模型架构。

联邦数据库的最重要特点是独立的资源系统可以在计算机上执行命令的同时, 加入联邦数据库的任务执行。联邦系统的普通使用者以及管理员都可以对单位 DBMS 进行集成操作。联邦系统客户端的请求决定整体系统集成的程度^[36]。参与联邦系统, 同时分享联邦系统数据库单位的数据资源。

数据仓库是 WH Inmon 在 1991 年提出的。其核心方法包含用于数据统一存储的数据仓库以及数据集合、数据提取、转换和加载(Extract-Transform-Load, ETL)技术^[37], 即元数据技术^[38], 在线分析处理(OLAP)^[39]。空间数据仓库(SDW)是 GIS 和数据仓库技术的组合产品。它通过加入分析不同维度的数据以优化整合数据仓库, 贴合不同的 GIS 数据应用场景系统^[40]。因此, 它可以为研究各类场景的数据资源决策支持有效的数据建议等^[41]。

国外的科技数据领先团队在数据集成领域方面, 早已开展相关的深入研究^[42]。Linkoping 大学基于面向数据对象的集成功能查询优化技术在数据库实验室开发^[43]、IBM 公司研发的 WebSphere Information Integrator 系统^[44]。Oracle Data Integration 工具能够整合各类数据以及为用户提供相应的访问链接, 同时对其他产品也能做到相应的融合使用。

2003 年 5 月发布的 IBM DB2 Information Integrator 通用数据库平台是联邦数据库系统的管理软件。它依靠 IBM 优秀的平台搭建能力以及最初的资金投入快速发展, 因此, 用户可以通过统一的方式对非集中式不同结构的数据进行处理整合。同时, 该平台具有合并、暂存、转换和复制数据的功能。

2019 年, 辽宁大学冯勇等人通过建立高校环境场景, 创建元数据的模式, 脱离特定场景数据本体, 通过元数据实现数据集成。

在国外已经有许多优秀的项目, 如 Myriad 移动软件领域系统、全局信息系统(IS-Global)、加拿大土地信息系统(Albertal System)、通用万维网接口系统。以上系

统的在集成研究、统一访问语句、数据划分、界面视觉处理技术、工程架构、性能、单元数据库类别、查询执行效率、安全性和访问权限、联合事务规范和其他方面是有区别的。

近年来,国际系统工程理事会(INCOSE)认识到工具集成的重要性: INCOSE 成立了工具集成和互操作性工作组(TIIWG),以满足拥有集成工具和环境以提高生产率和质量的需求系统工程^[45]。生命周期协作开放服务(OSLC)^[46]作为工具集成的新标准,如图 1-4,为软件产品整个生命周期中的不同集成方案提供了一组规范。在基于 OSLC 的工具链中,只要这些工具符合并实现 OSLC 规范,就可以轻松对其进行修改。OSLC 通过建立开发组件之间的关系,结合统一集成描述复杂系统的系统工程概念,提供这些组件或者领域数据的含义以及它们之间的关系来实现形式上的统一表达。因此,OSLC 表示系统工程的概念,并将其引入并应用于工具、数据集成解决方案。因此,使用 OSLC 也可使用到数据集成中使用。



图 1-4 OSLC 标准

OSLC 是一个开放社区支持的新标准,解决这些问题,用于集成在软件开发的各个阶段中使用的工具的新标准。它可以在应用程序的整个生命周期中建立不同数据组件之间的关系^[47],为工具集成提供规范^[48]。OSLC 得到了 IBM、西门子、SHELL、NASA 喷气推进实验室和通用汽车等多家公司和组织的支持。国际系统工程理事会(INCOSE)认识到工具集成的重要性。INCOSE 成立了工具集成和互操作性工作组(TIIWG),以解决集成工具和环境对提高生产率和质量的需求^[49]。此外,可以通过 OSLC 格式将不同的数据描述为统一的表示形式。统一的表示使跨域的数据集成成为可能^[50]。

1.3 研究目的与内容

在信息化战争的倒逼下,军用复杂装备逐渐向智能化方向发展。智能化的军用复杂装备涉及多个工程领域,导致领域间无法理解数据定义。同时也存在各领域间的异构数据集成困难,出现“信息孤岛”现象。复杂装备场景变化快速,工具平台多样,当前资源管理以及集成方案无法应对新情况的出现。因此选用复杂装备作为场景,研究多领域间的资源管理以及异构数据集成技术。

复杂装备相对复杂主要体现在,如何将多个领域的用户权限资源管理的复杂度降低,各类异构数据如何形成一个统一且包含领域定义的数据结构,系统任务的执行如何实现异步化处理,降低响应时间。

主要研究内容如下:

(1) 介绍复杂装备数据分析中涉及的相关背景概念和关键技术。

首先,介绍了复杂装备数据分析涉及的关键技术基本概念、技术结构。然后,分析了几种主流的实现方法,并对比总结它们各自的优缺点。

(2) 提出基于异步消息队列的装备资源管理优化技术。

针对各军种、各部门复杂装备数据的访问特点,本文研究了一种基于异步消息队列的负载均衡技术,该技术结合基于角色的访问控制,实现复杂装备资源动态管理优化。首先,本文分析了复杂装备数据权限分组访问和异步消息队列实现的可行性。然后,分别对以上技术进行实现。其中,权限分层访问控制包括分组权限控制和权限管理表设计;异步消息分发包括异步任务发布、任务调度器、任务执行单元三部分。最后,使用消息中间件存放消息数据以及任务结果。结果表明,本文方法降低了权限关系映射复杂度,减少权限响应时间;单机服务器响应速度对比其他主流方法平均提高 7.6%。

(3) 提出基于服务的复杂装备数据集成优化技术。

针对复杂装备异构数据的集成,本文提出了一种基于服务的复杂装备数据集成优化技术。首先,本文介绍了中间件集成架构以及轻量级的数据交换格式。然后,将复杂装备数据集成优化技术分为装备数据采集、数据结构转换、数据处理引擎、数据映射驱动模版和服务化发布五个部分。最后,针对耦合度、实时性以及查询性能方面,与其他集成实现技术进行对比。结果显示,本文提出的基于服务的复杂装备数据集成优化技术将实时平均响应时间降低为 1s,实现数据与服务的解耦合,解决数据服务发现问题。

1.4 论文内容组织结构

本文主要对复杂装备数据分析技术中重要的两个关键技术进行研究，包括基于异步消息队列的装备资源管理优化技术研究，以及基于服务的复杂装备数据集成优化研究。在此基础上对两个关键技术设计场景实验实现验证。全文一共分为六个章节并在最后加入致谢以及参考文献部分。下面是各个章节主要内容及其安排：

第一章，首先，介绍了复杂装备异构数据集成的背景与意义，梳理了在复杂装备数据领域中，异构数据的规模、数据资源动态管理以及异构数据带来的问题。列举了迄今为止国内外学者对异构数据集成技术与异构数据资源动态管理技术的研究状况。然后，介绍了本文的研究目的与主要研究内容。最后，介绍了文本的研究内容与结构上的安排。

第二章，本章首先对复杂装备数据分析领域进行介绍，列举本领域所涉及到的关键技术以及技术之间的关系。然后，针对每个关键技术研究点，分析目前所主流的研究方法，从技术的实现难度、查询性能、数据冗余等角度，对技术的优劣作对比。最后，针对列举的五个关键技术，选择复杂装备数据领域中影响较大的两个技术进行深入研究分析。

第三章，本章首先介绍了复杂装备异构数据动态管理的背景及原理。然后，使用异步消息队列、装备数据消息分发以及装备数据中间件，构建复杂装备数据动态管理架构，同时使用角色访问控制，实现权限分组管理。最后，针对数据查询效率、系统消耗资源占比等指标，将本方法与基于响应的动态管理技术对比。

第四章，本章首先介绍复杂装备异构数据集成的背景及原理。然后，介绍基于服务的装备数据集成技术的总体流程、设计架构以及模块分层。集成技术流程包括，装备数据采集、数据结构转换、数据处理引擎、数据映射驱动模版以及服务化发布。其中数据映射驱动模版还包括装备数据标准定义介绍以及装备数据映射规则定义。最后，针对系统耦合度、数据实时性、增加基表记录的性能影响等方面，将本方法与基于数据仓库、基于联邦数据库的数据集成技术进行对比。

第五章，本章首先介绍了复杂装备异构数据场景的背景、相关异构数据格式以及权限管理需求。然后，针对复杂装备数据场景做了以下两个实验分析平台：作战场景数据分析平台以及复杂装备异构数据测试分析平台。对复杂装备异构数据集成以及数据资源动态管理进行融合，验证本方法的整体实现效果。

第六章，对本文所做的工作进行总结。同时，对复杂装备数据集成领域中，异构数据集成优化技术以及资源动态管理优化技术提出了展望。

第二章 复杂装备数据分析关键技术

2.1 引言

本章首先介绍了复杂装备数据分析所涉及到的关键技术，同时分析了各个关键技术当前主流的研究方法及各自方法的特点。在复杂装备数据领域中，涉及多军种、跨部门的数据访问及管理问题，同时，数据结构以及标准定义不同导致集成困难，出现数据鸿沟。因此，在众多关键技术中，对复杂装备数据分析有关键影响的是异构数据动态管理技术以及异构数据统一集成技术，如图 2-1。

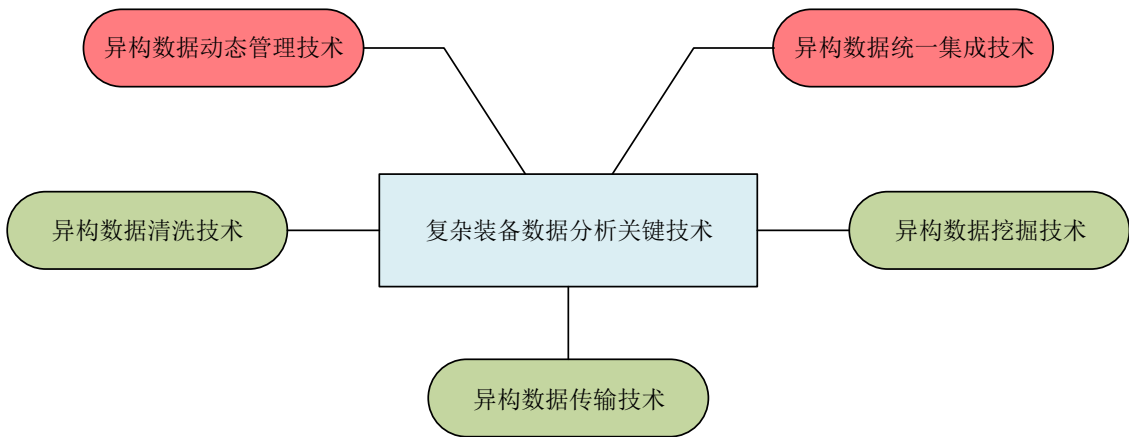


图 2-1 复杂装备数据分析关键技术

2.2 异构数据动态管理技术

2.2.1 基于轮询的动态管理技术

轮询算法(Round-Robin)是最简单的一种负载均衡算法^[51]。该算法实质是将集中的请求逐一分发到集群中的执行单元，考虑执行单元的负载均衡^[52]。从节点 1 开始标号，直到节点 N，重置节点指针下标后继续从头扫描。

假设有 N 个节点服务： $S = \{S_1, S_2, \dots, S_{index}, \dots, S_n\}$ ，一个标志变量 *index* 代表上一次选择的服务器 ID。变量 *index* 被初始化为 N-1，判断服务器正常运作后，即可选中该节点作为该次请求负载点，如图 2-2。

轮询算法假定所有节点的处理性能一致，并且忽略每个节点的实时连接数和响应时间。当某个执行单元响应时间变大以至于僵死状态，该算法可能由于执行单元的僵死而停滞在该处。因此，此算法适用于执行单元组中所有执行单元具有相对一致的处理性能。

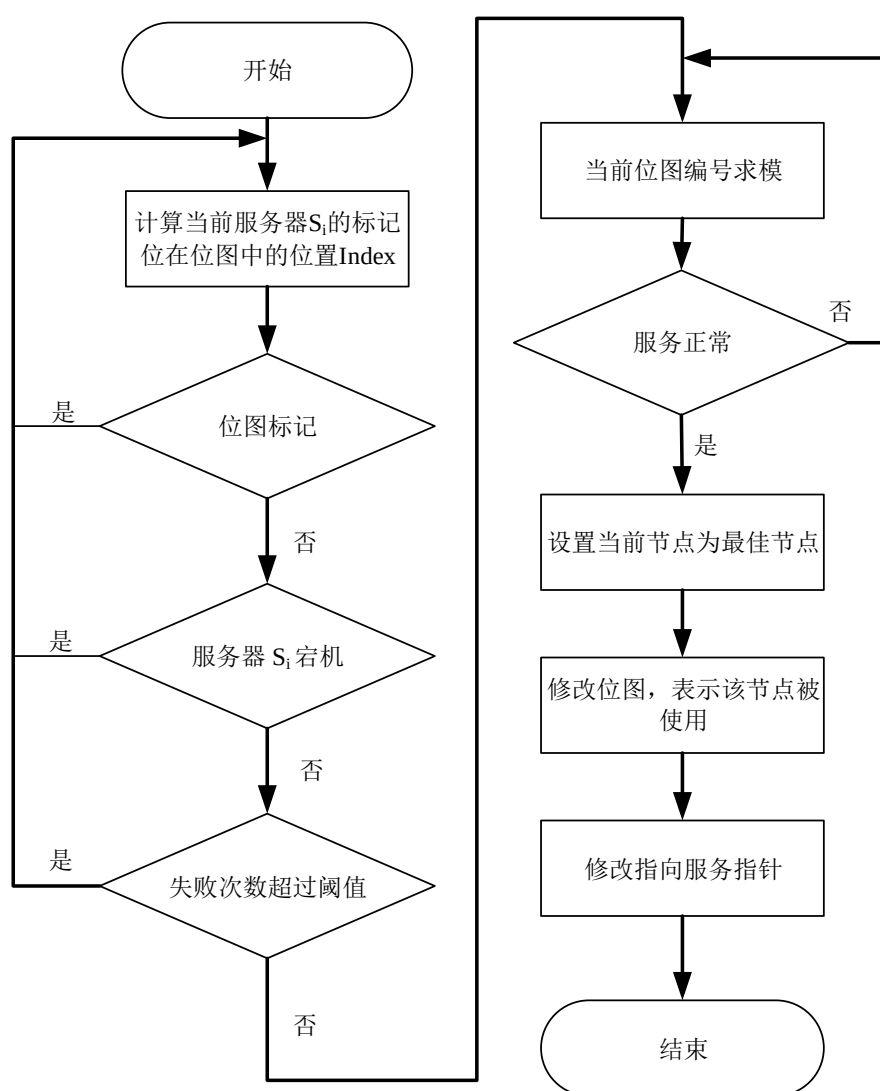


图 2-2 基于轮询的动态管理算法流程

轮询是负载平衡中一种相对基本且简易的算法，工程配置中不需要其他参数。假定有 N 个节点在配置文件中设定负载，使用算法扫描节点列表，同时根据节点顺序在每一轮中选择一个节点处理响应。扫描并执行过列表中的节点后，本方法将重新选取第一个节点再次扫描。

特征：不同的节点将根据算法均按时间顺序将各个请求进行分配。因此它适合集群的服务器连接方式，其中每个节点接收同样的负载。但是对于具有不同节点性能的群集，此方法可能会引起诸如不合理的资源分配的问题。

2.2.2 基于权重的动态管理技术

各个节点的数据处理性能在轮询算法中并未考虑，现实中有一定概率需要考

虑各个节点的性能。由于配置文件在各个节点中，已安装的领域应用程序不一致。它的处理能力将有所不同，超重负载将导致节点负载不平衡^[53]。如何合理地将用户需求分配给节点是负载均衡迫切需要解决的问题^[54]。

因此，加权轮询算法的原理是：根据服务节点不同的任务执行性能，为各个服务节点设置不同的权重，以使其可以接受具有相应权重的任务消息。

例如，将序列 $A=[1,2,4]$ 配置核心服务节点的权重值，权重对应相应的服务节点为 $S=[SerA,SerB,SerC]$ ，通过 Nginx 中 Upstream Cluster 的方式将服务节点权重配置进行设置。加权轮询算法通过权重设定，产生一组节点序列。该序列数组会在每个请求到来时，移动指向服务节点的指针到下一节点来顺序处理请求。对于以上示例，加权轮询算法将生成序列为： $[SerC,SerC,SerB,SerC,SerA,SerB,SerC]$ 。

简而言之，加权循环算法需要输出一组后端节点执行顺序数组。其中包含 n 个后端执行节点， $sumWeight$ 代表的是所有后端节点权值求和。按照此顺序，各个执行单元在一个周期内将执行与权重值相应的任务次数，使得最后各个执行节点处理任务的个数求和等于其权重值。

同时，生成的服务节点序列分配需要大致均匀，如图 2-3。例如，按照服务节点访问序列： $[SerA,SerA,SerA,SerA,SerA,SerB,SerC]$ 的顺序，前五个请求将分配给服务器 SerA。SerA 节点在一段时间被频繁访问，因此以上生成的节点序列分配不合理，因此更好的访问顺序应该是： $[SerA,SerA,SerB,SerA,SerC,SerA,SerA]$ 。

该算法的原理为：在服务节点集合 A 中，扫描整个服务节点权重集合得到最大值 $WeightMax(A)$ 。接着根据各个节点权值的情况，计算出所有服务节点权重的最大公约数 $WeightGCD(A)$ 。

分配权值的过程中， $getWeight$ 函数是关键代码。该函数内部通过扫描整个服务节点权重给系统提供参数：最大公约数 $WeightGCD$ 、权重最大值 $WeightMax(A)$ 以及权重之和 $sumWeight(A)$ 。

$currWeight$ 的值，其波动幅度是一组等差序列： $[WeightMax,WeightMax-WeightGCD,WeightMax-2WeightGCD,...,0]$ 。使得 $currWeight$ 赋值为 $WeightMax$ 降低至 0，称作一次扫描周期。对于各个 $currWeight$ ，算法是将整个服务节点集扫描一次，生成的结果序列将由权重大于等于当前指针指向的权重值服务节点加入。扫描服务节点集合一个周期后，将当前指向的权重指针指向下一个节点，然后再次扫描服务节点集合一个周期。

在更改当前权重的指针的过程中，无论当前权重的指针的当前值是多少， $WeightMax(A)$ 的服务节点都必将被替换。因此，具有最大权重值的服务节点将在结果序列中生成 $WeightMax(A)/WeightGCD(A)$ 次。

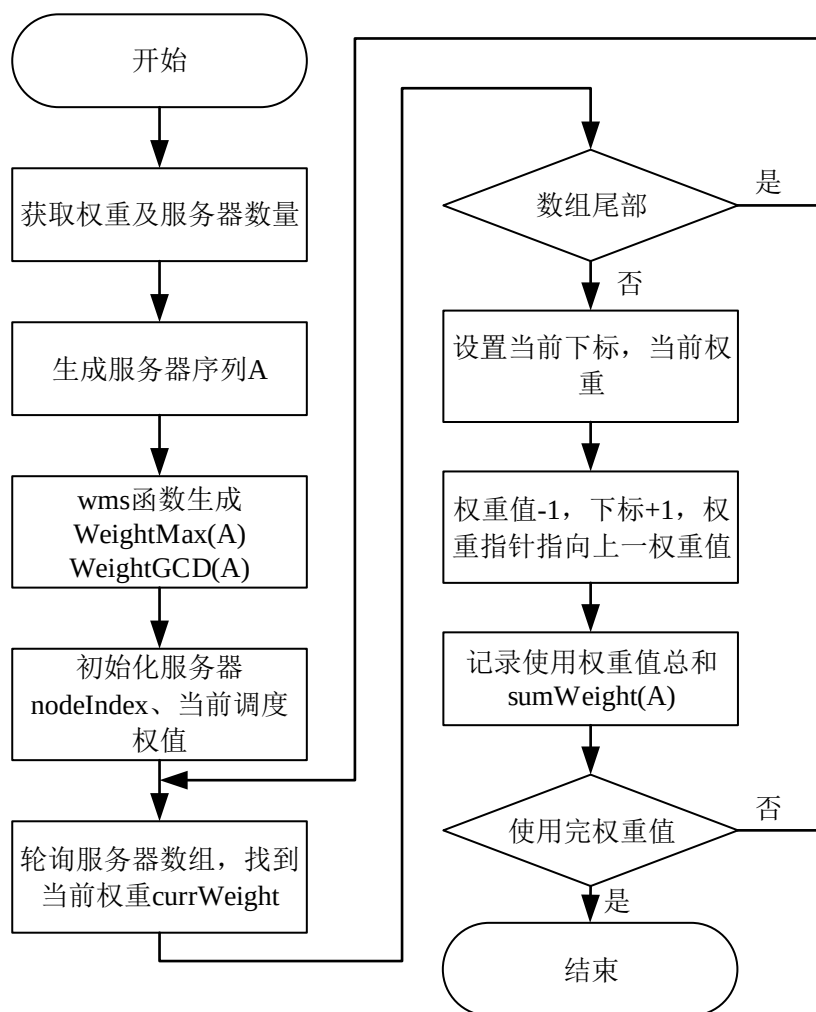


图 2-3 基于权重的动态管理算法流程

可以看出，Nginx 进行响应分发时，选择的服务节点可能不是目前权重（处理性能）最大的服务节点。如上所述，如果请求数量足够，选出的服务节点将不一定是按顺序的。最后，可以实现根据整个服务节点的权重，按照相应的比例将请求分配到各个服务节点上。

加权轮询算法分析服务器性能以分配每个服务器的权重，并依次遍历每个服务器。权重较高的服务器分配更多请求。轮询算法假定所有服务器具有相同的性能，因此所有服务器在调度时分配相同数量的请求。该算法简单易实现，但是当服务器关闭或过载时，循环算法无法相应地进行调整^[55]。特点：加权轮询在服务节点处理响应能力不一致的节点组中使用，以使响应请求分布更优。

2.2.3 基于响应的动态管理技术

如何选择候选服务并将其组合是网络服务组合中的关键问题^[56]。根据节点的

响应时间,调整各个服务节点服务的请求数,与固定次数的权重值相比,该方法可以更加智能、合理,同时最大程度地利用资源来实现负载部署。响应动态管理算法如下图 2-4。

针对搜集权重值的一次循环,能够整体地认知当前节点组的负载情况同时可以将时间保证在一个可控范围内。但由此带来的问题是频率较高地采集数据,导致系统大量时间进行采集而占用系统资源,给系统带来负担以及消息拥塞。以上问题原因在于,负载节点分配时收集器在进行服务节点状态统计,占用过多资源。

由实验可知,负载信息将收到收集器以及运算时系统资源占用参数的影响,导致负载均衡系统不能够接收到实际节点负载情况。因此,为了避免这种情况发生,一方面需要修改负荷信息的收集周期;另一方面,可通过平均值设定来减少计算负担,减少算计权值的负荷以及系统卡顿。

平衡算法的动态权重获取将会定期执行。假定默认初始权重存在,则扫描获取后端服务的权重列表,通过对后端响应状态的判断,得到动态的权重配置信息。最后比对后端服务最初的权重和收集的动态权重,通过权重公式计算,得到最后配置的权重值。

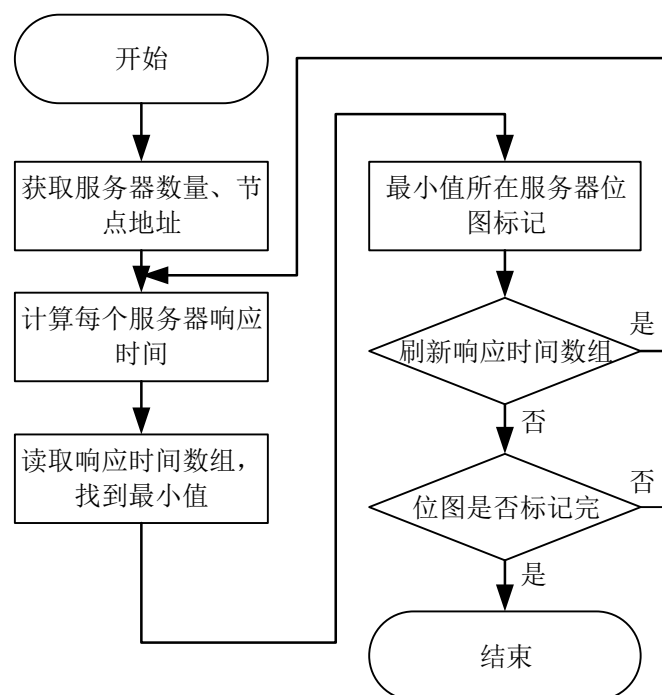


图 2-4 基于响应的动态管理算法流程

所提出的方法可在分配任务后预测虚拟机的状态。所提出的方法防止了该计算机上可能的额外负载,因此在虚拟机之间建立了负载平衡^[57]。该算法具有较高的灵活度,但现实中的网络环境复杂,即便动态配置也难以解决所有问题。

2.3 异构数据统一集成技术

2.3.1 基于 XML 的异构数据集成技术

基于 XML 的复杂装备异构数据集成技术的原理是，异构数据的融合核心根据 XML 标准使用格式转换来解决下层数据源的异构性，同时在此基础上构建异构数据集成注册层。为了提高数据信息的应用价值，将收集的 HTML 数据转换为 XML 数据，然后根据特定需求对数据进行转换^[58]。

在文献^[59]中，XML 技术作为异构数据源中间层集成的一种方法^[60]。使用网络服务提供中间服务封装异构数据源，注入集成交换层链接分布式异构数据源以支持跨平台的数据访问服务。各个数据源的连接信息、模式以及注册服务信息与数据源的对应关系，可通过数据集成中心建立注册服务完成。

利用以上特性，可使用该标记语言的方式实现数据交互，消除复杂装备数据之间的异构特性。一般的，使用标准化定义的集成规则，将带有参数的结构化查询语言写入至 XML 文件中。使得无需提前设置配置文件与异构数据之间的映射规则。根据用户的显示需求，此处的 SQL 语句可以与领域逻辑进行对接。

在数据互换步骤中引入并运行注入的 SQL 语句，通过 SQL 语句而返回的数据集展示在语句下方，进一步生成最终的 XML 文件。使用这种方式，将获取的查询数据集返回到数据共享层中，最后，用户通过共享层获取返回的资源，完成多维度领域资源整合的需求。

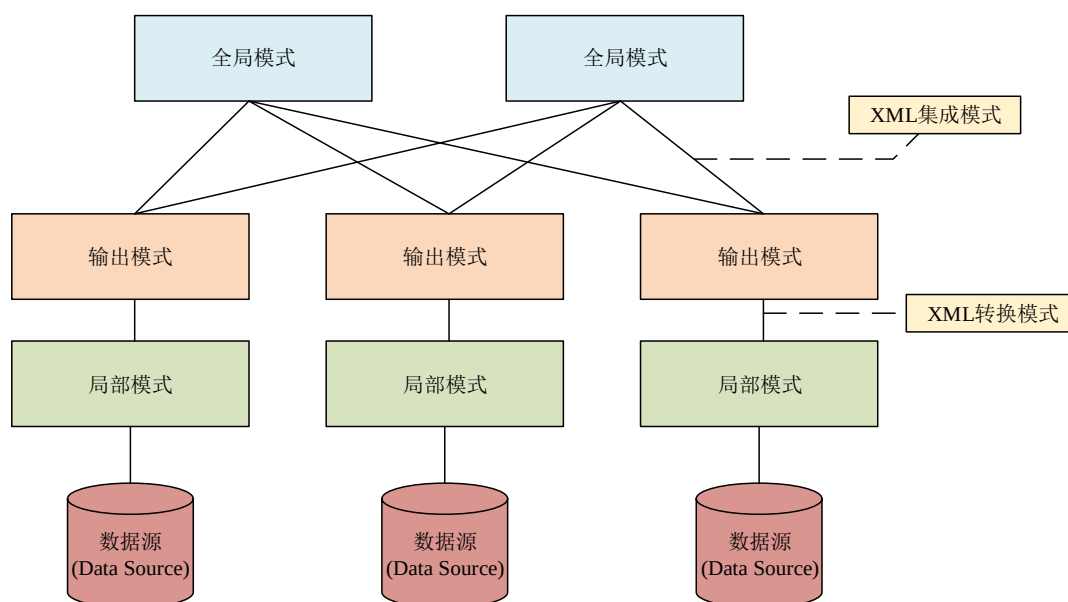


图 2-5 基于 XML 集成模式结构

在共享系统数据中心资源的基础上,合理挖掘数据特征,科学分析集成价值,并在异构交换中执行基于 XML 数据的集成处理步骤,以将异构信息转换为通用 XML 文档。

XML 规范模式统一表现为,将各类需要集成的模式通过模式转换进行集成。通过全局模式实现了数据的同一表达,图 2-5 所示为该方法的整体转换集成架构。图 2-6 为该方法集成步骤。其中局部模式为系统底层的资源接口或数据存储,通过 XML 规范定义各个局部模式的映射关系,得到融合的数据形式。全局模式代表的是用户最终访问的数据存储形式,通过通用的 XML 模板,屏蔽了各个资源的不一致性。因此,它提供了服务端异构资源的统一访问链接。这也使得异构资源融合系统的透明程度提高。

图 2-6 为一种由下而上的数据集成步骤,通过接入各类局部模式数据,通过对其格式进行转换后,接入到系统中进行数据解析处理。在比较阶段,将分析要融合的数据源组,以找到不同格式元数据之间的联系和异构。

在生成全局模板的步骤中,主要需要比较数据结构以生成定义和结构数据信息,此过程需要数据融合技术人员介入。最后使用获得的元数据之间的联系与异构信息来统一输出模板。

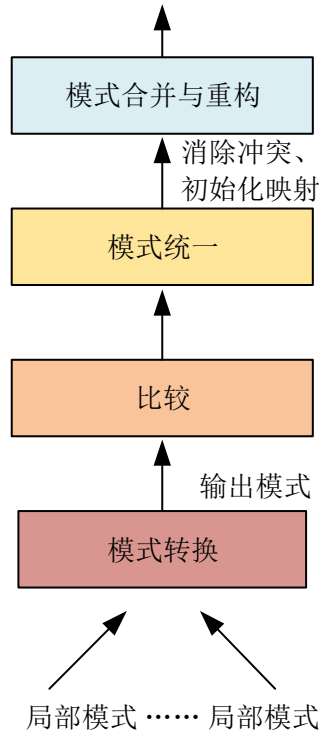


图 2-6 基于 XML 集成步骤

这种基于 XML 的异构数据集成方法更适用于领域资源访问量相对小但资源分布在不同领域场景应用中。XML 具有其自描述的数据格式，允许在单个行或文档中存储各种数据^[61]。在数据整合过程中，没有数据处理过程，这可以保证数据的质量。对原始领域场景数据完整度影响很小，可以较快地提升装备数据集成效率。

2.3.2 基于联邦数据库的异构数据集成技术

Federation（数据联邦）是当前复杂装备领域中异构数据集成技术之一。特别是局部非整体的系统，数据存储库内的数据与子系统存放的数据有不一致性。核心的数据集成模型是 FDBS（Federated Database System，联邦数据库系统）。数据库获取数据，数据资源更新和其他执行系统则在平台之间维护^[62]。

然而各部门都有一组系统，这些系统具有各自的一套数据库组织解决方法。CDBS（Component Database System，成员数据库）中的成员数据将存放在各自分布的区域中，但各个成员之间有着相互通信链接的通道。FDBMS（Federated Database Management System，联邦数据库管理系统）是组织成员数据库的管理系统，通过对成员数据一一链接进行控制，使用者通过该系统获取所连接异构数据源。

标准的 DBMS 可从拓展性以及模块复用方面提高软件的性能，从而节省装备系统的生命周期成本^[63]。联邦数据库系统和子类数据系统的链接关系如图 2-7 所示：

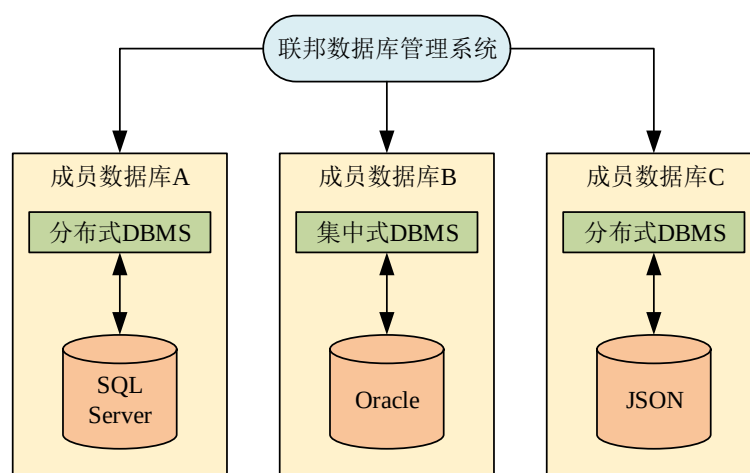


图 2-7 联邦数据库管理系统架构

从图 2-7 可知，联邦数据库管理系统连接着多个成员数据库，其中包含分布式以及集中式的数据源。如军工领域装备资源系统包括了各军种的战备资源应用，而每个军种的数据库子系统有着特定的系统应用进行管理。同时，对数据库进行构建查询、封装操作等。集中式 DBMS、分布式 DBMS 或其余的 DBMS 等，说明数据

库各自也有相应的类型划分。无论是 SQL Server、Oracle，或者是 XML 的数据库系统，亦或是集中式、分布式的子系统，都能够加入管理系统中。

联邦数据库系统(FDBS)有三个特征，例如装备管理资源系统：

(1) 各种服务的数据库系统所有者的特定资源处理规则，同时，也是顶层复杂装备联邦数据库的成员系统。

(2) 各个领域的复杂装备资源系统可由顶层复杂装备联邦数据库链接并获取资源。

(3) 由于成员数据的存储位置不是集中式的，导致复杂装备资源系统的成员系统中的资源都不集中存放。

紧密耦合的 FDBS 消除了数据结构不同等问题，但过于集中式的数据集成也会导致其他问题的出现。紧密耦合 FDBS 具有一致的模型，该模型是基于子数据库中数据的结构和特征重新构建出来。成员需要自己构建统一数据库，同时还要构建联邦模型。FDBS 系统的整体映射结构，如图 2-8 所示，该系统完成了主要的管理数据库和各个业务场景应用子系统之间的联系。

最简单的数据集成模式是联邦数据库，集合中的元素数据库都是逐一对应。为了达到各个数据库与其他数据之间的能够相互查询，因此必须解决各个数据库之间的结构异构，提供用于其他数据库数据结构变换的对应关系，如 SQL Server、Oracle，可通过集中式数据库集成到联邦系统中。

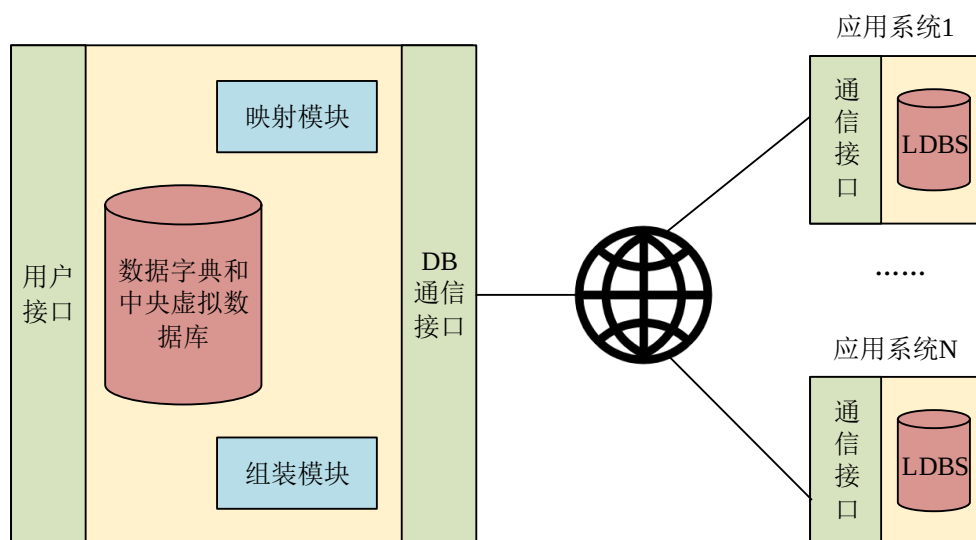


图 2-8 子应用系统与中央数据库映射关系

意味着异构数据库系统需要构建 $N(N-1)/2$ 映射规则或者设计 $N(N-1)/2$ 组代码规范等来提供各个数据库之间的查询数据功能。

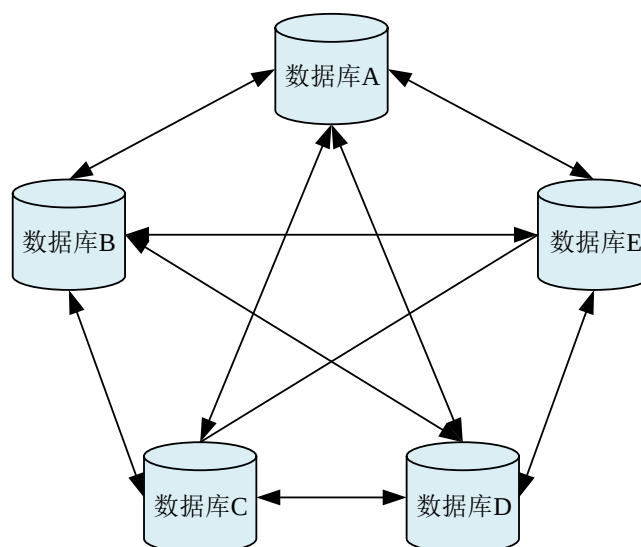


图 2-9 子数据库间映射关系

模型示意图如图 2-9 所示，在联邦数据库数据融合方法中，必须构建很多映射规则，才能够向集成应用中添加新数据源。这就导致只有通过修改软件结构来达到系统之间数据资源的正常交互。若各个成员系统需要修改，则会导致整体系统的应用拓展以及系统的高度耦合、鲁棒性等。

在图 2-9 中， N 超过一定数量范围时，使用该方法用于数据库的映射集成会导致实现方法复杂度的陡增，如以上数据融合方法中的映射数量 $N(N-1)$ 也会不断增大。可以得出结论，FDBS 在集成较少数据方面具有性能优势，并且可以达到系统自身管理，因此它不能用于集成和处理大量数据。

2.3.3 基于中间件的异构数据集成技术

中间件数据融合基于“共享数据模型”。本质上，数据仍然存储在加入集成的每个异构数据源中。数据通过每个数据源的“封装方法”集中转换为共享数据模型，并且用户查询基于基于模型的共享数据。也就是说，中间件模型用于集成各种异构数据库，而无需修改元数据的存储和组织模型^[64]。

中间件根据使用成员传达的共享数据模型，将查询任务划分并分解为对一个或多个数据源的查询。无需担心空间数据的特定格式^[65]。

然后，将数据源的查询集合整理转换为公共数据层的数据结构，并将集合发送到调用方。通过消除调用者受到基础数据源的结构不一致的影响，从而实现相应数据源与调用者的查询直接指向。但返回的结果集合是所有数据源结果的整合。中间件异构数据集成架构如图 2-10 所示：

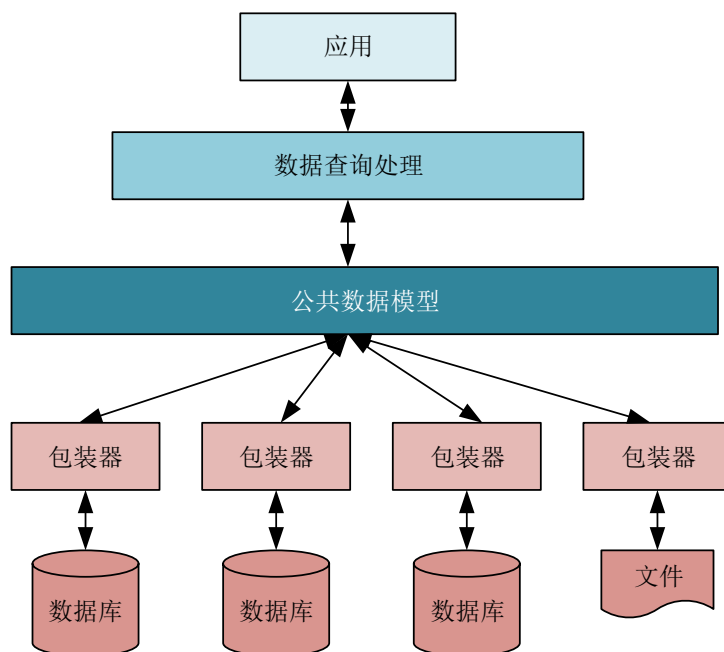


图 2-10 中间件异构数据集成架构

中间件集成方法也被称之为“Mediator/Wrapper”方法，这是另一种典型的模式集成方法^[66]。各个数据仍存储在各自的数据源中，但方法将通过一个在逻辑上屏蔽异构的方式，将所有对数据的操作统一为一种执行语句。中间件作为一个中间层，提供查询语句、结果集合的存放。

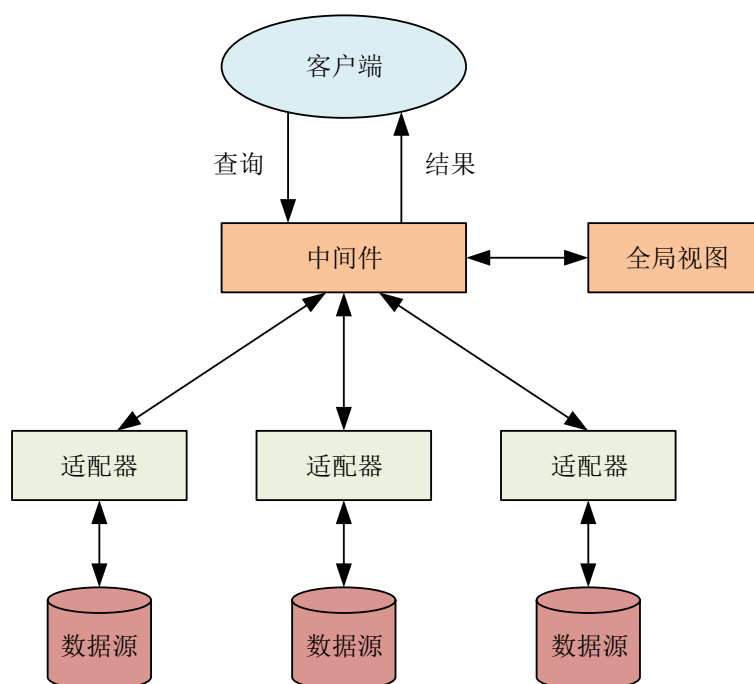


图 2-11 中间件适配器集成

同时,链接着与不同的数据源各自对应的适配器。构成数据融合中间件系统架构基本的组成内容。全局模式的执行响应通过中间件使用配置好的结构规范,将其解析为对每个数据源的本地模式的下层响应,同时相应的数据源匹配器将会接收到各个下层响应进行处理。然后,匹配器将每个下层请求解析为可以查询和执行响应的数据源,当查询到相应的数据资源后,会在反馈结果之前封装执行集合。如图 2-11 所示。

相比于联邦数据集成系统的设计方式,中间件系统架构的设计方式能够更好地兼容各类异构数据进行集成。同时,适用于数据源变化较快,对数据反馈响应要求较高的数据融合方案^[67]。但是,此方法通常仅支持对数据源的只读操作。在同时需要读取和写入任务的情况下,它不如联合数据库那么有效^[68]。

综上所述,模板集成方法经常用于资源高频率更新,资源响应时间要求高,领域场景难以判定的情况。在数据仓库等方式中,都需要将各个数据库链接耦合在系统中,若有场景业务需求变动,则需要大量改动。中间件集成方法能够较好地解决各类数据形式的适配不足等问题。

2.3.4 基于数据仓库的异构数据集成技术

在异构数据的处理中,有一种集成的解决方案可以通过复制数据来实现数据公开发布访问。在这种集成方法下,使用了数据仓库的设计模式。数据仓库核心思想是将来自各个场景和各个领域数据库的数据存储在数据仓库中。这样,使用者在获取数据时可以直接执行查询语句:公共数据库,这是数据仓库^[69]。

数据仓库的方法是构建共有的数据库,为了用户便捷访问数据,系统需要根据一定的规范对这些数据库中的数据执行处理。但是在处理资源结构以集成一致的数据仓库模型时,消耗更多的系统存储资源,导致资源过度冗余。同时,过滤数据的过程也增加了使用者查询资源的响应。

通常来说,使用 ETL 工具来处理异构资源,ETL 对数据执行提取、转换和加载任务,实现数据仓库法。例如,领域 A 的装备资源进行数据融合过程中,第一步要构建共享装备资源控制逻辑,然后使用 ETL 工具处理领域 A 中数据库中的所有数据,以获得一致的数据并将其加载到公共领域中。该工具的主要转换步骤如图 2-12。

如图 2-13,数据仓库法的体系结构,首先对不同的资源进行相应的转换程序,加入到 ETL 工具中。然后,通过 ETL 工具将数据进行采集,存放到一个统一的数据库中,通过数据的转换和加载。最后,用户可以通过统一的查询语句对公共数据库进行操作,实现资源融合。

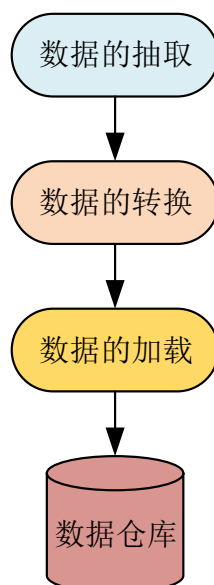


图 2-12 ETL 过滤步骤

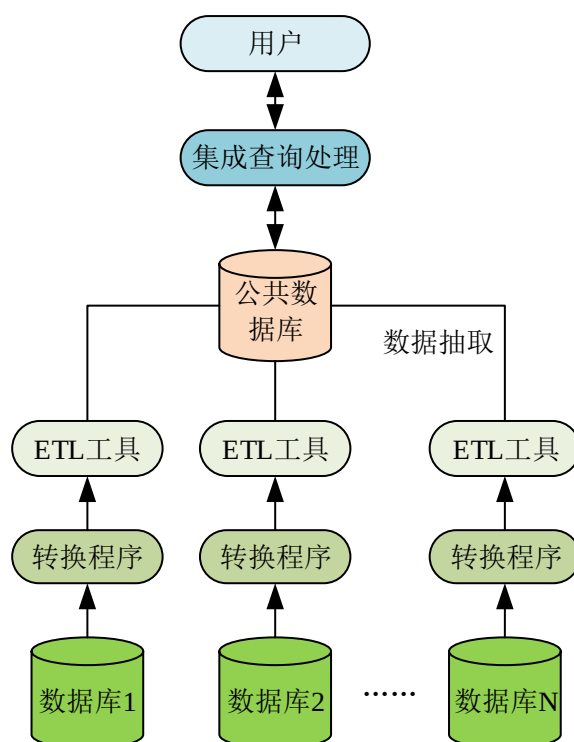


图 2-13 数据仓库法的体系结构

与对数据库进行归一化以减少响应时间相比，数据仓库在分析查询方面比数据库具有更高的性能，这与对数据库进行归一化以减少冗余数据的数据库相反^[70]。但是，公共数据库使用的是统一的存储系统对数据进行保存，因此某些最新资源信息无法及时同步。若存放的数据量较大，构建公共数据库需要有较大的资金投入以

及技术支持。因此，数据仓库方法在数据量达到情况下并不是数据集成的最优方式。

2.4 异构数据清洗技术

2.4.1 基于插值法的异构数据清洗技术

(1) 拉格朗日插值法

作为插值法参数的参数值，通过在若干个位置各异的点进行采集得到，使用拉格朗日插值法可以计算化简得到符合幂次规范的若干代数式，该多项式表达出来的仅仅是获得每个观测点的参数值。由此得出的多项式称为拉格朗日（插值）多项式。从几何平面上看，该参数集合可以使用数学函数作图的方式，通过在坐标轴上的多个给定点形成一条函数线，从而得到多项式函数。

根据数学公式，在一个平面上有 n 个点，然后可以找到阶数为 $n-1$ 的多项式，使得该多项式可以通过这些 n 个点，并且可以基于该原理预测以下缺失值。假设多项式：

$$y = 1 + a^1x + a^2x^2 + \dots + a^{n-1}x^{n-1} \quad (2-1)$$

将 n 个点的坐标 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ 带入得：

$$y_1 = a_0 + a_1x_1 + a_1x_1^2 + \dots + a_{n-1}x_1^{n-1} \quad (2-2)$$

$$y_2 = a_0 + a_1x_2 + a_1x_2^2 + \dots + a_{n-1}x_2^{n-1} \quad (2-3)$$

$$y_n = a_0 + a_1x_n + a_1x_n^2 + \dots + a_{n-1}x_n^{n-1} \quad (2-4)$$

解出拉格朗日插值多项式：

$$\begin{aligned} L(X) &= y_1 \frac{(x-x_2)(x-x_3)\dots(x-x_n)}{(x_1-x_2)(x_1-x_3)\dots(x_1-x_n)} \\ &\quad + y_2 \frac{(x-x_1)(x-x_3)\dots(x-x_n)}{(x_2-x_1)(x_2-x_3)\dots(x_2-x_n)} \\ &\quad + \dots + y_n \frac{(x-x_1)(x-x_2)\dots(x-x_{n-1})}{(x_n-x_1)(x_n-x_2)\dots(x_n-x_{n-1})} \\ &= \sum_{i=0}^n y_i \prod_{j=0, i \neq j}^n \frac{x-x_j}{x_i-x_j} \end{aligned} \quad (2-5)$$

将与缺失参数值相对应的点 x 赋值到上述推导的插值多项式中，以计算得到缺失参数的近似值 $L(X)$ 。

拉格朗日插值公式各项类型相似，便于参数推导以及数据上的分析。但是，采

集到的参数数量不同时,插值多项式个数也将相应更改,因此当应用场景相对变化时,变量个数不固定,该算法难以操作。为了克服这一缺点,提出了牛顿插值法。

(2) 牛顿插值法

求已知的 n 个点 $(x^1, y^1), (x^2, y^2), (x^3, y^3), \dots, (x^n, y^n)$ 的所有阶差商公式:

$$f[x_1, x] = \frac{f[x] - f[x_1]}{x - x_1} = \frac{f(x) - f(x_1)}{x - x_1} \quad (2-6)$$

$$f[x_2, x_1, x] = \frac{f[x_1, x] - f[x_2, x_1]}{x - x_2} \quad (2-7)$$

$$f[x_3, x_2, x_1, x] = \frac{f[x_2, x_1, x] - f[x_3, x_2, x_1]}{x - x_3} \quad (2-8)$$

$$f[x_n, x_{n-1}, \dots, x_1, x] = \frac{f[x_{n-1}, \dots, x_1, x] - f[x_n, x_{n-1}, \dots, x_1]}{x - x_n} \quad (2-9)$$

联立差商公式组推得以下插值多项式 $f(x)$ 。

$$\begin{aligned} f(x) &= f(x_1) + (x - x_1)f[x_2, x_1] \\ &\quad + (x - x_1)(x - x_2)f[x_3, x_2, x_1] \\ &\quad + \dots \\ &\quad + (x - x_1)(x - x_2)\dots(x - x_n)f[x_n, \dots, x_1, x] \\ &= P(x) + R(X) \end{aligned} \quad (2-10)$$

其中:

$$\begin{aligned} P(x) &= f(x_1) + (x - x_1)f[x_2, x_1] \\ &\quad + (x - x_1)(x - x_2)f[x_3, x_2, x_1] \\ &\quad + \dots \\ &\quad + (x - x_1)(x - x_2)\dots(x - x_{n-1})f[x_n, \dots, x_1, x] \end{aligned} \quad (2-11)$$

$$R(x) = (x - x_1)(x - x_2)\dots(x - x_n)f[x_n, \dots, x_1, x] \quad (2-12)$$

$P(x)$ 是插值逼近函数式, $R(x)$ 是误差函数式, 将与缺项参数值相对应的点 x 引入插值多项式, 以获得缺失项的近似参数 $f(x)$ 。

牛顿插值与拉格朗日插值一样,也是多项式插值的方式,但是它使用的研究法式计算生成插值多项式,与拉格朗日插值法相比,它具有沿袭程度高,多项式内容易变的特点。从本质上讲,两个方法得出的结果是相同的,但是表达式的形式不同。

2.4.2 基于最近插补法的异构数据清洗技术

使用(KNN,K-NearestNeighbor)算法进行填补,实际上将对象列视为对象标量。使用该算法进行数据拟合,实现预测补充丢失的数据字段。对于连续对象数据,采用的是判断数据权重进行加权平均的算法进行预测数据。

作为一种插补方法,K最近邻算法非常有效且易于实现^[71]。在这种方法中,考虑到与目标样本最相似的 k 个样本,估算了样本的缺失值。使用距离函数确定两个样本的相似性。算法如下:

1、将数据集 D 分为两部分。令 D_m 为包含样本的集合,其中至少缺少一个变量。其余样本将完成名为 D_c 的集合中的变量信息。

2、对于 D_m 中的每个向量 x :

将丢失的样本向量划分为观察到的部分和丢失的部分。

计算缺失样本与集合 D_c 中所有样本向量之间的距离。仅使用完整集 D_c 中样本向量中的变量,这些变量在向量 x 中观察到。

让 $x_j = [x_{j1}, \dots, x_{jp}]$ 包含缺失数据, $x_i = [x_{i1}, \dots, x_{ip}]$ 表示完整数据。

定义了输入空间中数据点之间的距离度量,其计算公式为:

$$d_{ij} = \sqrt{\left(\sum_{l=1}^p (x_{il} - x_{jl}) \right)^2} \quad (2-13)$$

重复 d_{ij} 的计算,多个变量填写缺失值。

2.4.3 基于固定值的异构数据清洗技术

修复数据缺失以及重复资源只是数据清理的其中一步。还存在不同的处理方法:使用摘要记录资源删除、区分或修改不正确或不准确的资源、修改资源格式。现实中收集的数据资源往往含有误差和逻辑不一致的部分。在理想场景下,模拟和数据解析过程中会产生误差,这对分析判断影响很大。通常,此类误差现象可归因于不确定性。不确定性有两个含义,包括可通过概率来描述的每个资源信息点本身以及可通过多种方式来描述的资源信息点属性值的不确定性。例如使用概率密度函数或者方差描绘属性值。

当缺失的数据集中存在某个初始值或平均线时,可以使用常量替换它。如果缺少普通装备的制造价格,则可以根据部门的平均装备成本标准进行评估。如图 2-14。

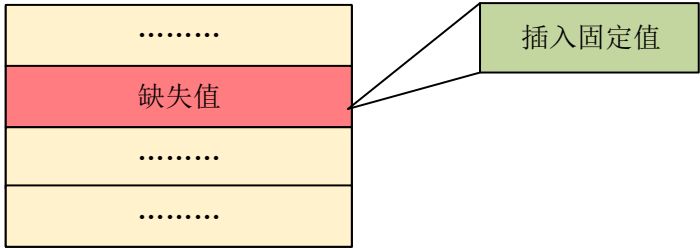


图 2-14 固定插值法

2.5 异构数据挖掘技术

2.5.1 基于关联规则的异构数据挖掘技术

关联规则描述通过找寻数据集中元数据与其他数据的关系，推导出若干数据之间的依赖或联系。如果若干个元素中间关系能够通过数学式子进行表达，则这种表达关系叫做关联。

通过寻找的潜在可表达的联系可将关联分为以下几类：关联参数少的简单关联、随着自变量变化的时间关联以及参数前后相互影响的因果关联。如果两个或多个元素之间存在关联情况，则另一个数据可以预估其中一个数据。

项的集合，定义为 U ，表示项集。项集的数量为 k ，称为 k 个项集。组成数据项集合的基本元素，定义为 u 。

关联规则 $A \Rightarrow B$ 的映射关系。其中 A, B 为 U 的真子集，并且 $A \cap B = \emptyset$ 。 A 称为前提， B 称为结果。关联规则反映出， A, B 参数的变化或波动是一致相关的，存在相应的关联规则^[72]。

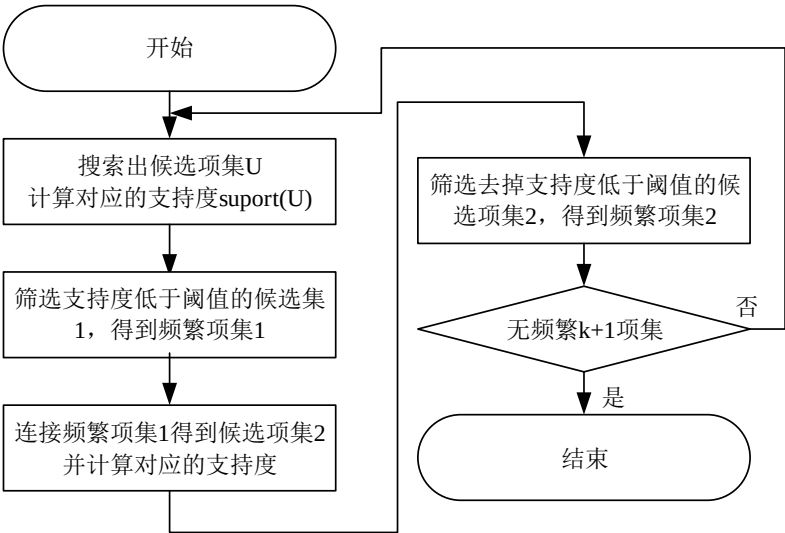


图 2-15 Apriori 算法流程

关联规则的相关程度是同时包含 A 和 B 的事务数与总集合中所有事务数之比。它反映了 A 和 B 中包含的项目同时出现在事务集合中的可能性，如公式 2-14。

$$support(A \Rightarrow B) = support(A \cup B) = p(AB) \quad (2-14)$$

关联规则的置信度是事务集包含 A 和 B 的事务数与总集合以及包含 A 的事务数之比，如公式 2-15。

$$confidence(A \Rightarrow B) = \frac{support(A \cup B)}{support(A)} = p(B|A) \quad (2-15)$$

通常情况下，使用者要求特定的某个范围的支持度能够符合规范的最小值和置信度符合规范的最小值。关联规则必须符合这两个最小参数范围。如果关联规则符合大于或等于支持度的最小值，则它只是一个强关联的情况，反之亦然。一般情况下，事务符合强关联规则。项集 U 在 T 中所占的比例就是他的支持度，如公式 2-16。

$$support(U) = \frac{\| \{t \in T | U \subseteq t\} \|}{\|T\|} \quad (2-16)$$

对于元素集 U ，满足使用者在事务数据库 T 中指定的最小支持的元素集称为频繁元素集。Apriori 算法的主要步骤如图 2-15。

算法首先在候选集中找到并计算支持度 $support(U)$ ，通过计算得到的支持度，筛选出低于阈值的候选集合 1，得到频繁集合 1。将频繁集合 1 与候选集合 2 进行计算支持度，再次筛选得到频繁集合 2。若指针指向集合为空，则计算支持度完成，否则返回候选集中继续计算支持度，直到遍历完成。

2.5.2 基于回归算法的异构数据挖掘技术

回归在统计上定义为：一种统计分析方法，用于研究随机参数对另一组参数的依赖性。其中，线性回归法是一种统计分析研究方式，在数学统计中使用回归分析法来确定两个或多个参数之间的数量关系。不仅如此，当自变量参数是固定数值而因变量参数是随机数值时，分析它们之间的关系称为回归分析。当两者都是随机参数时，称为相关分析。回归分析存在在研究领域中的技术面向不同方面的差异是自变量的数量，因变量的类型和回归拟合描线。线性回归的方式是通过暂定原方程为线性方程，若多个参数则变化为多变量的线性方程。例如领域部门的设备占用地和设备个数是自变量 x ， x_1 是设备占用地， x_2 是设备个数；设备的维护成本是因变量 y ，我们使用 $h(x)$ 表示 y 。假设设备占用地，设备个数和设备的维护成本是线性的。他们满足公式：

$$h(x) = h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 \quad (2-17)$$

上述公式中的为 θ 参数,也称为权重,可以理解为 x_1 和 x_2 对 $h(x)$ 的影响度。对这个公式稍作变化就是:

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x \quad (2-18)$$

公式中 θ 和 x 是向量, n 是样本数。假如我们依据这个公式来预测 $h(x)$,公式中的 x 是我们已知的,然而 θ 的取值却不知道,只要我们把 θ 的取值求解出来,我们就可以依据这个公式来做预测。

2.6 异构数据传输技术

2.6.1 基于 XML 的异构数据传输技术

可扩展的标记语言不同于 Oracle 以及 SQL Server 等数据库结构方法。这些数据库管理软件会支持更多、功能更加优异的数据存储管理技术,能够达到更高的用户需求,实现更优的性能。可扩展标记语言仅存储信息资源。在具体场景实践中,可扩展标记语言表达数据的方式能够更轻量级,不同于数据库传输整个数据表的方式。

可扩展标记语言有以下特性:

(1) 自我描述特性。可扩展标记语言中元素的树状结构可以通过语法规则的语义标识定义。同时,树状的节点每一层也解释了元素的描述意义。这使数据冲突发现和处理变得更加容易。

(2) 平台无关特性。可扩展标记语言文件保存为字符流形式,可以通过任何语言进行读取,因此,可以将该文件可在各类平台应用环境中传输以及访问。

(3) 可以表示异构程度不同的数据。XML 支持领域应用中所涉及到的结构化的数据表示形式,同时也便于在进行领域异构数据之间的修改查询等操作。例如,用户可以标记以下内容:普通文档,结构化字段表,操作对象,数据库基础操作历史记录,网络上所有可用资源之间的链接。XML 可以使用一致性的高可用资源模型来表述数据集成中来自各个应用场景的数据源信息,从而避免了大量的不同场景之间的异构数据源中程序环境和不一致的资源。

(4) 可扩展特性。XML 可以添加新的数据字段,而不会损坏当前的架构。程序服务节点使用 XML 来建模所有资源信息。如果要更改资源模型,则只需要更改数据资源模型定义,例如 DTD、XML Schema 等资源模型定义格式,而不需要重新对当前项目编码。

2.6.2 基于 JSON 的异构数据传输技术

随着 WEB 技术的飞速发展,人们对数据传输格式的需求逐步提高。作为一种轻型数据传输格式,JSON(Java Script Object Notation)已逐步出现在当前网络技术中。JSON 逐步大面积运用,对 JSON 的开发深入使用日渐增加。JSON 在网络应用程序中具有许多优点,例如传输速度快,多种语言和易于编写,因此现在在应用研发中被广泛使用。

(1) 可读性: JSON 和 XML 的可读性十分出色。一方面是简单的语句,另一方面是标准化的标签格式。

(2) 可扩展性: XML 本质上拥有可扩展能力,JSON 也是如此。但是,JSON 是 Javascript 的父级格式,并且可以存放 Javascript 组合对象,这具有 xml 所无法相比的优点。

(3) 编码难度: JSON 如同 XML 一样,在实际场景使用中具有丰富的辅助开发工具,例如 Dom4j、Dom、SAX 等。但在无辅助开发工具的情况下,由于 XML 文件需要大量的结构字符,因此 XML 的编写难度要远大于 JSON。

(4) 解码难度: XML 的解析方式有两种:一种是通过文本结构进行解码,即,一组标识符扫描父元素建立各个根节点元素。但是如果在开发编码之前知道文本的组织方式并且在该场景中无法进行高可用的封装,则可以使用这种方法。第二种方式是在相应业务场景下,通过找到 XML 文本扫描所有节点以及子节点。这可以通过程序调用自身的编程技巧来实现,但是解码后的数据仍然采用不相同的组织结构形式,并且通常无法满足预定义的要求。而 JSON 可高度自定义解码规范,大多语言都可对 JSON 字符串进行解析,还原数据对象。

2.7 本章小结

本章为复杂装备数据分析技术整体流程所涉及到的所有关键技术介绍。针对复杂装备数据分析研究中的问题,分析指出该领域的主要问题是异构数据动态管理和异构数据集成,本文将重点研究以及解决。后三个技术点介绍当前数据清洗、数据挖掘、数据传输常用的方法,选取与复杂装备场景相符的方法。本文异构数据清洗技术使用基于固定值的异构数据清洗技术,数据挖掘使用基于回归算法的机构数据挖掘技术,异构数据传输使用基于 JSON 的异构数据传输技术。。

第三章 复杂装备异构数据动态管理研究

3.1 引言

根据复杂装备异构数据管理的现状，在第二章中列举出几种常见的数据动态管理方法，但其中，性能较为优秀的基于响应的数据动态管理仍未能更好地解决复杂装备数据大量请求的技术问题。与此同时，点对点的权限与用户的关系导致数据管理冗余复杂。围绕这两大技术问题，本章首先根据复杂装备异构数据动态管理的特征，以异步消息队列为核心突破点，研究能在复杂装备异构数据大量的异构数据请求下，稳定处理请求且响应时间较低的方法；研究在复杂的异构数据权限与用户的关系中，以 RBAC 为基础权限模型，拓展组别与属性的概念，达到更便捷的权限管理模式。以上两个技术研究部分的技术框图，如图 3-1，图 3-2。

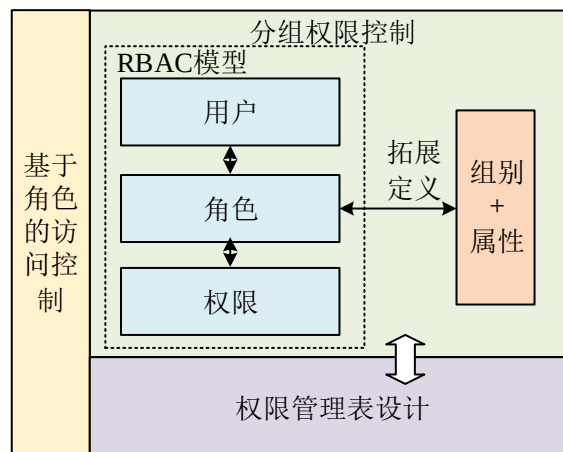


图 3-1 基于角色的访问控制技术框图

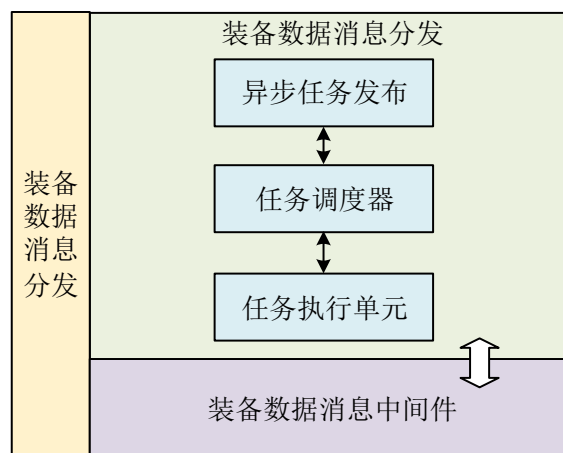


图 3-2 装备数据消息分发技术框图

3.2 基于异步消息队列的装备资源管理技术

本节主要说明基于异步消息队列的装备资源管理技术由两部分构成，装备数据的访问控制以及异步消息队列分发。研究以上两部分设计过程中需要使用到的相关技术以及架构模型，包括角色访问控制模型、Celery 异步消息队列架构等。

3.2.1 基于角色的访问控制

3.2.1.1 分组权限控制

目前，在复杂装备数据管理中有着杂乱的权限赋予关系，而在装备资源管理中权限管理还没有统一的方法。简单的，如使用硬编码划分权限管理的形式，在大多数情况下，使用修改代码逻辑的方式修改用户权限，以代码分支等形式将此逻辑与业务代码耦合，这使得权限的修改异常困难。在数据权限管理中使用中间权限组件，即规则引擎，通过第三方的验证以及权限分配，管理复杂装备领域数据资源的权限信息，部分系统权限是更为复杂的点对点的映射，如图 3-3。

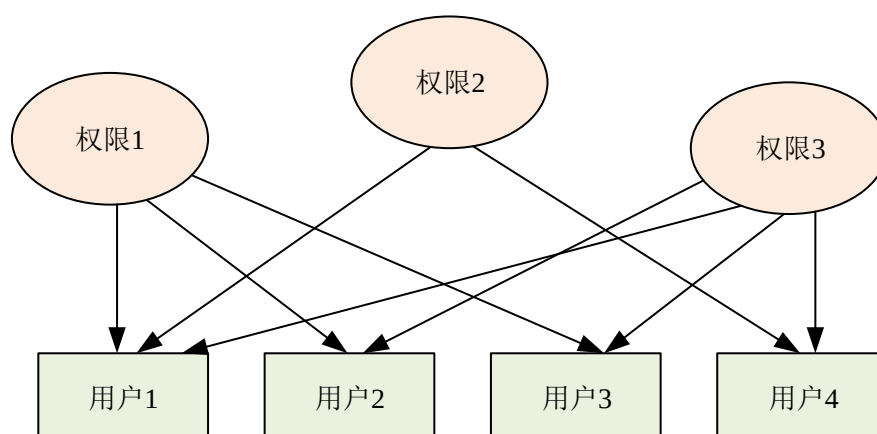


图 3-3 点对点权限映射

权限划分关系通过权限规则文本进行存放，规则执行组件用于解析规则；或使用框架中包含的第三方权限控制管理组件，例如 Spring Security、IBM Tivoli Access Manager，使用框架的内部组件通过注释控件执行权限，以上方法缺陷在于，权限规范不能够定制符合复杂装备数据场景。

因此，复杂装备数据分析的资源动态管理中，装备权限所依据的模型参考的是市场中较为主流的 RBAC，即基于角色的访问控制(Role-Based Access Control)。在抽象出来的权限模型中，权限赋予的对象不再是用户，而是用户通过设计好的映射模型，即角色。通过映射模型构建用户与权限之间的联系，如图 3-4。

用户通过标记指定成为相应的角色，用户将根据角色映射到所对应的权限集

合继承权限。通过角色层即可大大简化装备数据权限的管理。使用该模型能够将错综复杂的权限与用户的关系转变为层级的映射。因此，权限分配到角色层，用户通过对角色层的映射获取相应权限，清晰的映射关系使得装备权限关系管理简便。

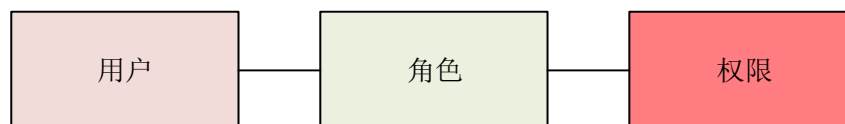


图 3-4 RBAC 关系图

核心原理是：分配权限时，不会将特定用户与各种权限建立关系，而是通过插入中间结构，即角色层，隔离两者之间的具体联系。间接地通过角色层对用户层和权限层之间构建关系，各个角色都关联着不同的权限。当用户划分指定的角色后，该用户便拥有该角色所关联的权限。加入角色层的优势在于，无需点对点地对用户和权限进行映射，只需用户指定为相应的角色，即可分配权限，简化权限映射过程。

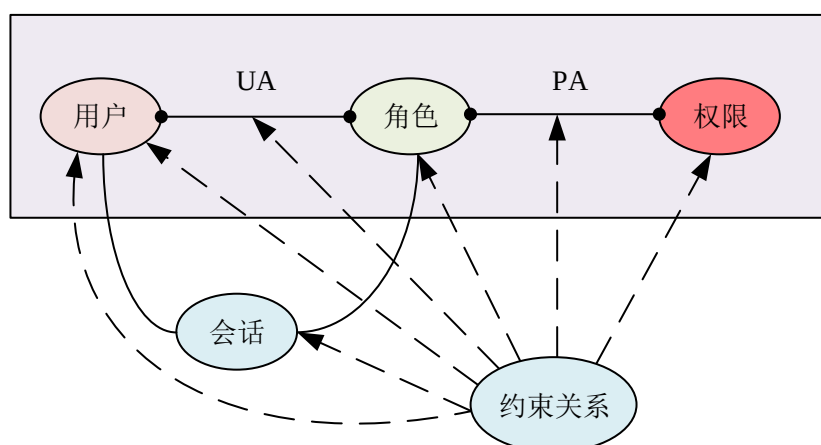


图 3-5 RBAC 模型

如图 3-5 所示，RBAC 模型包括：权限赋予最终的对象即用户(Users)、权限划分的规则即角色(Roles)、权限(Permission)、约束关系(Constraints)和一组会话(Session)。

用户是可以单独发出响应请求获取系统相应数据以及网络中指定数据的单独个体。

角色是组织或任务执行中的职位或任务执行人。表示着职责的划分以及一组数据资源的访问认证。

权限是计算机系统中，通过安全规则等，能够被特定用户访问到的资源映射规则。权限的性质取决于系统实现的详细信息。

图中虚线表示约束关系(Constraints)包括用户之间的约束、用户分配角色时的约束 UA ，通过对各个部分之间建立联系通道，还可以管理控制，角色之间的约束、分配权限 PA 时的约束以及约束角色之间的联系等，以便更好地对权限以及用户进行映射。在图 3-5 中， UA 线段指向的两端代表用户层和角色层之间的互相连接的关系，权限分配线段链接的两部分代表角色层和权限层之间的关系。

会话是用户到不同角色的关联通道。当系统接收到响应后，通过会话建立用户之间的链接，解析响应请求，将用户加入到指定的角色中。如图 3-5 所示，会话向用户以及角色建立连接通道，能够将用户信息传递到角色层进行角色分配。

由于复杂装备数据具有动态以及复杂多样的特点，所以，直接将角色层角色数量、类别固定是不可取的。因此，本文的复杂装备异构数据研究是在 RBAC 的权限模型的基础上，角色的概念延伸为组别与属性，组别层内部的分组数量是可动态地增删，灵活地对权限进行划分，同时组别可设置管理员属性，同其他普通组别进行区分，如图 3-6。

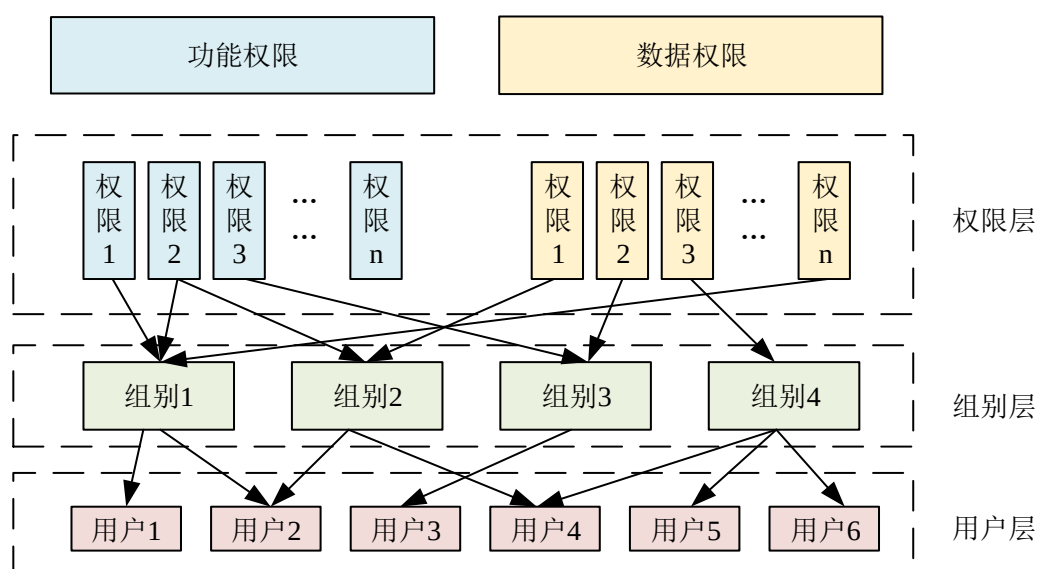


图 3-6 RBAC 映射模型

用户层和组别层的关系，它由用户和分组组成，两者之间的关系是多对多的。

权限层：包含资源的管理，如添加、删除、修改、查询所控制的对象，同时也包含授予角色或用户的功能权限，如允许对某些复杂装备数据报表进行编辑设计、允许查看都大屏展示等。该层所需要做的是存储各类权限信息，接收组别的权限加入请求，发送相应的权限信息至指定组别中。

组别层：存放各类权限信息形成一个数组，同时保留一个标志位属性，可拓展为普通管理员设置的标志。另一个数组为用户集合数组，存放着权限范围指向该组

别的用户信息。

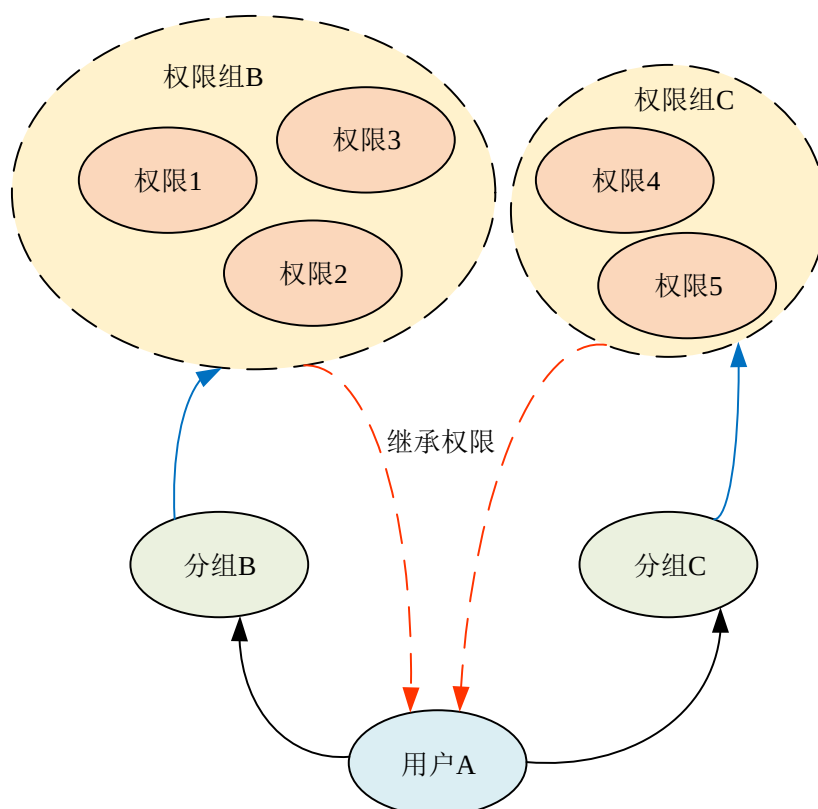


图 3-7 RBAC 模型继承特性

如图 3-7 中，蓝色箭头作为分组的权限约束，红色箭头表示权限集合继承到相应的用户中。组别 *B* 与组别 *C* 在复杂装备数据场景中可以作为作战管理组群，而在其他场景中可设定为其他类型的权限层次。用户层中用户可以属于单个或者多个组别，组别中的权限分配可以自由设置，不同于硬编码的方式设置权限，采用组别的方式存放权限标志信息更为灵活。

组内用户 *A* 所获取到的权限，则是由分组决定，不同组别保存着各类权限信息，当某一用户处于组别 *B* 中，在用户获取资源前，将获取组别权限标志信息，以验证该用户是否能够获取资源。若用户处于多个组别中，则将从头遍历用户所在的所有组别，以获取所有权限配置信息。继承特性：这意味着这些权限通过分组的权限使用户拥有，而不是直接设定用户的个人权限，如图 3-7。

若用户用集合表示 $U_i, i \in \{1, 2, 3, \dots, I\}$ ，分组集合表示 $G_j, j \in \{1, 2, 3, \dots, J\}$ ，权限表示集合 $AC_k, k \in \{1, 2, 3, \dots, K\}$ 。如图 3-8 所示，假设用户 U_1 属于分组 G_2 ，分组 G_3 ，则在默认情况下，将为用户 U_1 分配分组 G_2 和分组 G_3 的所有权限 $AC_2 \cup AC_3$ ，即，初始配置下，用户 U_1 拥有所有在分组中的权限。如果用户属于多个分组，则用户权限是这些分组所分配到的权限集合， $U_1 \in G_2 \cup G_3 \Rightarrow U_1 \in AG_2 \cup AG_3$ 。

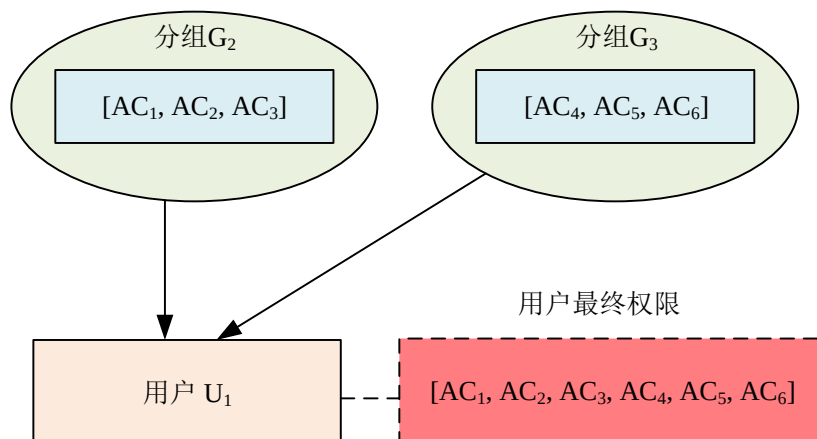


图 3-8 权限合并

如果用户存在于多个分组中，但是分组之间存在权限冲突，例如，权限组 *B*、*C* 中的权限 4 对于相同的资源进行相反的操作，即分组的许可“允许”，而另一个分组的许可为“不允许”，如图 3-9。

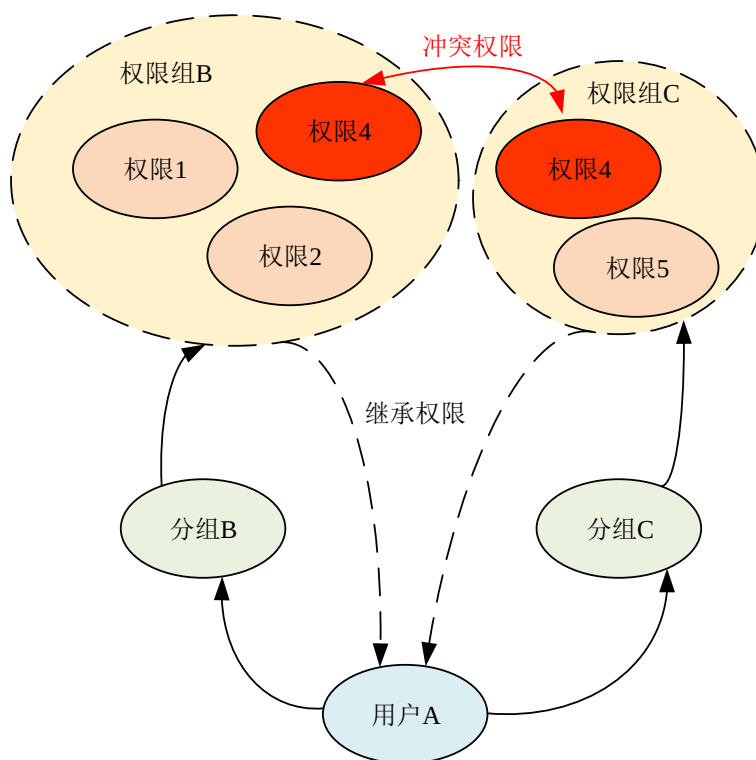


图 3-9 权限冲突

高优先级分组：所谓的优先级，由用户为此分组拥有，按照顺序，相同分组在不同用户中可能具有不同的优先级，可通过分组加入的顺序也可使用权重函数 $Weights(AC_k), k \in \{1, 2, 3, \dots, K\}$ 进行判断，定义 $\neg AC_k, k \in \{1, 2, 3, \dots, K\}$ 为原权限取反，

通过判定权限优先级的方式，使得权限优先级大的覆盖权限优先级小的以解决权限冲突问题，即： $U_1 \in G_2 \cup G_3, \text{Weights}(AC_2) > \text{Weights}(!AC_2)$ ，如图 3-10。

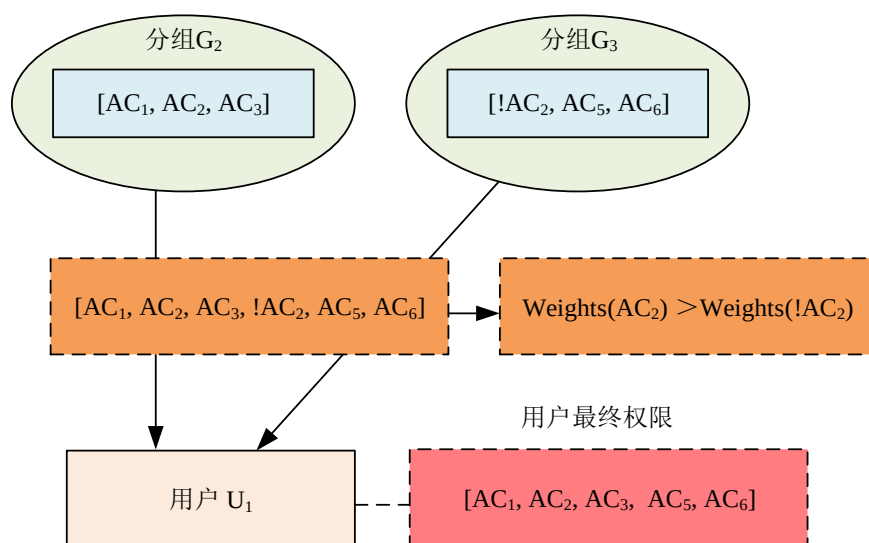


图 3-10 权限冲突判定

除了分组的权限外，还可以为分组添加功能级权限属性。如为分组 B 添加普通管理员属性，则分组内成员在继承分组所包含权限的情况下，同时统一的设定为拥有普通管理员的权限，具有相应的功能级权限设置，如图 3-11。例如，我们可以为用户设置某些资源可设置对外开放的权限。

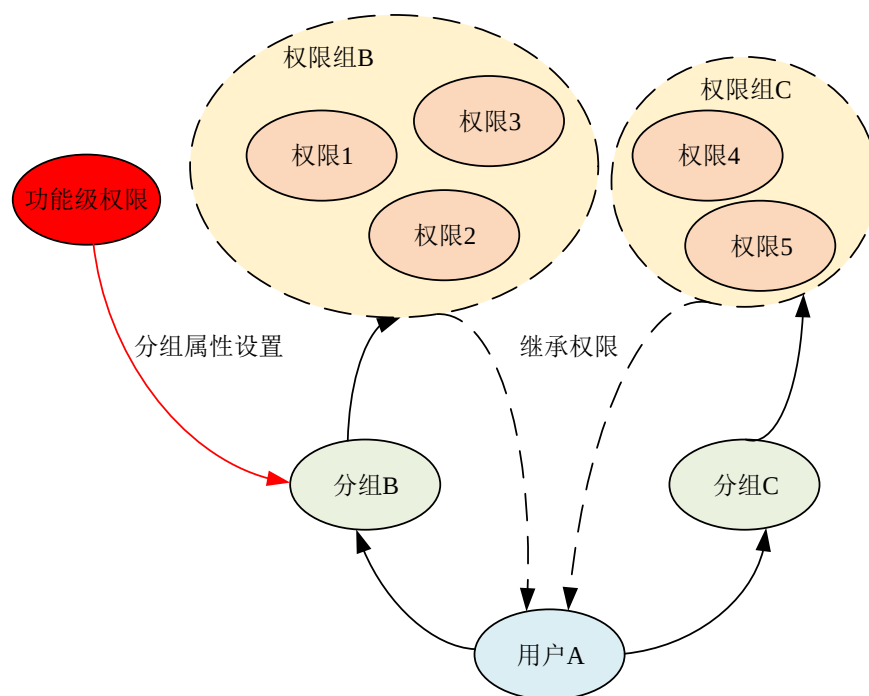


图 3-11 分组属性标记

3.2.1.2 权限管理表设计

由上一小节可知，需要设计用户表、分组表、权限表。通过各个表中的外键，对三张表进行联合，从而实现用户、分组、权限的映射关系，如图 3-12。

用户表(User)，Id 作为自身的 unique id 用作索引标识，用户本身的属性包括名称、邮件、所属部门等，Group_ids 作为外键，将用户表(User)与分组表(Group)进行外连接，表示两个表关系之间的相关联系。

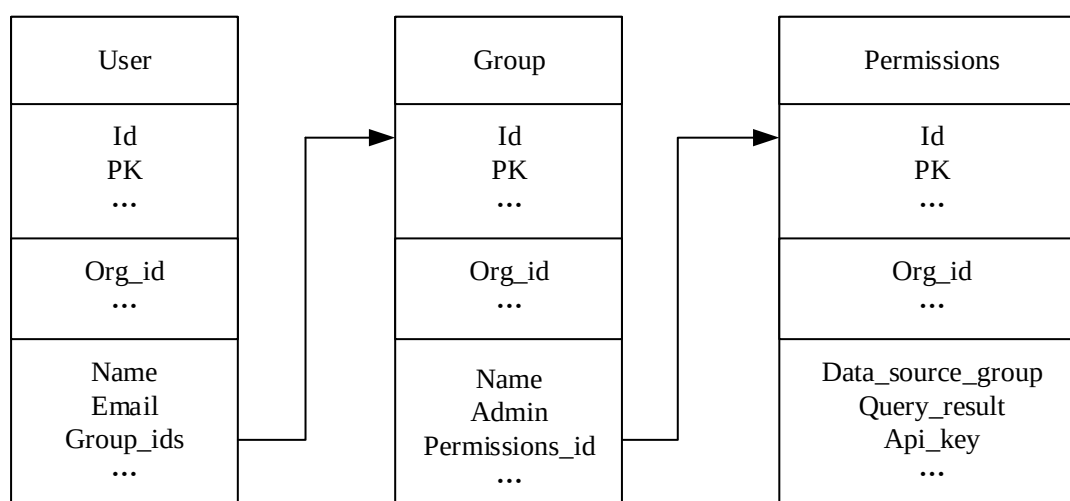


图 3-12 权限管理数据库表结构

分组表(Group)，Id 作为自身的 unique id 用作索引标识，分组本身的属性包括，名称、组织 id 等，其中 Permissions_id 作为关联权限表(Permissions)的外键，Admin 属性表示该分组具有普通管理员的功能级操作权限，可进行数据资源的分享发布操作。

权限表(Permissions)，Id 作为自身的 unique id 用作索引标识，权限本身的属性包括，组织编号等。Data_source_group、Query_result、Api_key 等字段，作为标识该组是否有相应功能级或者数据级的权限。

使用外键将三者联合起来，作为映射关系模型的实现，保持模型内容的统一性，准确表达权限模型，关键是控制各个表中保存在外键列表内的资源映射关系，使得多张表实现关联关系。

3.2.2 装备数据消息分发

在复杂装备数据分析的生命周期中，一个时间段往往会有多种请求操作。如，高并发的查询任务、多个装备集合同时查询或者多个装备消息任务发布等。定时任

务,定时地对数据集进行更新或者定时清理过期消息数据等。异步任务,一些耗时较长的操作,比如 I/O 操作,网络请求等。根据以上场景,若使用同步队列或者顺序响应请求,则需要等待网络、I/O、任务操作所需要的时间,造成大量任务的拥塞。因此,适应复杂装备数据分析的场景,使用异步消息队列构建动态资源管理。

异步消息任务队列通常在计算机结构中用作在新建线程完成相应系统任务或计算机之间分配工作的机制。异步消息任务队列的工作流程中的输入称为任务的消息数据。开启一条特定使用处理任务消息的工作进程(Worker)来持续监听异步消息任务队列并执行新创建的任务。

Celery 是基于分布式任务队列的框架,适用于复杂装备数据大量处理的场景,使用异步消息队列的方法,获取各领域场景的系统、进程、线程中的复杂装备任务消息进行调度。同时,该框架可使用多种语言进行调用通讯,对异步任务进行分发,同时可以执行一定程度的进行异步任务(Async Task)和定时任务(Crontab)。其框架组成结构见下图 3-13。

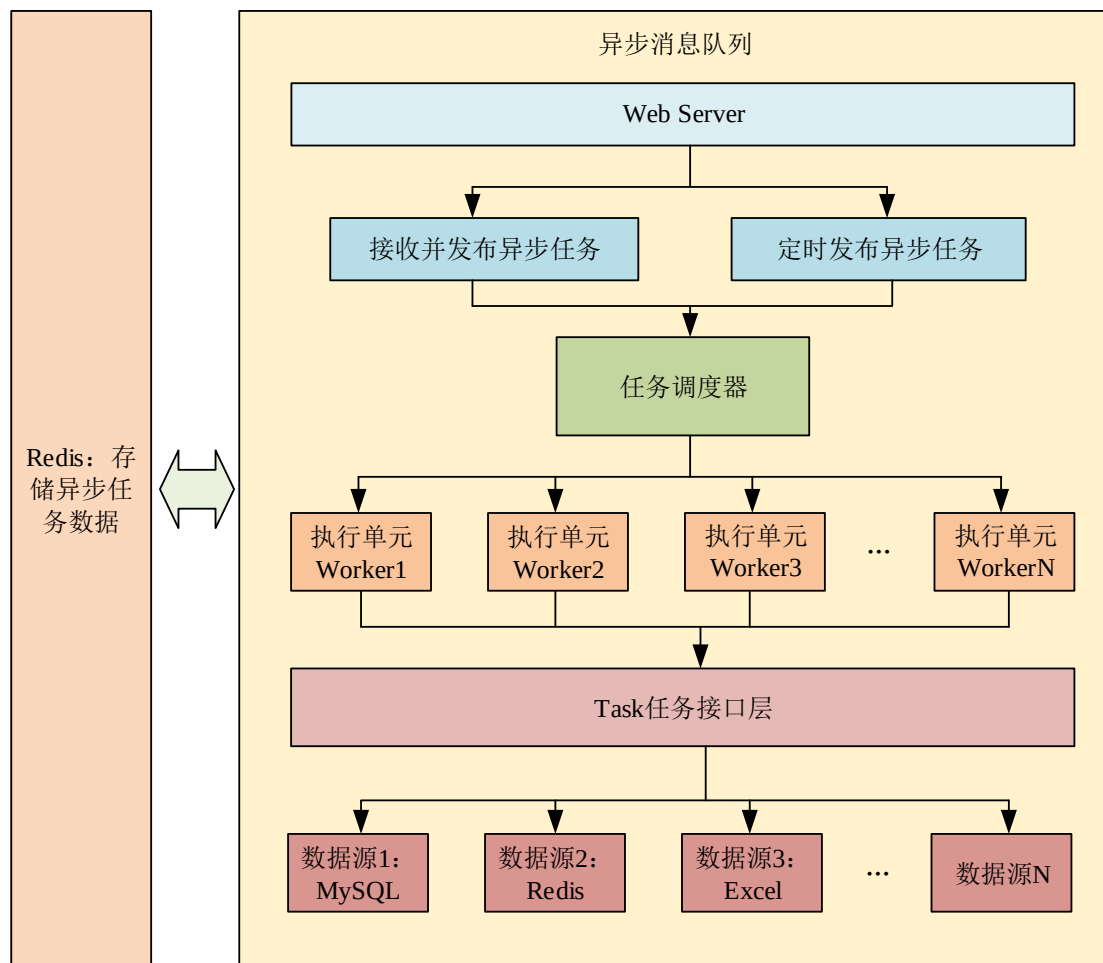


图 3-13 装备数据消息分发架构设计

异步消息队列组件包括:

1、任务发布者(Producer): 接收响应请求, 创建 Tasks 任务消息, 传入任务队列执行装备消息的发布。

2、任务调度器(Broker): 保存着 Tasks 任务消息, 当大量任务消息涌入, 调度器需要有足够的空间存放以及一定的任务丢弃机制。使得执行单元能够获取到任务消息进行处理。

3、执行单元(Worker): 用于执行 Tasks, 也就是实际调用系统在 Tasks 中定义的函数。

4、任务消息(Tasks): 系统设计的执行方法函数调用, 保证获取到的任务消息能够被处理消费, 提供给外部处理函数的表示。

5、消息中间件存储(Backend): 作为装备数据消息中间件, 把执行 Tasks 返回的结果进行存储, 以供用户查看或调用。

3.2.2.1 异步任务发布

由于在复杂装备领域中, 存在大量任务需要处理的情况, 但若是串行执行任务并返回结果, 则会造成系统阻塞, 后续的任务需要等待剩余任务完成后才可执行。这样导致系统运作处理效率低, 同时浪费系统资源。因此, 使用异步的方式对系统任务进行安排, 任务先交给执行单元处理, 等到执行单元返回结果时系统才调用该方法获取结果, 大大利用系统资源提高效率。若系统故障导致执行单元暂停任务执行, 待系统恢复正常工作, 执行单元会利用先前任务保留的缓存信息继续执行任务, 直到任务返回结果。

复杂装备数据的异步任务通过异步任务发布者(Celery producer)进行发布, 首先获取网络中的任务请求, 使用 RESTful 网络应用程序的编写规范以及指定的开发模式, 设计 GET/POST 任务请求。基于业内主流的 HTTP 网络协议, 能够轻易地运用 XML 规范编写或 JSON 规范编写请求格式。通过解析 JSON 格式字段, 获取请求资源 ID 或其他参数。

通过引入执行单元(Worker)中定义的函数, 将任务请求映射到各个函数体中。任务发布前, 获取到注册成功的函数, 带入请求参数进行正式的异步任务发送。当任务发布成功后, 将写入 Redis 存储中, 保存消息信息, 以保证消息在任务队列中丢失后, 仍可以对该消息进行再次发布, 防止消息遗失; 若任务发布时出现系统故障或程序异常, 则再次发起函数调用, 重新进行任务发布。整体流程如图 3-14。

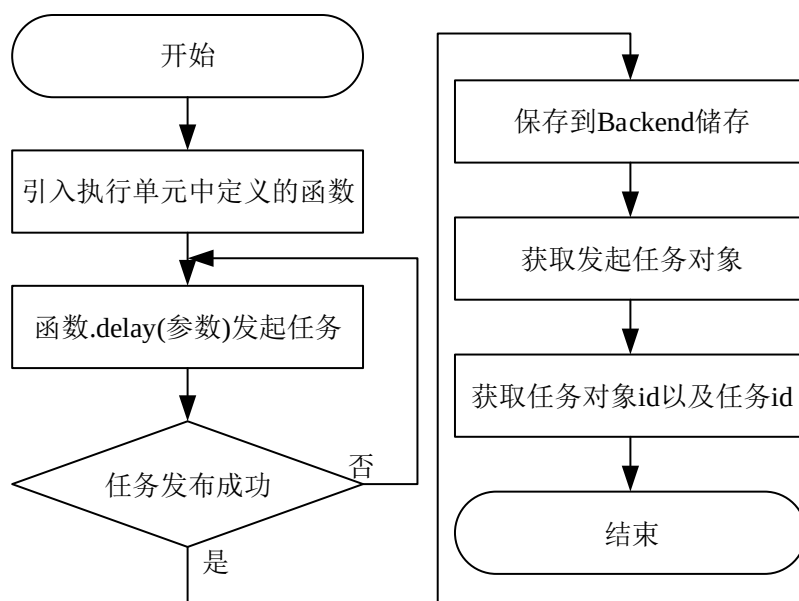


图 3-14 异步任务发布流程

其中，引入执行单元中定义的函数，首先初始化 Celery 对象并读取项目配置文件，获取相应函数列表。初始化完成后，Celery 通过设定的 `BROKER_URL`，配置消息队列中间件，提供任务消息、中间件等数据的存放数据库地址。找到相应存放中间件消息数据的 Redis 数据库地址，数据库中另一部分作为数据查询的暂存，优化数据查询效率。

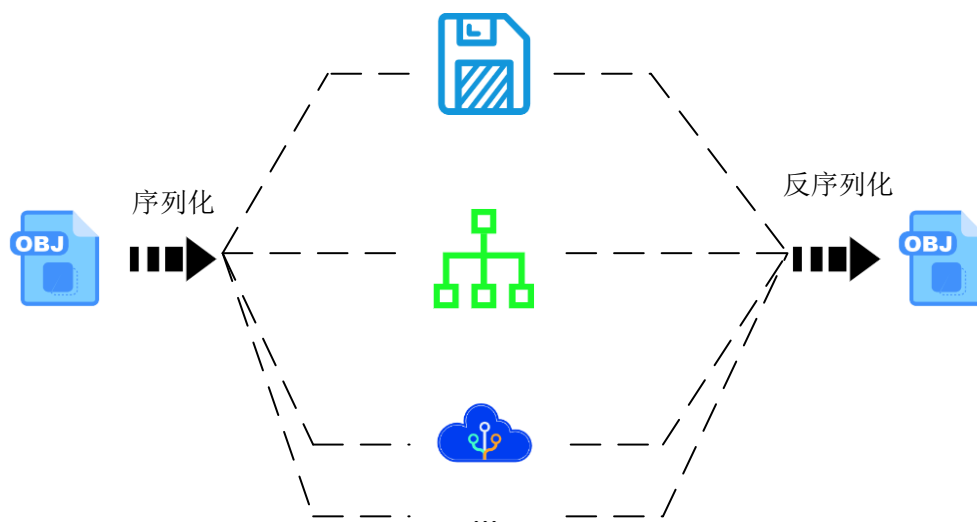


图 3-15 序列化和反序列化

如图 3-15，由于在传输和保存系统以及用户定义的数据对象时，需要确保数据对象的完整性以及通过网络传输的可行性。因此，需要将部分指定对象转换为有

序的字节流字符串，才能够利用网络协议以及规范传输或者保存至特定节点或本地路径中。

CELERY_TASK_SERIALIZER 以及 CELERY_RESULT_SERIALIZER 则是配置序列化的方式，CELERY_ACCEPT_CONTENT 为配置接收序列化封装的使能字段，即反序列化的解析方式。系统调用该字符流时，需要进行反序列化，根据保存的特定对象序列方式和表述资源存储在字节流中，使用相应的反序列化技术重建保存的特定对象，以保证特定数据对象解析的完整性。

考虑到任务消息在任务调度器中存在消息堆积的情况，在消息队列参数配置过程中需要针对特定场景设置相应的任务消息失效时间，以清空消息中间件中无效的任务消息数据。使用 CELERY_TASK_RESULT_EXPIRES 对异步消息队列设置消息过期时间。

当 Celery 读取配置，初始化完成，与所有数据库进行连接后，即可接受外部接受的任务请求。接受到相应的任务请求后，调用函数的 delay 方法，传入相应任务的参数，等待任务发布的成功，若发布失败，则重新对该任务进行发布。发布任务成功后，将任务信息存放到 Backend 进行存储，等待系统有序获取任务信息并调用相应任务方法执行任务请求。

3.2.2.2 任务调度器

系统发布任务后，若是直接将任务交给执行单元执行，当执行处理速度小于任务消息发布速度，任务依旧阻塞在执行单元中，则无法实现大量任务异步执行的优势。因此，需要一个任务调度器，管理发布后的任务，通过任务的标题、路由信息，将各类任务分发到相应的任务缓存队列中，等待执行单元调用，以实现异步任务分发。

Celery 通过消息机制进行通信，通常使用任务调度器(Broker)作为用户端消息和执行单元(Worker)的中间消息存储。任务调度器，使用一种任务队列的工作模式，存放着接收到的任务消息，开启一个监控线程对任务消息队列进行监测，一旦出现新的任务消息，将接受该消息信息。当接收到用户端发送的响应时，系统将该任务信息发送至任务调度器(Broker)。调度器将消息传递给一个空闲的执行单元(Worker)，执行单元(Worker)获取并执行任务调度器(Broker)分发的任务，最后返回结果集合。

当异步任务发布触发，系统将任务消息发送至任务调度器中，通过任务消息中的标题、PID 等信息，将任务分配到交换中心进行注册。首先进入 Exchanges 组存放相应的任务消息，通过“消息路由”的机制使 Exchange 对应任务标题相同的异

步队列。将装备任务消息存放到对应的异步队列中暂存，当执行单元组中存在空闲的执行单元，则会去异步消息队列中获取任务消息，并返回执行结果集合到系统，如图 3-16。

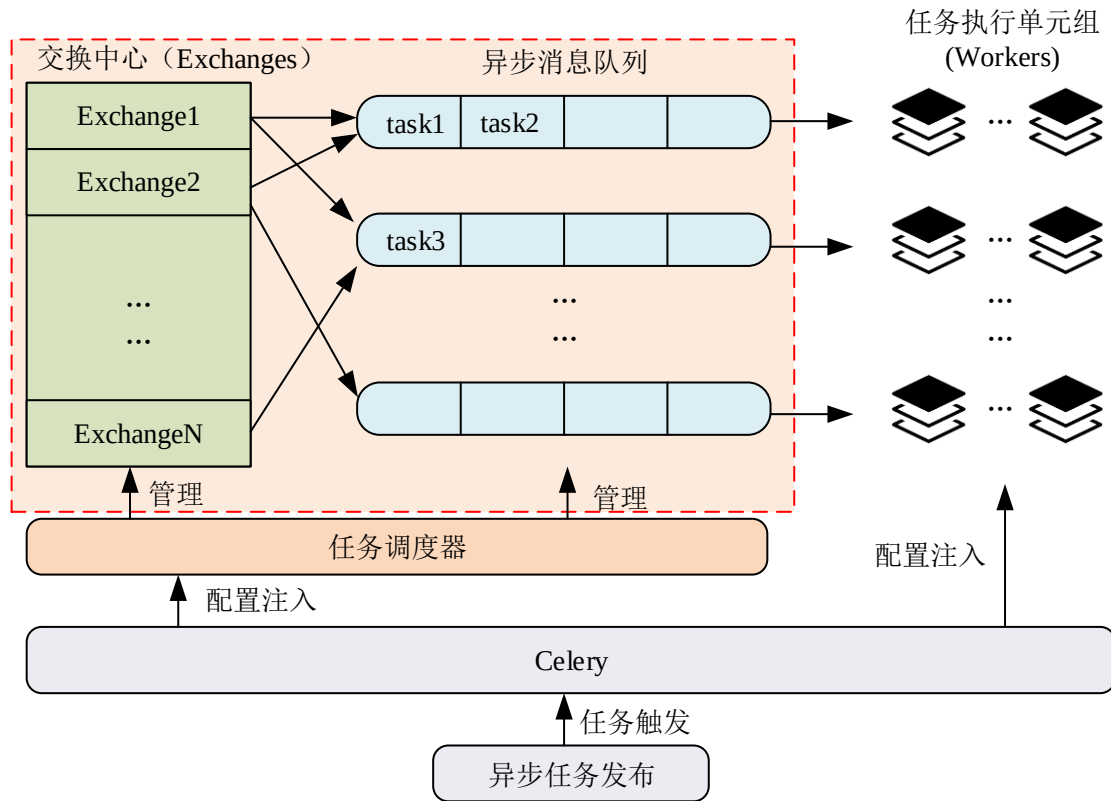


图 3-16 任务调度器组成架构

在本文研究中，任务调度器使用的消息交换的方式是直连类型(direct)，即任务名与路由键值一致。设定两类异步任务，查询任务以及定时任务。通过创建任务队列标题为 `queue`，队列类型为 `direct`，当异步任务发布时，系统将对应标题的任务通关任务调度器，发送到相应的异步消息队列进行存放。

当有空闲的任务执行单元时，将从异步消息队列中获取任务信息，调用相应的任务执行方法处理任务。

设置另一队列的标题为 `time_task`，类型为 `direct`，该队列将接收定时任务的信息发布，设置定时任务的周期，当触发定时任务后，定时任务信息将加入到相应的异步消息队列等待执行单元获取任务信息执行。

当一时间有多个装备任务消息 $Task_i, \{i \in [1, 2, 3, \dots, n]\}$ 发布时，若是同步阻塞的处理任务方式，当任务 $Task_k, k \leq n$ ，是一项高耗时的任务，则所有剩余任务消息将会阻塞在 $Task_k$ 处，而 $Task_k$ 与其他任务没有先后执行关系，这将会白白消耗大量时间。因此，使用异步队列的方式，通过任务调度器，将任务存放在队列里，由若干

执行单元取出任务消息，执行相应任务，返回结果。

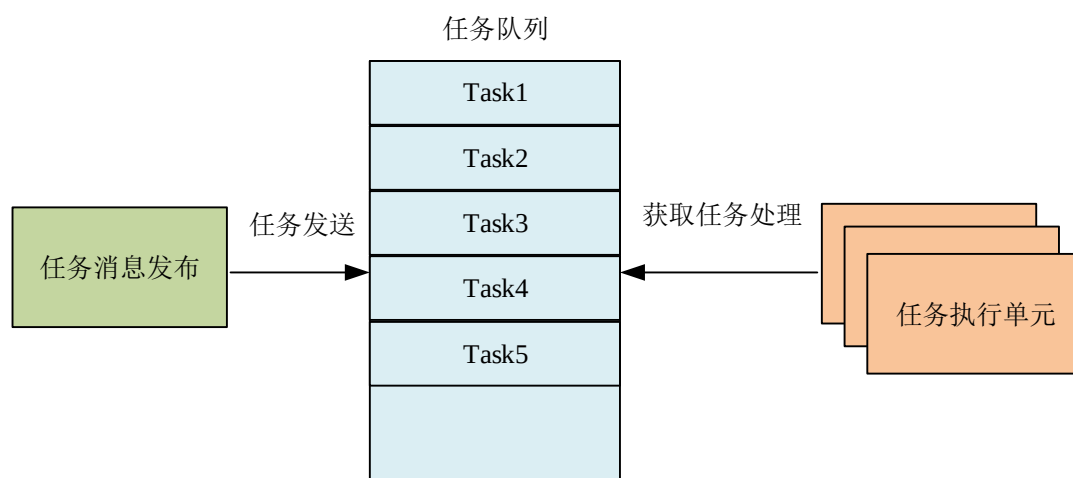


图 3-17 任务调度过程

如图 3-17，假设有一组任务消息发布， $Task_i, \{i \in [1, 2, 3, 4, 5]\}$ 。

$Task_1$ ：表里创建一条新记录。

$Task_2$ ：给该用户发送提示信息。

$Task_3$ ：给该用户初始化各种装备数据。

$Task_4$ ：查询某装备信息。

$Task_5$ ：执行定时更新任务。

部分任务请求响应时间很长，若等待该任务执行完成后，再对下一任务进行处理，则会导致系统拥塞，进而减少服务端处理请求的并发量。由分析可知， $Task_1$ 请求的响应内容只是任务 1 的结果，与后续任务无关。同时，创建一条新纪录也不必要在发出请求的时刻立即完成。因此，该任务无需阻塞在这一组任务消息的生命周期里完成，可做异步化处理，只需关心最后的结果是否完成该任务。并且在 $Task_1$ 异步处理的过程中， $Task_2$ 、 $Task_3$ 、 $Task_4$ 皆可分别开启各自的异步任务处理，而无需相互等待。

将任务 $Task_i, \{i \in [1, 2, 3, 4, 5]\}$ ，存放到任务队列中。步骤就变成了：异步处理 $Task_1$ 表里创建一条新记录；异步处理 $Task_2$ 、 $Task_3$ 、 $Task_4$ ；不用管 $Task_1$ 的处理，直接返回新建记录成功响应。大大的缩短了响应时间。

总的来说，在一个请求到达服务端时，Web 程序需要处理一组任务，然后把响应返回给客户端。但有时因为一组任务的某些执行响应时间太久，而且我们并不在意这些任务的执行顺序，只在意它们在有限时间内一定会被执行，即异步消息队列处理任务消息。

3.2.2.3 任务执行单元

Worker 是 Celery 框架在设计环节中支持对特定消息进行处理的任务执行单元, Worker 在分布式的系统节点中自动创建分配任务并发执行。如图 3-18 所示, 当任务调度器中的异步消息队列有任务消息时, 首先发送应答信号给任务执行单元, 若任务执行单元正常工作, 则会返回一次应答信号并返回任务执行单元 PID 号。否则, 将读取任务执行单元配置信息, 创建一个 Worker。当任务执行单元获得到任务调度器中的任务 id 后, 执行相应任务函数, 若执行成功, 则将任务结果保存到消息中间件中, 同时, 为了避免重复消费任务, 需要清除任务调度器中异步消息队列存放的任务信息。若执行失败, 则再次获取任务调度中的任务, 再次执行。

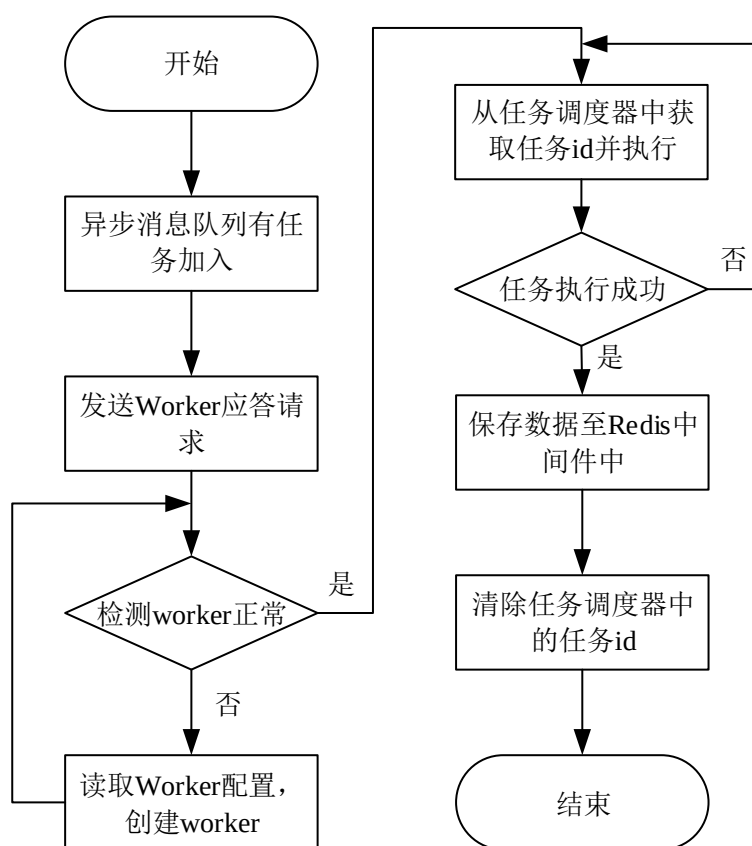


图 3-18 执行单元工作流程

当异步消息队列中加入了相应的任务信息, 此时任务调度器通过与任务执行单元的应答, 判断是否有空闲的执行单元可处理任务。同时, 获取任务执行单元的状态信息, 判断执行单元是否可用。

若执行单元异常不可用, 则重新读取执行单元初始化信息, 创建新的执行单元。若执行单元工作状态反馈正常, 则获取任务信息, 如: 任务状态、任务名称、执行

单元链接地址、异步队列标题名称以及创建执行单元的 PID 信息等。

通过以上配置，创建执行单元运行环境。同时，任务执行单元分为刷新数据、刷新查询表以及同步用户数据的定时任务。此时任务调度器将监听任务执行单元是否正常运行以及返回相应的结果集合，将结果集合存放到 Redis 中间件中。若异步消息队列发布的是定时任务，则执行单元还需获取定时周期(schedule)，通过任务执行单元的定时监听方法，定时订阅相应异步消息队列的任务并对任务进行处理。

3.2.3 装备数据消息存储

任务结果存储区用于存储消息任务执行的任务结果数据。Celery 能够采用多种方式对消息任务数据结果进行保存，包括 Redis、MongoDB 等。由于 Redis 易于部署，可用性高，查询性能良好并且自然幂等，因此本文使用 Redis 来存储消息任务执行的结果。

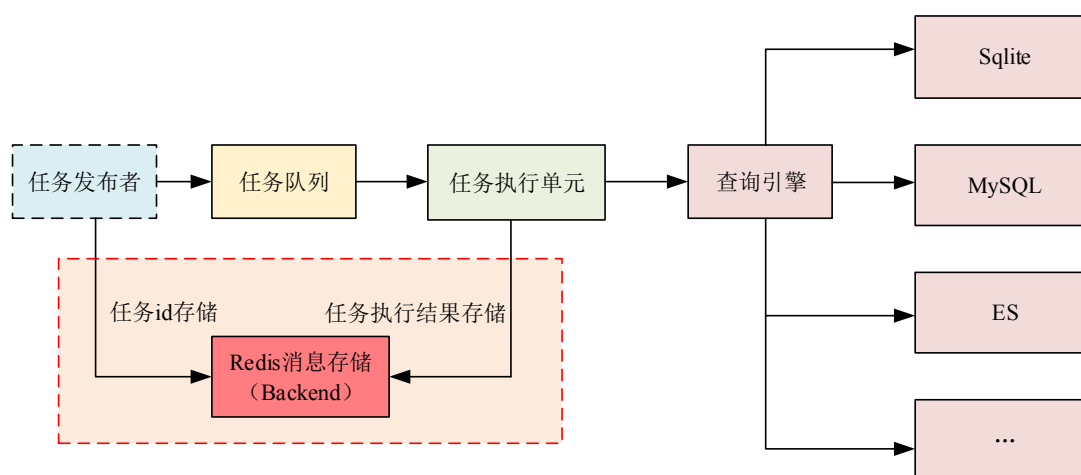


图 3-19 消息中间件存储结构

如图 3-19 所示，一般任务式系统的结构为“队列+存储”的模式，队列用于任务的传递，存储用于任务历史信息的记录，以便后续查询。当任务发布者将消息加入到任务队列中，会将该任务消息 Redis 消息存储。因此，任务发布者以及任务执行单元，需要将发布的任务信息以及任务执行结果存放。

3.3 性能比较

3.3.1 装备数据动态管理部分

硬编码形式的缺点非常明显。系统耦合牢固，子模块难以划分检测，导致系统

多模块调用的组件重复使用难度高；装备系统开发后期变更组件场景需求难度增大。使用规则引擎可以支持处理装备领域中常见的大多数问题，但是规则引擎并不适用于特定领域的权限资源等管理模式，因此对于更复杂的、更贴近复杂装备场景的权限管理，难以实现装备数据的权限访问需求。

将本文方法与对比点对点以及硬编码的方式进行对比，功能上的优势如表 3-1 所示，本文方法解决了复杂装备数据权限管理灵活添加的问题，降低了映射关系复杂度，减少数据冗余，提高权限划分的开放性。

表 3-1 权限管理定性比较

权限管理方式	灵活添加权限	权限划分局限	映射关系简单	复杂度低	数据冗余
点对点	√	√	×	×	×
硬编码	×	×	×	×	√
本文方法	√	×	√	√	×

由表 3-1 结果可得，本文将使用点对点的权限管理方式进行对比响应时间，可以看出本文方法在用户数较大的情况下，权限响应时间短于点对点的管理方式如图 3-20。

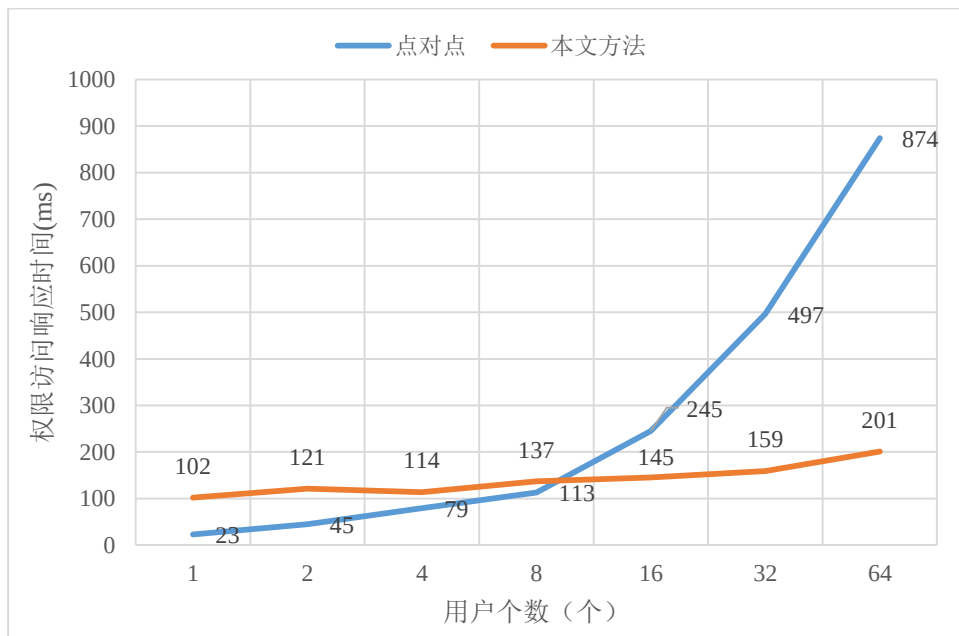


图 3-20 文本与点对点权限管理平均响应时间对比

本文方法通过加入中间分组层，很好地解决了数据权限映射关系复杂的问题，将复杂度从 $N \times N$ 降低为 $N+N$ ，使得平均响应时间在用户数量较大的情况下仍然较低。

3.3.2 装备数据分发部分

使用异步消息队列的装备数据分发优势在于：

(1) 异步化任务执行

一组装备任务消息无需在同一时刻全部处理完成，大量的请求响应，系统也不可能一瞬间全部处理。

因此，只要整体任务处理时间在允许的一个时间段完成即可。那么，我们可以设置不同数量的执行单元去异步处理任务，使得这些 Task 排队去执行响应的请求，从而优化整体的最大响应时间。

实验环境在虚拟机环境 4 核 4G 内存的硬件情况下。将本文方法：基于异步消息队列的动态管理与基于响应的动态管理方法进行对比。

在以上条件下，对比两种方式在各个量级以及执行单元数量不同的情况下，相对并发量下的 90%最大响应时间线的对比。其中相对并发量定义为：指在一个时间段内发生的请求量。90%最大响应时间线为所有响应时间中的 90%分位数，即 90%请求的最大响应时间在该数值以下。

首先对比，在执行单元数量为 2 的情况下，基于异步消息队列的异构数据任务将发布至消息队列中存放，开启 2 个执行单元进行接收任务的处理，每次响应请求一组复杂装备测试数据。

通过使用 JMeter 压力测试工具，用于对系统做并发请求的检测，通过对 JMeter 配置请求次数以及循环次数，设置测试相对并发量分别为 1、10、 10^2 、 10^3 、 10^4 、 10^5 、 10^6 ，测试结果如图 3-21。

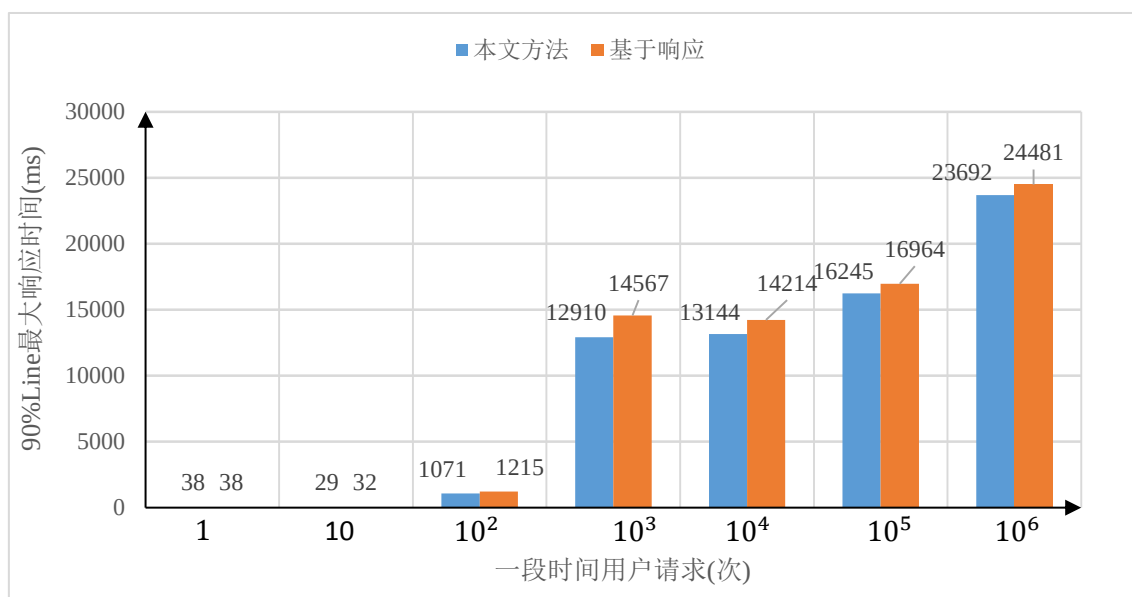


图 3-21 本文方法与基于响应的动态管理最大响应时间对比

由图 3-21 可知,在执行单元数量为 2 时,基于异步队列的动态管理的最大响应时间均小于等于基于响应的动态管理。对比本文研究方法与基于响应的动态管理,最大响应时间平均下降 7.6%,在相对并发量 10^2 - 10^3 之间,响应时间下降最大,随着相对并发量的逐步增大,由于单机性能的限制,响应时间的下降率逐步下降,如图 3-22。

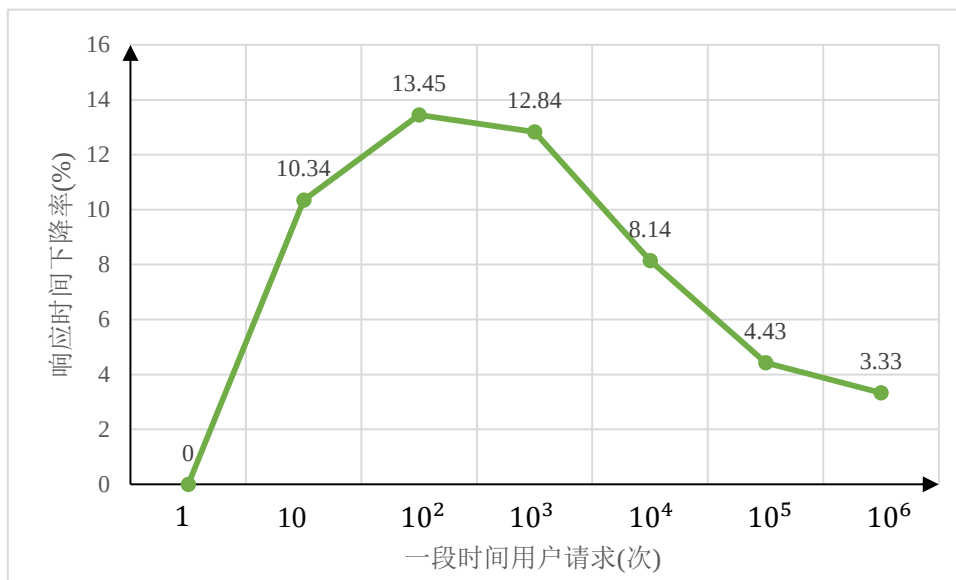


图 3-22 各相对请求量的最大响应时间下降率

同时,在高相对并发请求时,系统往往会因为大量任务处理而出现 CPU 占用率极高的情况,最终导致系统崩溃。而使用异步消息队列处理任务请求,系统 CPU 使用占有率稳步于 40%,如图 3-23。

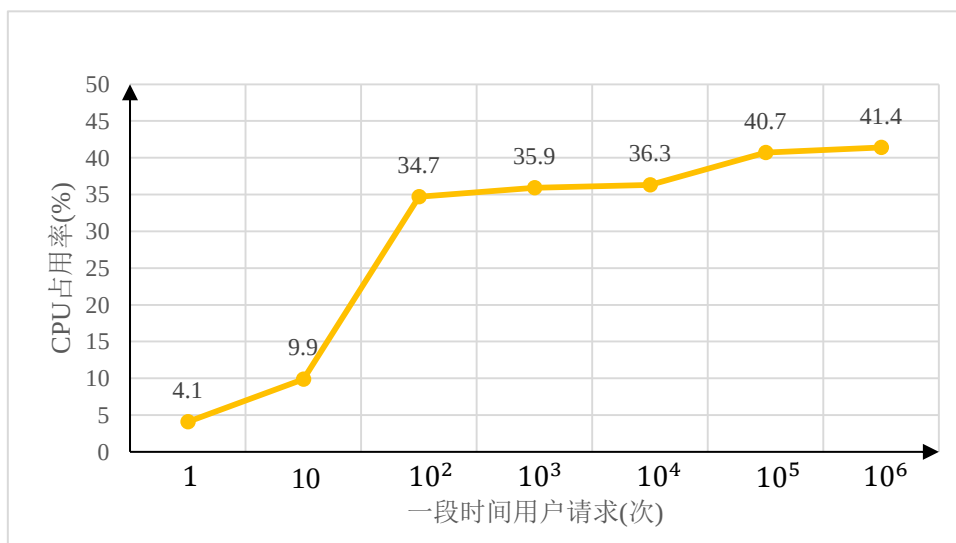


图 3-23 各相对请求量的 CPU 占用率

针对不同执行单元数量,做出以下实验对比。测试相对并发请求 10^4 、 10^5 、 10^6 , 在执行单元数 2、4、8、12 的情况下, 90%最大响应时间的对比, 如图 3-24、图 3-25、图 3-26。

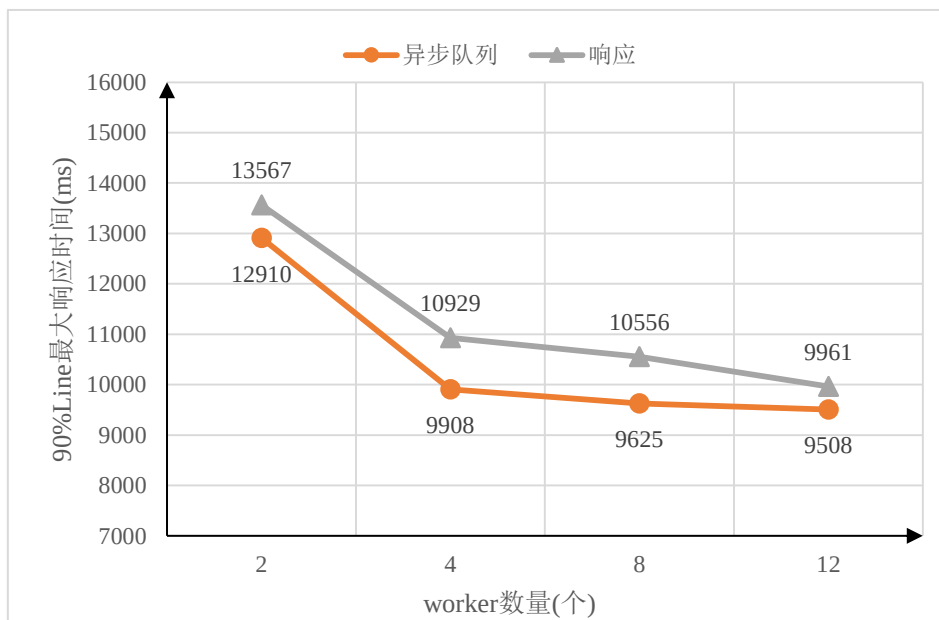


图 3-24 相对并发请求量为 10^4 的最大响应时间

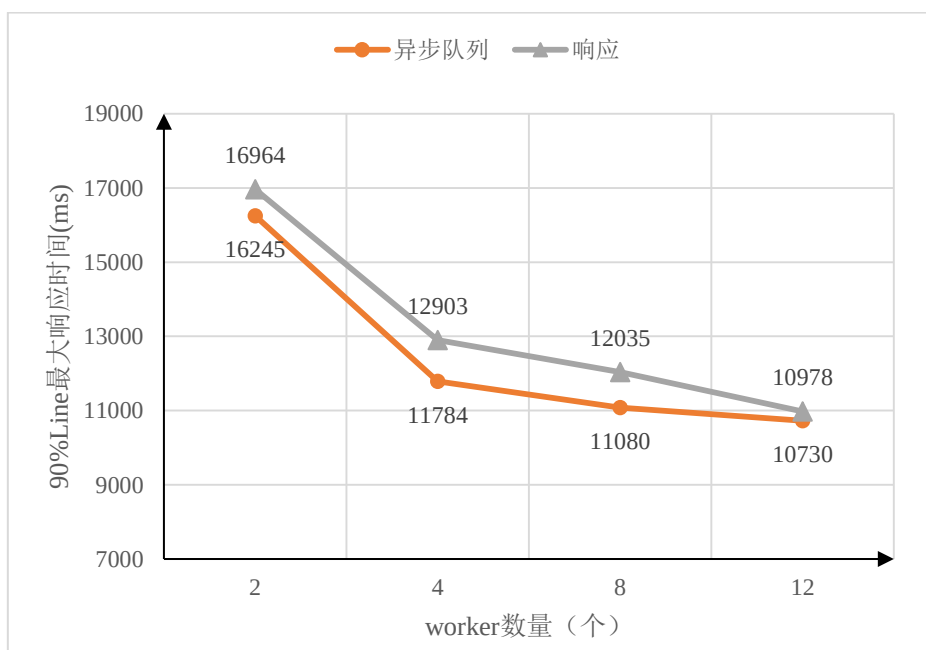


图 3-25 相对并发请求量为 10^5 的最大响应时间

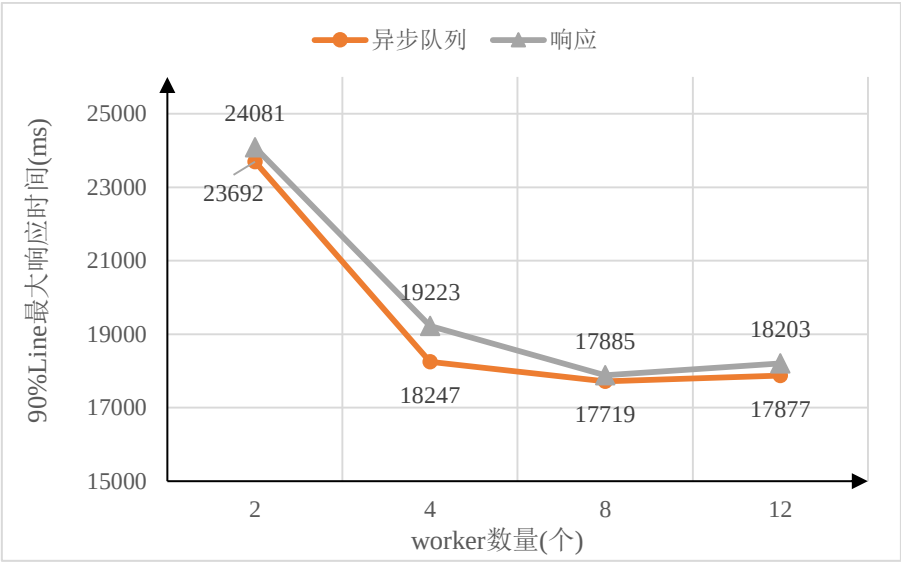


图 3-26 相对并发请求量为 10⁶ 的最大响应时间

由图 3-24、图 3-25、图 3-26 可知，在不同相对并发量下，随着执行单元数量的增加，最大响应时间也在不断减少。同时，不同相对并发量、不同执行单元数量下，基于异步队列的动态管理最大响应时间均低于基于响应的动态管理。同样，由于单机性能的限制，各个相对并发请求中，响应时间下降率随着相对并发量的上升而逐步下降。在执行单元数量为 4 个的情况下，响应时间下降率为最大值，此时，虚拟机硬件 4 核与执行单元 4 个保持一致，使得 CPU 调度执行单元效率最高，充分发挥 CPU 性能，如图 3-27。

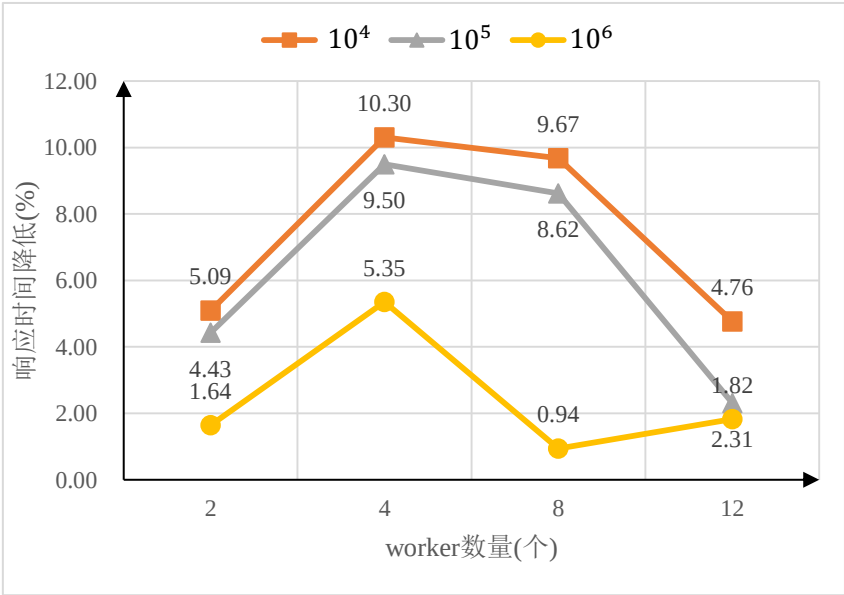


图 3-27 不同相对并发与执行单元的最大响应时间下降率

（2）错峰控流

在各装备部门系统联调时，系统可能接收在某个时刻有上万的请求，但是处理 Task 的速度很难提升。比如数据库和网络 I/O 等耗时的处理、信息提示数量限制。那处理完几百条请求我们就必须等待了。所以，利用异步消息队列这种中间的通信系统，等到执行单元有能力处理消息的时候再去处理，是一种合理的解决方案。

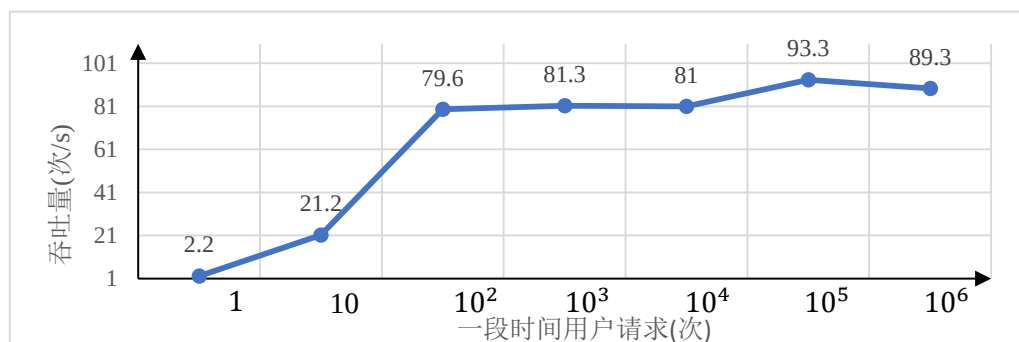


图 3-28 系统吞吐量变化

在吞吐量一定的情况下，仍然需要处理大量的响应消息请求，使用异步消息队列可避免系统接收处理请求异常的情况。如图 3-28，吞吐量随着请求次数的增加，逐渐达到瓶颈 90（次/秒），但此时系统的异常请求仍低于 3%。即多余的请求存储在了异步消息队列中等待系统处理，以保证系统正常运行的同时处理请求。

（3）解耦

异步消息队列的核心就是在多个连接紧密的组件模块之间加入缓存等待区，实现多模块之间的解耦。各模块只需完成自身处理任务，将结果集合存放到队列中等待下一模块调用处理。如复杂装备数据应用场景中，系统接收到用户注册任务消息，注册模块从异步消息队列中获取该任务，往用户表插入新用户注册记录，如：用户编号、初始权限、注册时间等。若任务场景变换，在注册前需要进行安全校验，此时只需在系统中添加新模块，将验证结果加入到消息队列中，此时该模块响应结束，原注册模块从消息队列中获取验证消息继续执行数据库插入操作，如图 3-29。

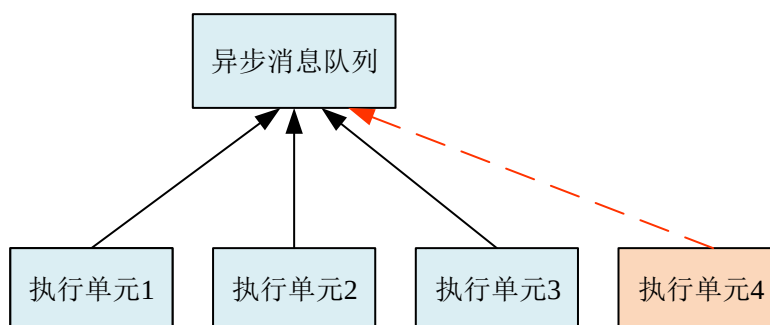


图 3-29 异步消息队列解耦特性

3.4 本章小结

本章围绕复杂装备数据动态管理中两个核心的问题：复杂装备资源动态资源多用户多权限的管理问题，以及在单机资源有限的情况下讨论复杂装备数据消息任务分发的异步化问题。首先，根据复杂装备数据的资源管理权限复杂和请求容易阻塞的特性，以 RBAC 为核心权限管理架构，研究能在用户-角色-权限的框架下，灵活添加权限组别的方法。在不改变 RBAC 的基础架构的前提下，使用组别层的概念替换了角色层，实现了异构数据资源的权限灵活地分组管理目标。其次，研究能在复杂装备数据管理中建立异步化的任务消息处理方法，在系统最大吞吐量相对固定的前提下，解决了复杂装备数据请求同一时刻大量涌入的问题，达到了对装备消息异步化处理的目的。最后，融合权限管理以及异步消息队列，构建权限灵活分配、任务消息高效处理的基于异步消息队列的装备资源管理架构。

第四章 基于服务的复杂装备数据集成研究

4.1 引言

OSLC (Open Service for Lifecycle Collaboration, 面向生命周期协作的开放服务)。OSLC 作为网络服务采用 REST 的 HTTP 风格,同时其严格遵循 OSLC 规范。通过网络资源链接定义规范中所常用的统一资源定位符(Uniform Resource Locator, URL)来链接对应的网络资源。在 Web 请求协议中,通过请求响应中标记相应的方法来获取资源,例如 GET、POST、PUT、HEAD 和 DELETE。本章根据 OSLC 核心规范,结合复杂装备数据领域的映射驱动模版,给出基于服务的复杂装备数据集成中的集成步骤与装备数据标准定义。这是一个由核心规范构建系统架构的过程。最后将本章研究方法与其他集成方法进行对比。

4.2 基于服务的装备数据集成技术

经过长期以来的数字化构建,已在军事、工业领域电子装备设计、制造、使用和维护等军工装备领域生命周期的各个方面生成了大量结构各异的数据。随着各领域的复杂装备向技术难度高、设备精度高的产业领域更新迭代,各种工业互联网、数值化设备和智能工业软件得到了大面积的装备使用。复杂装备数据生命周期中,不同的装备研发部门根据自身需求开发了自己的装备保障资源系统。但是,由于缺乏统一的集成和总体规划,系统数据的存储模型和数据源是不同的。这些也导致数据源在各个方面的异构性,并且资源之间的操作性和资源共享也很困难。如何整合这些异构数据,成为迫切解决的问题。

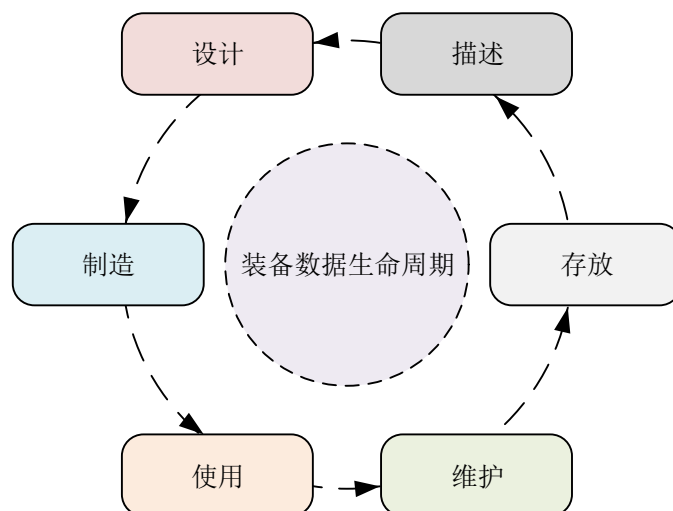


图 4-1 装备数据生命周期

目前,整个复杂装备部门都为特定的场景应用研发相应的资源接口,并为场景需求变更重写新的接口功能。资源访问接口需求也在不断变化,没有相对一致和约定俗成的结构规范,使接口之间操作性变得不再简单。

因此,规定一个结构统一、可重用性高的资源访问接口,是复杂设备数据集成的主要目的,使得接口调用方不需要不考虑异构子系统集成、领域数据资源整合的问题。

首先,复杂装备数据集成首先要定义数据层,实现各个装备部门异构数据的接入,同时,将接入的异构数据进行格式转换,归一为异构数据对象。然后,定义逻辑层,包括数据映射驱动模版以及数据处理引擎。数据映射驱动模版,采用 OSLC(Open Services for Life-cycle Collaboration)生命周期协作开放服务思想,结合复杂装备的业务数据标准定义,将每个字段具体的含义表达包含在驱动模版内,为复杂装备数据整个生命周期中不同集成提供一组规范。同时,数据映射规则将字段定义映射到相应异构数据对象中。

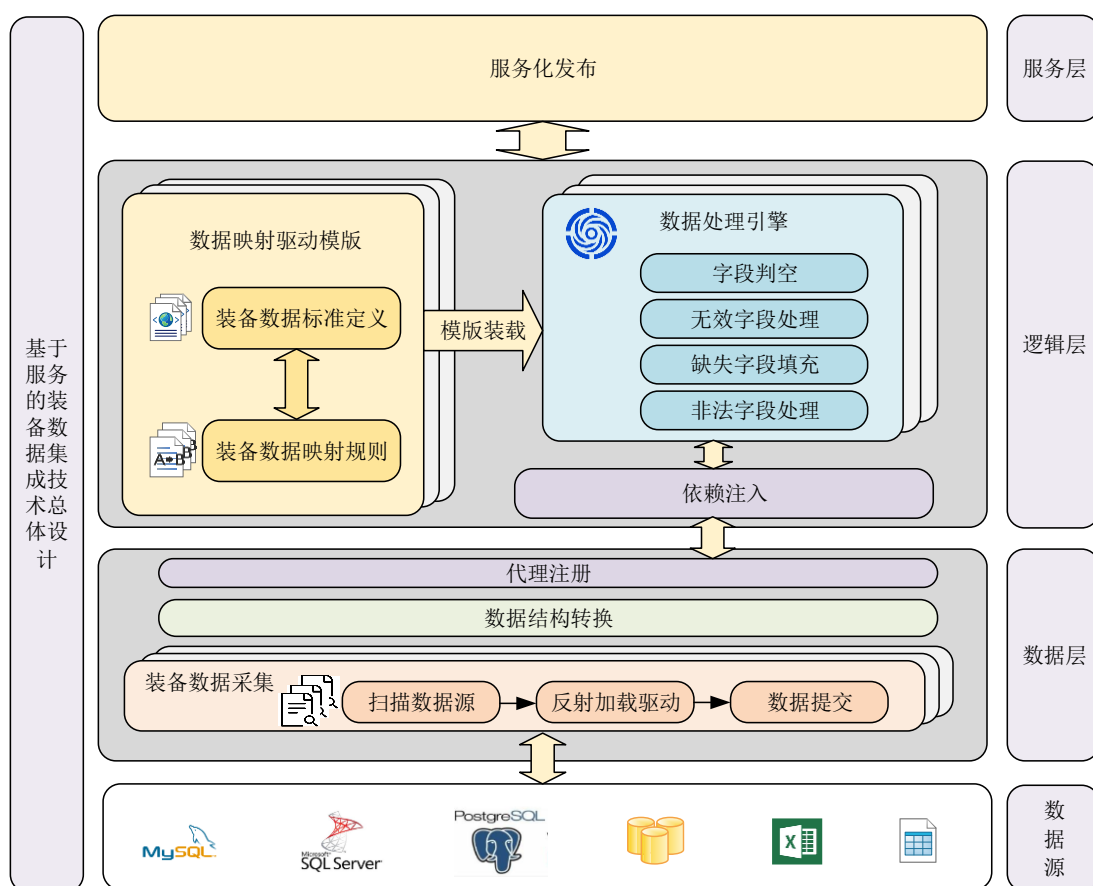


图 4-2 基于服务的复杂装备数据集成架构设计

数据处理引擎,将数据对象中无效或者非法字段进行填充处理,保证异构数据对象的完整性。最后,通过使用 REST 架构风格的服务层,将统一处理后的异构数

据对象以 JSON 的形式通过网络 URI 进行数据的传输，如图 4-2。

4.2.1 装备数据采集

由于装备数据具有数据异构的特点，异构数据源根据数据存放结构不同，一般分为关系型数据库、非关系型数据库和文本数据。一般来说，关系型数据库可使用 JDBC(Java Database Connectivity)的体系结构来进行复杂装备异构数据的采集。JDBC 是一种接口协议，使得应用程序能够与各类数据库建立连接，实现对话交互、获取数据。JDBC 体系结构由以下四层组成：Java 应用程序层、JDBC API 层、JDBC 驱动程序管理层和数据库层，如图 4-3。

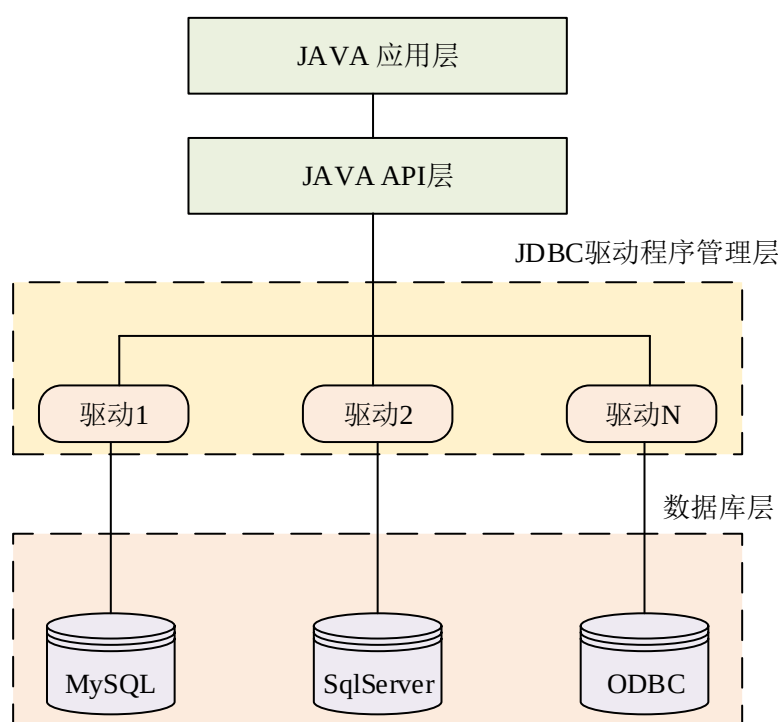


图 4-3 JDBC 体系结构

Java 应用程序层包括各类场景需求实现程序。JDBC API 层定义了可以在程序中连接数据库，对数据库进行操作的许多方法，通过接收上层请求对数据库进行业务操作。管理层从底层驱动层面上将不同数据库的异构性消除，实现 Java 应用访问各类数据库。各类配置文件列表存放在 JDBC 驱动程序管理器中，当 Java 应用设置相应配置后，管理器调用相应的配置以及驱动文件对数据库进行链接，完成后续的数据查询处理等相关任务。

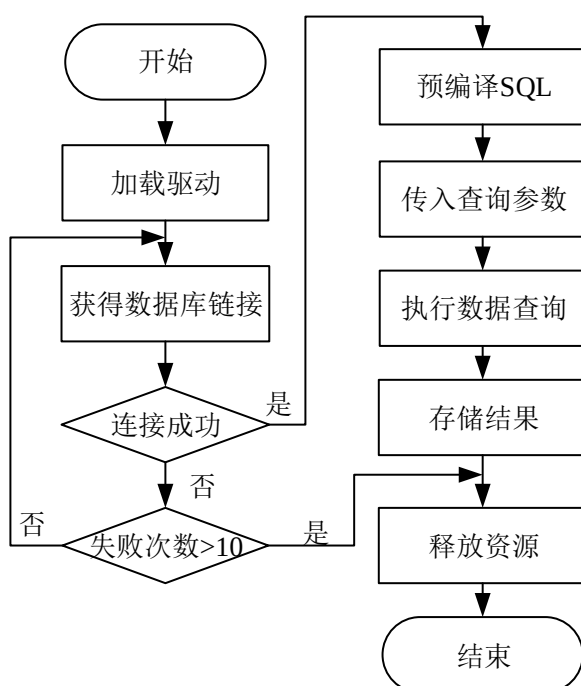


图 4-4 JDBC 链接数据源

如图 4-4，首先通过读取驱动名称，找到相应数据库的链接驱动。通过 Java 反射机制，加载相应数据源的驱动到系统中。通过程序内部提供的 JDBC API 驱动对象，通过代码配置数据库连接对象，并且用于与数据库进行握手链接操作。然后程序中的 JDBC 接口通过 Java JDBC 驱动程序的转换而转换为代码可直接访问对象，以便访问数据库。

驱动装载成功后，系统开始获取与数据源的链接，开始尝试通过 API 获取链接配置数据。通过本地代码配置链接属性字段，包括所需连接的驱动程序名称和 URL，登录所需要的用户名密码等。不同的数据库所需配置的属性字段不尽相同。程序与数据库尝试连接，

若连接失败，数据库设置失败连接次数限制，当失败次数超过一定阈值，则放弃连接，返回错误信息，避免消耗系统资源导致应用阻塞宕机。

当系统与数据源建立连接后，开始对查询语句进行编译。由于需要对不同的数据表进行查询，部分查询语句需要进行反复的拼接，使用 Statement 需要对 SQL 语句进行拼接，灵活度较低。而 PreparedStatement 接口提供预编译机制，性能更高，由于预编译已经将查询语句加入到缓存中，传入的参数都会被填充为字符串，可避免 SQL 注入式攻击。因此，采用预编译的方式对查询语句的拼接。

查询数据源中的数据后，将其暂存在缓存中。由于会话链接仍然在缓存中，下次的数据查询会话会在数据库中的一级缓存和二级缓存中先去获取，若存在该会

话的缓存信息，则直接获取使用，加快查询事务效率。但连接资源被大量占用而不进行会话连接的断开，后续数据源连接将无法连接。

因此，需要在一次会话建立、获取数据完成后，将连接断开释放资源。但频繁地进行连接和断开，会导致更加繁重的系统负担。所以，使用 JDBC 连接池，设定数据库最大最小连接数和超时等，需要多少分配多少连接，当超过最大连接数的时候，为了保证系统的可用性，多余的连接请求将进行等待。这样即可解决大量数据访问引起的效率问题。

4.2.2 数据结构转换

在复杂装备领域中，考虑到各个部门数据的理解以及系统的设计的差异、数据的格式和定义多种多样。因此，本文需要将二维数据表定义为统一的集成数据结构，用来接收解析到系统的数据字段。首先介绍集成前的数据结构定义，数据结构转换前的类型有：

(1) 关系型数据：该类数据结构由一维数据元素构成，整体为二维数据表格，如图 4-5。一张表表示一类关系结果集合，一行数据表示一组装备信息数据便于用户理解。采用表格的方式存储，读取和查询直接明了。在存入数据之前，需要定义各个字段的名称，数据稳定性以及完整性较高，但会导致复杂装备表结构的改动十分困难；

字段	装备名	装备指标	...	
数据1	...	...		
数据2	...	...		

图 4-5 二维数据表结构

(2) 非关系型数据：该类数据结构为键值对类型的哈希表，如图 4-6。不同的键值能够指向与之对应的数据内容，而与关系型数据不同的是，存放的数据内容可以是一个值，也可以是一张表的树状结构字符流，通过反序列化或者特定的解码方

式恢复表结构数据。该类数据结构拓展性很高，但数据存放的稳定性较低；

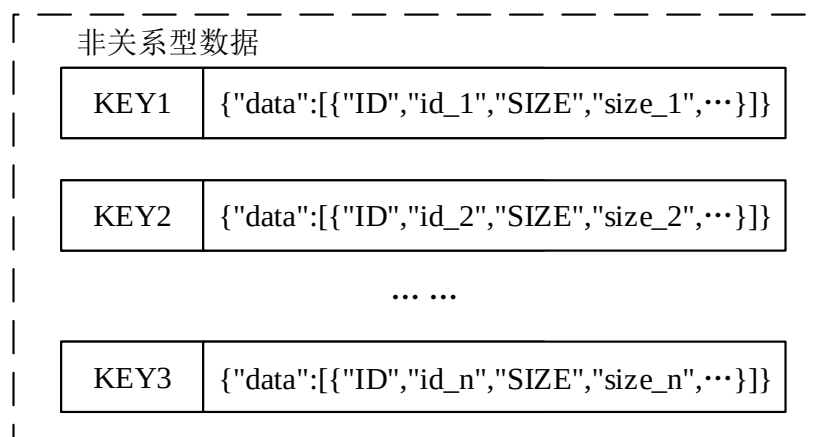


图 4-6 非关系型数据结构

(3) 文本数据：普通的需求文档或者管理报表文件等数据都存放在文本数据结构类型中，不同的文本结构有着不同的解析数据的方式，因此需要对不同的文本类型设计解析接口获取数据，如图 4-7。

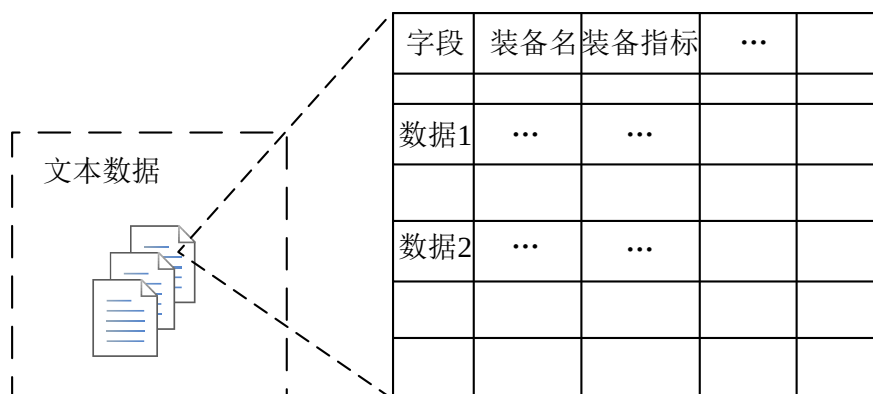


图 4-7 文本型数据结构

从各个数据源采集到的复杂装备数据，由于其数据格式的异构性，如图 4-8。关系型数据的格式皆为使用表结构，按行读取，按列匹配数据即可读入到数据容器进行保存；非关系型数据，由于其匹配关系为键值对形式，以键为该数据索引，值的数据格式灵活多变，一般保存的是 JSON 字符串。

因此，需要对该字符串进行 JSON 解析，分离字段名以及数据保存至数据容器；文本数据则是以 Excel 和 CSV 为复杂装备数据的主流文本保存形式，通过调用相应的 JAVA API，解析字段名以及数据，将其保存至数据容器中。最后，系统可通过各类封装好的数据容器对其内部数据进行调用。

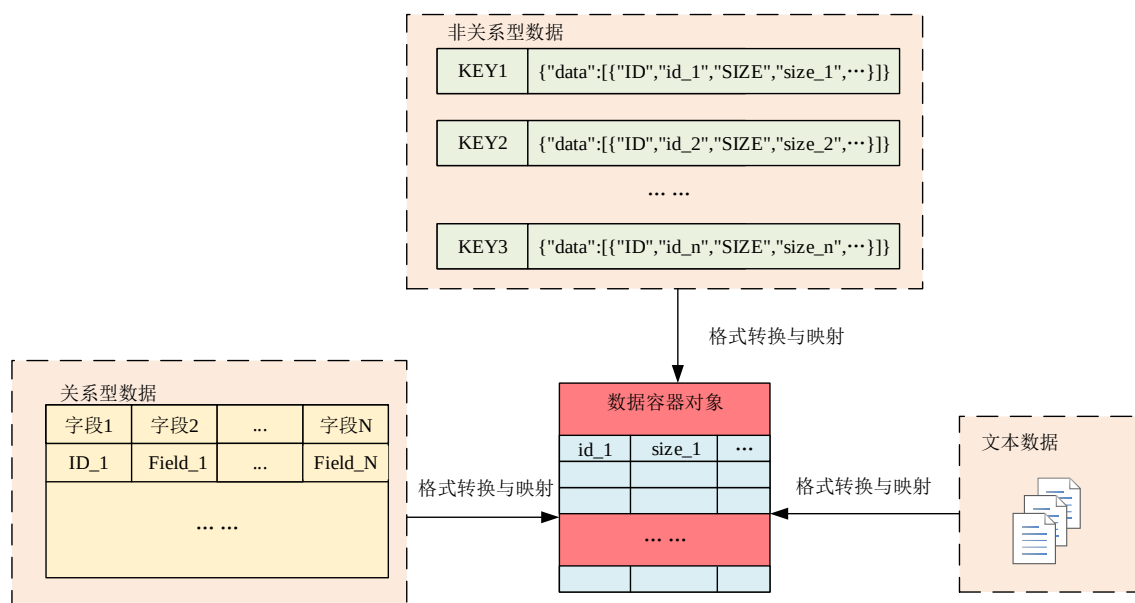


图 4-8 数据容器对象转换

通过面向对象开发的思想，将各类异构数据的字段以及具体数据抽象为对象属性。实现将异构数据转移至数据容器的过程，如图 4-9。

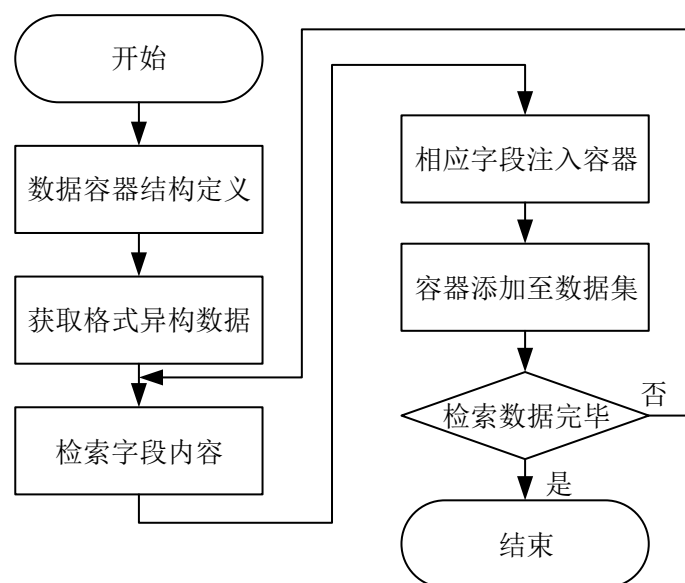


图 4-9 异构数据转移流程

关系型数据、非关系型数据、文件数据各类复杂装备异构数据源格式都不同，因此，可将其抽象为一个数据的对象类，在系统中进行使用。

首先定义接入的异构数据源的数据格式，采用面向对象的开发方式，创建一个对象类，类内部定义异构数据的属性。根据复杂装备数据类型的不同，可在容器中定义整型、浮点型、字符型等字段属性，容器通过继承用户自定义抽象资源类，使

用容器内的 GET/SET 方法，获取与设置各个字段的资源内容。

定义该异构数据的属性字段，通过使用在数据容器内部定义属性字段的方式，可将相应字段屏蔽在数据容器中，保证外部不能够直接对属性字段进行操作，同时提供相应的属性访问方法，可对属性进行获取以及修改。

完成数据容器的定义后，通过装备数据采集的连接，获取格式异构的数据集合。异构数据通过数据采集部分，采集数据的格式为按行读取解析，多维数据加入到下一行进行存储的方式，形成一个表的结构进行暂存。

接着扫描异构数据集各个字段信息，使用容器内部定义的属性 GET/SET 操作方法，将其映射至定义好的数据容器中。定义的数据容器内部结构可看作一个表结构，每一行存储一个数据对象，对象设置的属性即为异构数据的各个字段。

通过遍历的方式，扫描暂存的异构数据集合后，以对象的方式加入到容器中，形成定义的数据容器有多行，存储着多个异构数据对象，表示异构数据源中所有的异构数据。

4.2.3 数据处理引擎

复杂装备尺寸或雷达辐射范围等，无趋势、无季节性、与时间不相关的参数。可采用基于固定值的异构数据清洗技术，将缺失的数据用平均值或固定值进行填补。

将采集到的数据列表中的类型分为数字类型和非数字类型，分别对两大类进行缺失值进行填充。如果缺失字段为数字类型，则基于该列所有字段或者其他关联列字段的平均值来填入缺少的数据列。如果缺失字段是非数字类型，则根据复杂装备属性的定义使用默认字段进行填充。或使用所有其他数据列中具有最高频数的数据列值来填写缺少的数据列值。

另一种填充规则为通过扫描其他数据列并计算得到相应的平均值，将该值作为当前数据列的默认补全内容。该方法称为条件均值补全法(Conditional Mean Completer)与上一类方法不同的是，不是取缺失值当前列的数据平均值设为默认补全内容，而是找到与之关联的第二列进行平均化处理，生成补全内容。

例如图 4-10 所示，数据 1 表示缺失的异构数据字段，其中 SIZE 为数值型字段，在该条数据中值为 null，表示该字段为空。数据 2 表示无效的异构数据字段，其中字段 Current 在该复杂装备数据的范围为(0.50-3.50)，该数据字段由于人为记录或者测量数据错误等原因，导致数据无效。数据 3 表示非法的异构数据字段，由于复杂装备有各自的命名规范，若某复杂装备数据名称与命名规则不同，那么认定该字段为非法字段进行处理。

数据1: 缺失数据					
NAME	Equipment_1	SIZE	null	Current	1.30
...					
数据2: 无效字段					
NAME	Equipment_2	SIZE	25.12	Current	450
...					
数据3: 非法字段					
NAME	123	SIZE	25.36	Current	1.41
...					

图 4-10 缺失字段情况

数据 1: 将通过空值判断是否需要进行缺失数据插值处理, 若为缺失数据, 则考虑需要到该字段所具体代表的概念含义, 考虑到所可能的属性来取值。此处为 SIZE 字段缺失, 由于装备尺寸为浮动数据, 可用平均值的处理方式, 填入平均尺寸信息作为缺失信息补充。

数据 2: 通过判断字段范围是否符合复杂装备定义后, 此处为超出数据范围的非法字段, 可使用该列数据的平均值或给定的零值进行填充替换。

数据 3: 通过判断字段类型是否符合复杂装备定义后, 如数据 3 的 NAME 字段为字符类型, 此时填入整数类型的字段为非法异构数据字段, 即可将无效字段使用平均值进行替换。填充流程如图 4-11 所示。

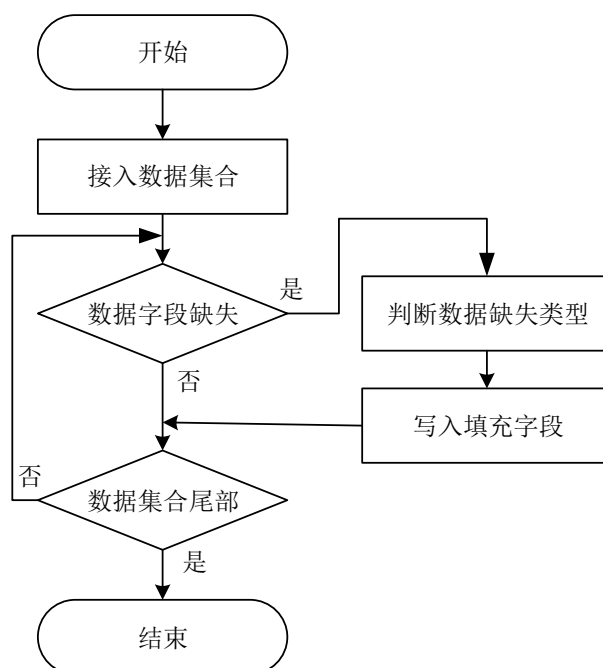


图 4-11 缺失字段填充流程

当数据处理引擎获取自定义的数据容器后,将指针 1 指向数据容器集合头部,指针 2 指向数据容器集合尾部。指针 1 指向数据容器中的每一行存放的对象,判断给对象是否符合复杂装备数据的定义规范。

若存在数据缺失的情况,则跳转至判断数据缺失的类型,选择填入默认字段或者填入该列的平均值进行填补缺失字段。处理完成后,指针 1 指向下一数据对象进行处理,直到指针 1 与指针 2 相遇,则遍历结束,该数据容器对象处理完成。

如图 4-12 所示,以上三类需要处理的数据段,通过数据处理引擎后,将会处理为完整的异构数据段供后续使用。

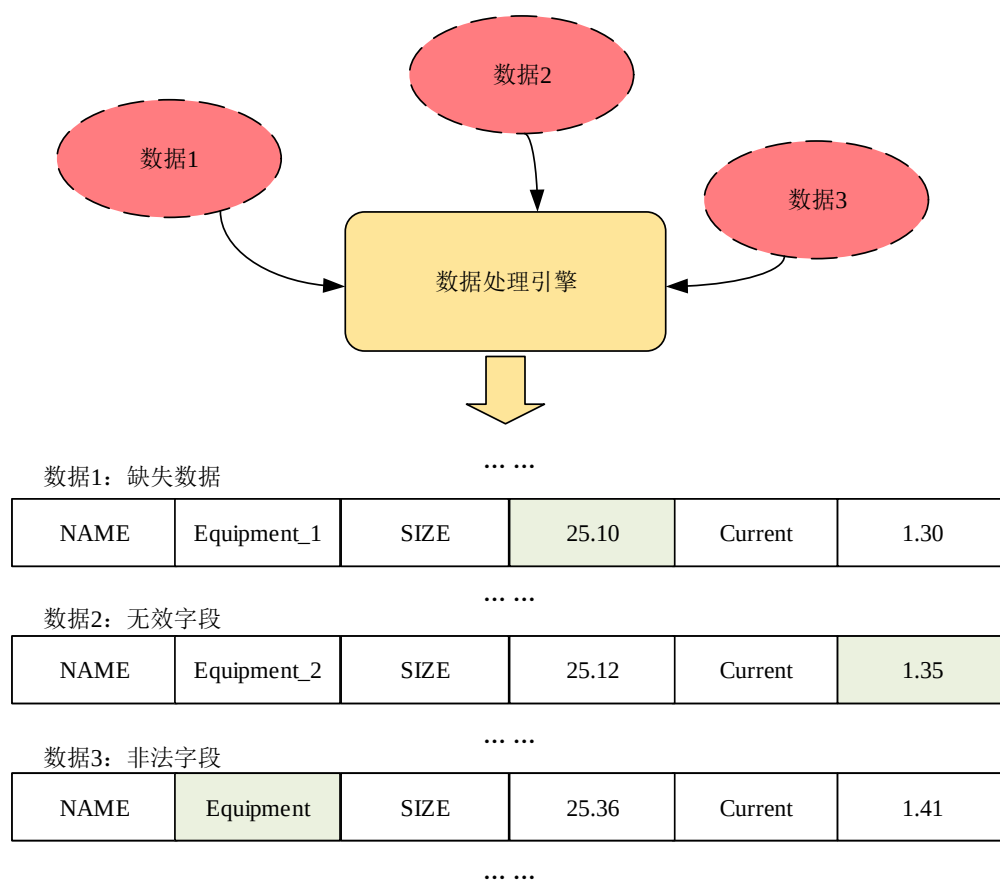


图 4-12 缺失字段处理结果

通过数据处理引擎对各类数据内容进行解析,分别判断数据字段缺失类型,填入默认或平均数值数据,使得后续数据资源服务化过程中获取到的 URI 字段值不为空,避免后续系统获取错误数据导致决策错误。若对非法字段不做处理,会导致获取部分数据字段定义时,数据值与领域定义的不匹配导致领域数据理解错误。若进行数据填入处理,还可以保证用户获取到的领域数据集合的完整性以及该字段数据定义的准确性。

4.2.4 数据映射驱动模版

4.2.4.1 装备数据标准定义

使用生命周期协作开放服务(OSLC)的资源标识框架,来进行对装备数据标准的定义,OSLC 核心规范的概念和关系如图 4-13 所示。OSLC 服务可以与其他工具创建一系列 Web 服务规范,这些规范支持复杂装备领域中软件和产品生命周期的所有阶段。在复杂装备领域中,异构数据的生命周期每个部分都有其自己的组织和规范,例如,变更管理,质量管理,估计和度量等等。目前复杂装备数据集成涉及到多个领域的知识,意味着使用者需要大量阅读文档了解跨领域的数据定义标准,花费更多时间开发管道代码进行异构数据结构的转换。更重要的是,这些集成都需要在案例的基础上完成,从而实现点对点的集成。

因此,需要 OSLC 对跨领域的数据标准进行统一封装描述,每个域规范都基于 OSLC 核心规范进行编写。OSLC 确保用户在接入数据的同时也能够理解不同领域中的数据,以便提高集成或分析效率。

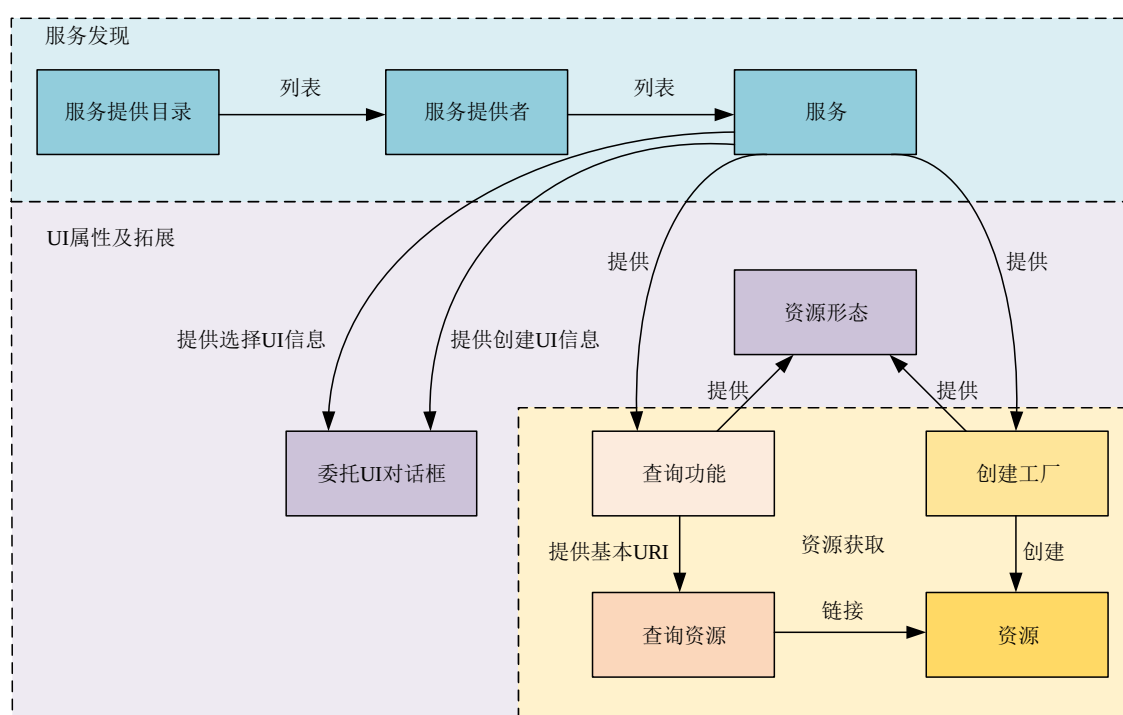


图 4-13 OSLC 核心规范的概念和关系

图 4-13 中分为服务发现、UI 属性及拓展以及资源获取三部分,其中模块定义:

资源: 在网络交互中可以由 URI 标识的网络数据资源或服务。资源可以有不同的类型,如语言种类、数据格式、所属类型等。

URI: 统一资源标识符是简单格式化的字符串, 通过设定名称、位置或任何其他特征来标识资源。

服务(Service): 一组功能, 使 Web 客户端可以创建、检索、更新和删除一个 OSLC 域相关联的资源。

服务提供者(ServiceProvider): 一种模块或在线服务模块, 提供一个或多个 OSLC 服务的实现, 这些服务本身可能实现不同的 OSLC 域规范。

资源(Resources): 由 OSLC 服务管理的资源, 可以具有属性, 并且可以链接到其他资源, 包括其他 OSLC 服务提供的资源。

资源形态: 定义资源中, 期望用于其每种操作的值类型、允许值、基数和可选性的特定操作, 即创建、更新或查询的 OSLC 属性集。这样的操作如 OSLC 创建资源和查询资源, 其他应用包括在表中显示数据的应用程序。

创建工厂: 创建工厂提供用于通过 HTTP POST 创建新资源的 URI, 并且还可以提供描述可能创建的资源类型的资源形态。

查询功能: 查询功能提供基本 URI, 用于形成查询资源 URI, 以及可选的资源形态, 它们描述可在查询表达式中使用并在查询结果中返回的属性值。

OSLC 技术的核心规范首先通过服务提供目录, 通过迭代的方式对所有链接到的资源进行查询, 捕获所有领域资源信息。当获取到的资源定义包含有子集元素时, 服务扫描将以深度优先的遍历方式逐级向下进行查找, 将子集服务查找结束后返回父级, 最后以列表的形式保存所有信息, 如图 4-14。

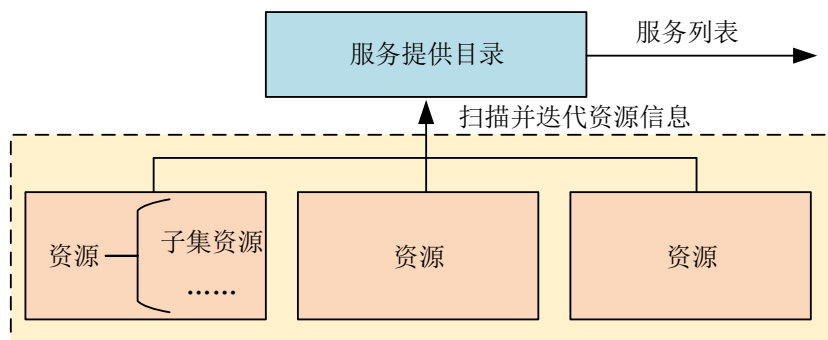


图 4-14 OSLC 服务注册

当下一级服务提供者调用相应的服务资源时, 发送查询请求到上一级, 获得服务提供方目录, 从目录列表中找到注册的服务信息以及服务对应的 URI, 如图 4-15。

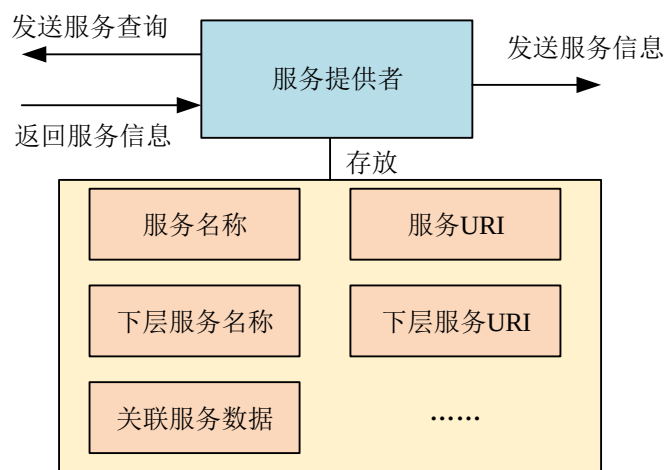


图 4-15 OSLC 服务搜寻

所有 OSLC 资源都位于指定的服务提供者(ServiceProvider)中。每个 OSLC 资源 `oslc:serviceProvider` 都有一个可选属性,用于说明该资源属于的 ServiceProvider。客户端可以检索 ServiceProvider 中现有资源的列表。在 OSLC 中定义的创建任何新 OSLC 资源的唯一方法是在 ServiceProvider 中创建它们,直接通过 POST 或通过 UI 界面。ServiceProvider 本身是带有的 OSLC 资源 HTTP URI。

若用户请求为 GET 对 ServiceProvider 资源进行操作,则不会检索该资源包含的资源的列表。将检索 ServiceProvider 的常规属性,包括可用于查找或创建资源的 URL。若使用 POST 对 ServiceProvider 的 URI 进行操作,则很可能只会收到 HTTP 错误。

获得所需要的服务后,将服务信息转发到下一层“服务”中,这一层存放用户调用的服务信息,准备下一步对数据资源的操作。在服务中,用户可以在领域中创建相应的数据资源信息,也可通过资源所具备的查询能力获取相应的资源信息。通过用户相应的操作,可在相应的网页界面中呈现创建的数据、数据格式以及查询到的数据信息和领域描述等,如图 4-16。

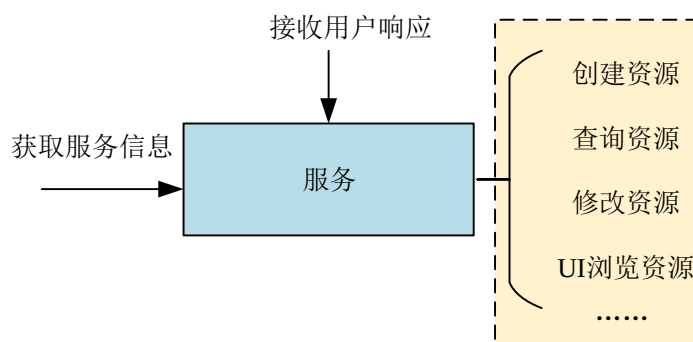


图 4-16 OSLC 服务操作

OSLC 的核心思想是链接数据,将各种领域中涉及到的资源信息都作为可链接的数据,通过在资源中标记相应的 URI,用户可以通过链接访问特定的 URI 链接获取相应的数据资源。同时,在 OSLC 中运行在服务发现列表中找到数据资源间的相互联系链接,使得用户可以在该领域中获取更多相关的数据服务信息。

OSLC 基于这一核心思想,针对领域中的复杂的数据集成链接问题转变为数据资源获取以及服务发现问题。将复杂装备数据集成以及各类软件涉及到的信息进行资源化,如在装备制造中的需求方案、复杂装备检测文档、特定复杂装备场景可视化集成等。通过接收用户的请求响应,可由获取到的服务信息进行判断,该资源是否支持相应的操作。若用户选择在该领域中创建资源,服务将找到请求处理哈希表,找到创建数据方法类,处理完成相应的资源操作后,若有返回结果,则逐级返回,直到服务再转发给用户。通过 OSLC 核心规范,可对复杂装备数据的资源标准进行定义,如图 4-17。

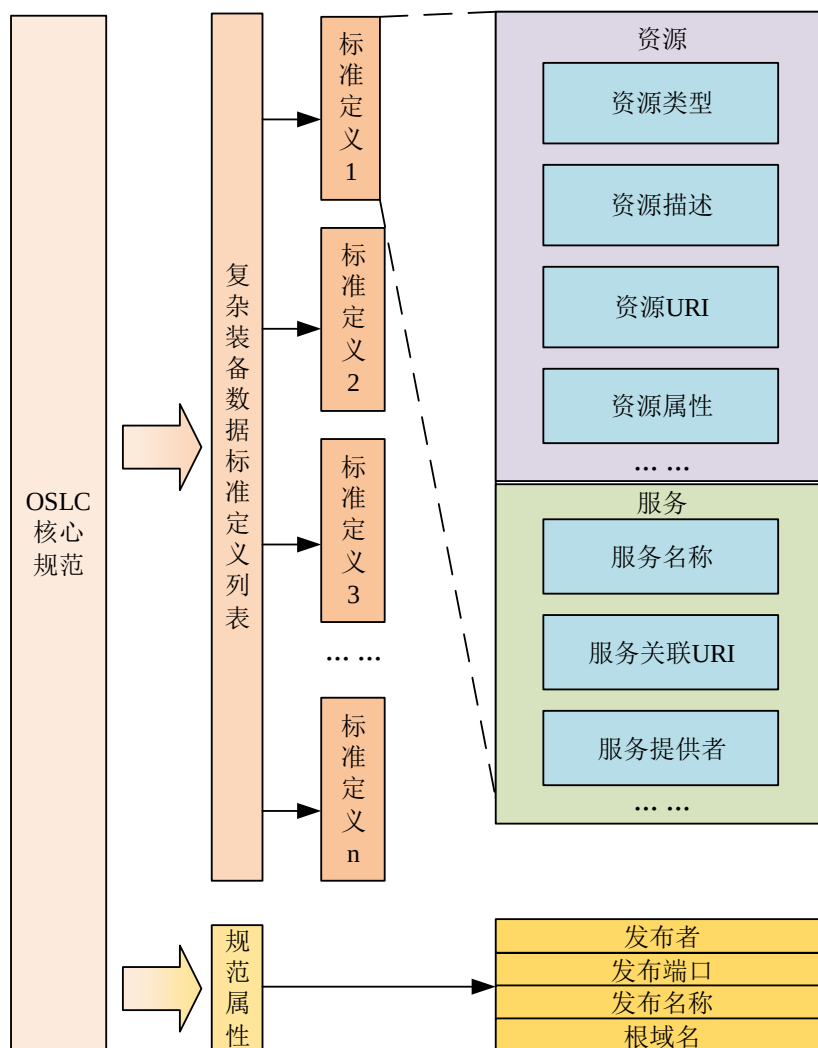


图 4-17 OSLC 定义复杂装备异构数据

标准定义列表(LISTS)包含所关联的复杂装备异构数据源配置, 每一组配置表示对该数据源的标准定义(Definition), 表示为 $LISTS = \{Definition, i \in [1, 2, 3, \dots, n]\}$ 。标准定义包括资源(Resources)与服务(Service)两部分。

资源部分描述所关联资源的名称、资源标题、各个字段的属性, 其中字段属性包含字段描述、RDF 描述、字段类型等, 用于描述领域资源的数据定义等信息, 如图 4-18。

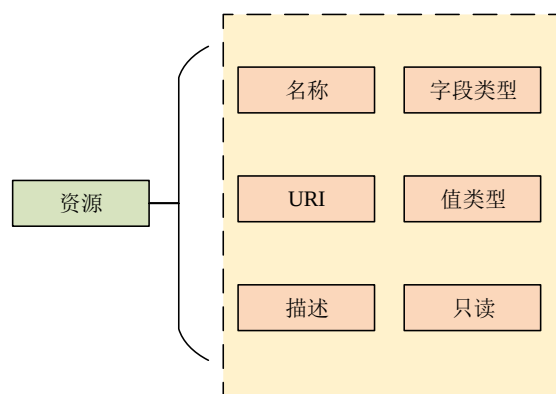


图 4-18 资源包含内容

名称(name): 必须作为复杂装备数据的本地名称部分有效的属性名称, 与 XML 命名空间类似。

URI: 标识属性的 URI。通过将 Name 附加到与该属性关联的 Namespace URI 的末尾来形成 URI。例如, 名为 `oslc:ServiceProviderCatalog` 的资源定义了一个以 `demoURI` 命名的属性。

描述(description): 属性的描述, 可将加入对复杂装备数据在特定领域中的数据描述。

字段类型: 值必须为以下之一, Exactly-one、Zero-or-one、Zero-or-many、One-or-many, 用于标识该资源数据字段个数的可能性。

值类型(occurrence): 一个属性可以允许多个值类型, 并且一个值必须满足其中的一个或多个。每个值类型都必须是与以下之一相对应的, URI、文字值类型、布尔值、时间类型、十进制数字类型、资源值类型: 资源值是指定 URI, 即 URI 引用的资源。

只读(readonly): 布尔值指示在创建资源后是否允许客户端替换属性的值。允许的值为: true, false, unspecified。True 表示提供者在创建资源后不应允许客户端更改属性的值。False 表示提供者可以在资源创建后允许客户改变属性的值。

服务部分包含服务描述、服务提供者、查询功能、创建功能等。以上配置项以 JSON 树状格式进行描述, 一组异构数据源定义为目录对象的一个元素, 系统通过

加载该 JSON 描述配置文件，遍历属性对象以及资源对象里的各个元素，即可获取所有资源描述定义以及配置信息，将复杂装备数据领域标准定义注入到容器服务中。

首先是以 catalog 字段作为 JSON 键值对的属性键值，该键值对象中包含有目录数组，存放着所需要扫描的服务定义信息，如：资源描述、资源名称、服务提供者等信息。系统通过读取该字段获取服务信息，与自定义的数据容器进行匹配对应。

资源信息则是保存在 resources 键值对字段中，包含有所需集成的异构数据源信息，如数据源名称、数据源标题、数据源中各字段的属性等信息。

4.2.4.2 装备数据映射规则

系统通过引入由 OSLC 核心规范定义的复杂装备数据标准，在复杂装备数据容器中，通过注解的方式对各个字段进行映射，映射结构如图 4-19 所示。

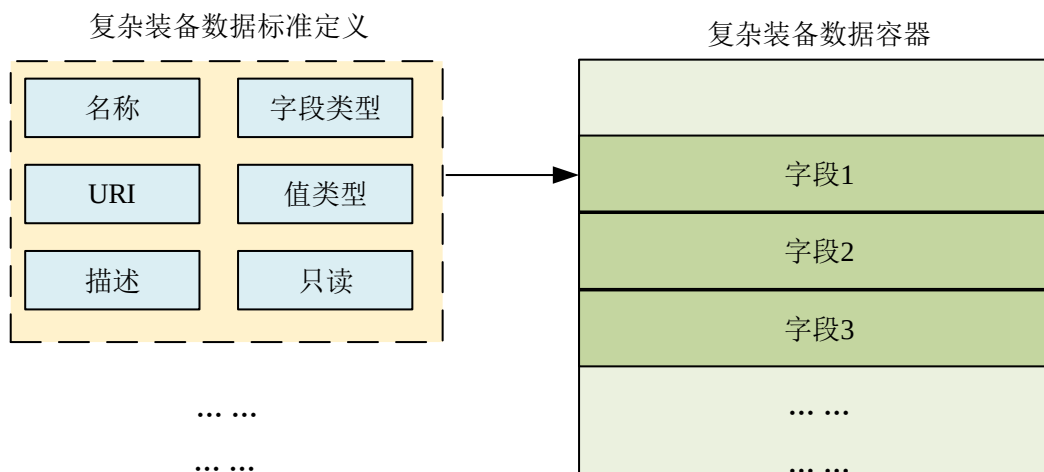


图 4-19 字段映射结构

注解可以通过反射获取注解内容。JVM 将留下相应的注解内容，在函数执行时将接收到注解内容，通过在注解处编写响应的复杂装备数据描述，即可对该数据类型添加领域定义等信息，提供给用户数据集合的同时也提供该数据集合的意义描述。每一个注解都与枚举类进行关联，枚举类包括注解的类型声明，通过枚举的方式对数据集合的各个字段可进行相应的描述，如 OSLC 数据集描述、属性定义、标题、值类型、字段类型等信息。

通过 4.2.2 中数据容器的定义，将 OSLC 规范与数据容器进行结合。参考 4.2.4.1 中的装备数据标准定义，引入符合 OSLC 资源描述规范的注解描述。如：OslcDescription、OslcPropertyDefinition 以及 OslcTitle 等复杂装备资源描述定义。通过在容器中以注解的方式对所需集成的数据源加入该领域的的数据规范等信

息，以便在服务化发布数据时，能够携带相应的数据规范，使得跨领域使用者能够理解接入的集成数据。

4.2.5 服务化发布

通过异构数据服务化访问技术，对所涉及的系统应用进行抽象，结合 OSLC 核心规范，生成相应的 URI 链接提供外部访问，请求架构如图 4-20 所示。该架构通过服务层作为中间层，接收来自外部的各类请求，通过内部请求映射后获得相应的执行方法，调用不同的系统应用实现请求任务。

最后通过返回服务层结果，返回到相应用户方法调用处。服务化后的资源将分配一个特定的 URI，资源被 URI 定义并链接之后，可以通过 URI 去访问获取所需要请求的资源。一个 URI 包括三个组成元素：IP 地址、标识字段、相对 URI。

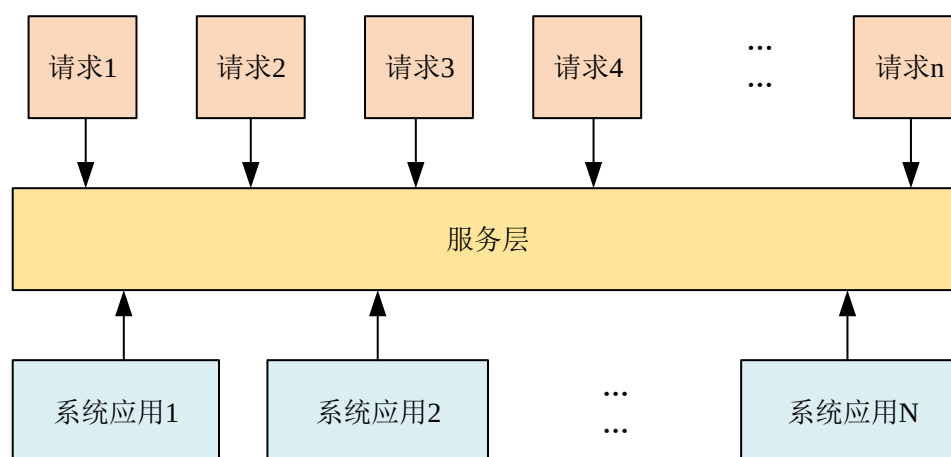


图 4-20 服务层请求架构

本文研究的基于服务的异构数据集成是一种基于 REST 风格发布 URL 链接，结合面向服务的体系结构设计，参考 OSLC 核心规范以及领域集成标准定义的异构数据集成系统。

面向服务的架构(service-oriented architecture,SOA)是一种设计方式，使得复杂装备数据服务在其生命周期中，可以通过服务 URI 的方式访问各阶段的数据。设计相应的复杂装备领域应用服务时，采用面向服务的体系结构作为开发核心思想，通过将各类服务功能统一成相应链接的形式发布到网络中，该服务将促进快速的复杂装备数据集成和相应装备业务流程信息一体化。

面向服务的框架可以根据复杂装备数据的不同场景，对不同的应用系统进行松耦合式的链接，通过简单的 URI 的方式将数据资源进行发布，使用者无需对数据接口过多的参数传递，即可获取不同领域中的数据资源信息。通过定义 URI 接口的规范信息，使用者可通过 URI 的层级描述找到特定的数据资源，也可通过父

级的 URI 信息找到相关数据信息。

本文提到的面向服务的体系结构的含义是系统架构包含三类对象，即系统应用、接口请求以及服务层。复杂装备异构数据的集成核心是按照 OSLC 规范对相应装备领域数据标准映射、Java 定义的数据容器以及 URI 服务访问数据的方式来消除底层分布式数据源的异构性，并基于此建立复杂装备异构数据服务层。

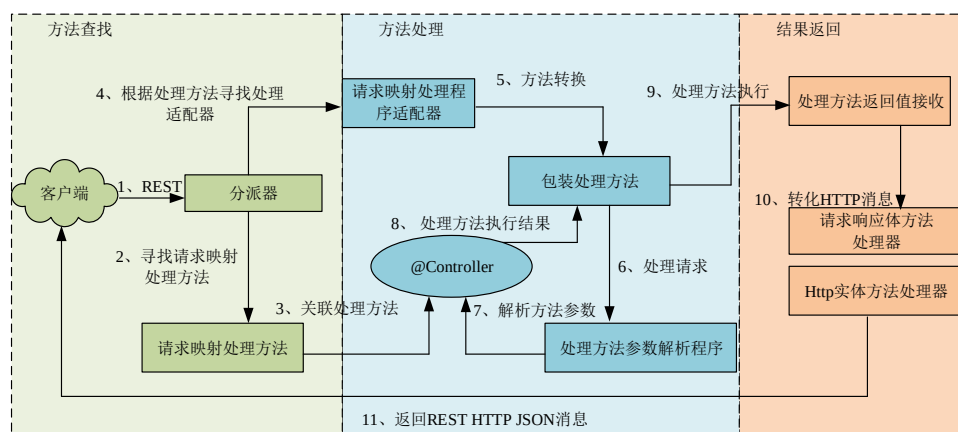


图 4-21 REST 服务流程

REST 服务流程如图 4-21，客户端发送 REST 请求复杂装备数据，通过分配器 (DispatcherServlet) 找到所需要处理请求的映射。步骤 2、3，将请求所需要的方法名以及地址获取，分配器返回方法信息至处理适配器 (Handler Adapter) 中，通过 Handler Method 接收方法参数传入到所需要调用的执行方法，将方法信息传入控制中心。使用注解 @Controller 注册标识的方法将会被扫描到相应的处理适配器。处理适配器将获取所需要处理请求的方法，得到执行结果，通过处理方法返回值接收转换为 JSON 格式通过 HTTP 传回客户端。

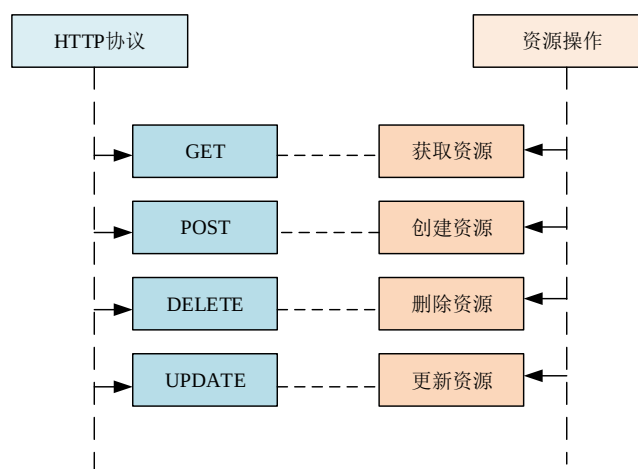


图 4-22 基于 HTTP 协议的资源操作

如图 4-22 所示，通过 REST 形式的 URI 传输方式，OSLC 采用 REST 风格，特点是所有数据信息都可资源化看待。复杂装备数据中的资源都可以进行统一的资源化描述。可以简化用户获取资源的难度，用户可通过 URI 的各级标识名来区别不同领域数据源、数据集合、数据字段等信息；通过请求方法，即可判断该 URI 的执行类型以及功能；通过返回的状态码，即可判断该次资源操作是否成功。

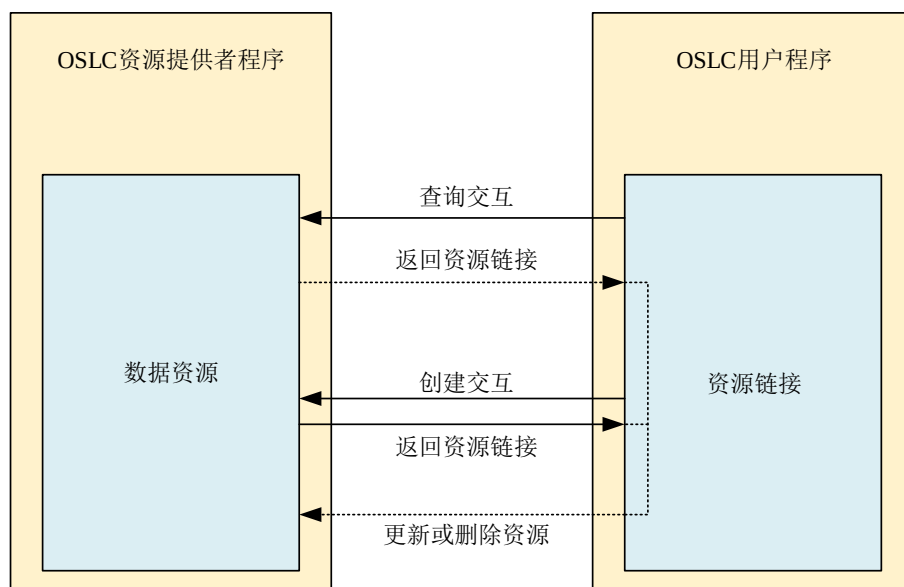


图 4-23 资源交互

用户发起交互，OSLC 返回相应资源链接，用户使用链接即可访问数据，通过以上步骤即可实现资源链接的交换，如图 4-23 所示。通过 REST API 服务代理封装所连接的复杂装备异构数据源，首先通过数据采集层构建适配器，连接各个的异构数据源获取异构数据。数据处理引擎将 OSLC 定义的标准模版进行字段映射后，通过反射的方式将资源注入到系统容器中。最后，通过服务层，通过服务代理的方式开放 URI 链接提供数据访问服务。

由于 JSON 具有跨平台性，可读性与 XML 相比相差不大。但 JSON 是简单的字符串语句，更容易生成以及传输。可扩展性，JSON 是 Javascript 的父级格式，并且可以存放 Javascript 组合对象，这具有 XML 所无法相比的优点，同时 JSON 是一种层级键值对的形式，则可更加灵活地对数据进行添加拓展等处理。

编码难度，JSON 是一类键值对象字符串，更容易被人理解以及编写修改。同时，在无辅助开发工具的情况下，由于 XML 文件需要大量的结构字符，因此 XML 的编写难度要远大于 JSON。

所以采用 JSON 技术进行系统数据之间的传递。在发布服务时，服务层通过各个注册的服务发布数据信息。

4.3 性能比较

复杂装备数据具有格式异构、各领域标准异构的特点,因此,数据集成方法需要解决数据异构的同时,还需要保证各领域标准的统一表达。从结构上看,基于服务的复杂装备数据集成是将数据连接归一到一个系统进行发布服务。

测试数据来源:项目合作的研究所提供本论文实验所需的 30000 条测试数据,30000 条测试数据包括关系型数据、非关系型数据以及文本数据各 10000 条;异构的测试数据,数据的异构性体现在,第一,各类数据的字段定义、字段数量各不相同。第二,数据解析方式不同,关系型数据通过扫描每行每列读取数据,非关系型数据通过读入序列化后的对象字符串,需要通过反序列化得到数据对象,文本数据则通过相应的文本解析接口,读取数据。第三,三类数据源所在的工程领域不同,各字段数据有相应的数据领域描述以及属性描述。

异构数据场景还原:获取到这三类测试数据后,将测试数据分别写入 Windows 系统中的 Mysql 关系型数据源, Linux 系统中的 Redis 非关系型数据源以及本机 Excel 文本数据源。模拟出异构数据源,数据结构异构、数据解析异构、数据领域异构以及数据分布异构的场景。

(1) 异构数据语义规范统一定义对比

现有数据集成方式没有将异构数据领域定义进行集成,导致各领域间数据无法理解。本文采用服务化的方式进行数据传递,则可以通过 URI 访问的方式包含数据描述信息。设置文本类数据源有 5 个领域文件数据。领域内数据文件序列,如: [kmeans_data.xls,T1.xlsx,T20.xlsx,T22_data.xlsx>tag1.xls]。

通过服务扫描后,将在文本服务中显示出以上 5 个文本数据源信息。读取 Service Provider 中存放的文本服务信息,对相应的领域数据文件进行标记 URI 后,使用者可通过分配的资源 URI 访问数据信息。

当服务访问 URI 链接: <http://localhost:8088/catalogs/excel> 后,将扫描到所有符合规范的数据文件。

获取到系统生成的链接后,点击单个数据领域属性 URI,如:

http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/1/kmeans_data.xls_1,可查看相应的部门或领域对该装备定义的信息内容,如该领域数据的字段以及字段所对应的数值、数据描述等信息。

通过对各个数据源配置描述、标题、属性、字段的属性定义等,使得用户可通过返回的 JSON 数据以及 UI 界面查看理解不同领域之间的数据定义。

(2) 数据源集成响应时间对比

对基于服务的数据集成与基于数据仓库以及基于联邦数据库集成进行获取数据的平均响应时间对比,获取若干条数据进行查询的平均响应时间进行测试,如图4-24。

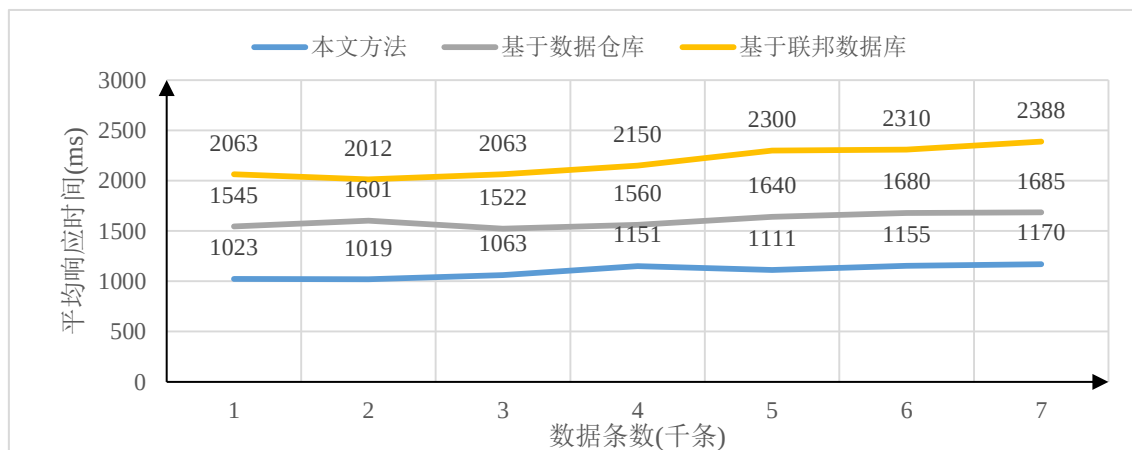


图 4-24 数据查询平均响应时间对比

在第三方平台获取数据时,基于数据仓库的数据集成需要经过数据查询客户端进行代理,将数据请求转入服务端中,因此,对比本文方法,需要多消耗平均505ms的响应时间。基于联邦数据库的集成方法,由于数据库连接映射关系为两两链接,数据需要经过若干数据库才能获取,因此,对比本文方法,需要多消耗平均1084ms。

由于是即时扫描相应领域数据源,基于服务的复杂装备数据集成技术是在用户获取数据时才对数据进行刷新,无需占用过多的系统资源进行监听数据。因此,增加基表的原始记录数或者增加基表的个数,影响查询时间的主要原因是网络或者页面渲染时间。

表 4-1 异构数据集成定性比较

集成方式	读取效率高	包含领域定义及属性	跨平台	实时数据	服务发现	复用简单	异构数据源全覆盖
基于数据仓库	√	×	×	×	×	√	√
基于联邦数据库	×	×	×	√	×	×	×
本文方法	√	√	√	√	√	√	√

而数据仓库则是采用定时刷新策略,对各个数据库进行一段时间内的更新,这将导致刷新不及时,若刷新频率提高,则会增加资源消耗。将本章研究方法与对比

方法在集成架构以及性能上进行对比，如表 4-1。本文方法解决了当前数据集成无法将领域定义传递的问题，同时实现了数据间的服务发现功能，使得各领域相关数据可通过层级 URI 进行访问。

由于基于数据仓库的数据集成的系统架构采用的是一对一的连接，实现各个数据库的数据联通。在连接过程中，需要创建于各个数据源连接的适配器规则，这就造成集成装备数据的冗余以及频繁的转换导致的查询缓慢，也影响了系统的可拓展性。基于联邦数据库方法则是数据库两两链接，在跨平台获取数据时，也需要经过数据客户端进行获取。若采用本章研究的基于服务的复杂装备数据集成方式，由于使用了面向服务的思想，使用三层架构对请求与数据源的隔离，降低耦合，同时解决了数据没有服务发现以及领域定义获取的问题。

4.4 本章小结

本章主要介绍了基于服务的复杂装备数据集成技术架构，指出复杂装备数据异构性以及领域接口不统一的问题。设计数据采集、数据结构转换、数据处理等模块，介绍 OSLC 规范以及对复杂装备数据集成步骤的实现，给出集成映射规则以及领域标准定义。采用面向服务的架构进行设计服务层，将资源作为服务。通过 REST 风格 HTTP 请求，将集成的复杂装备数据发布。最后将本章提出的研究方法与基于数据仓库、基于联邦数据库的数据集成方法进行各性能对比，结合异构数据集成的数据查询实时性以及关系型数据、非关系型数据、文本数据的可集成性进行分析。

第五章 复杂装备异构数据作战场景的实现

5.1 引言

在第三章和第四章中，我们基于异步消息队列设计消息分发架构，结合基于角色模型设计权限的访问，实现复杂装备数据异构数据的动态管理；基于 OSLC 服务核心规范，结合复杂装备领域数据标准定义，实现基于服务的复杂装备数据集成，资源统一命名，接口统一发布。本章目的是通过研究所提供的测试数据，使用本文设计的分析系统进行集成，模拟复杂装备数据分析流程，设计可视化组件以及数据决策界面，最后验证本文所研究的关键技术在实际复杂装备数据分析系统应用的可行性。

5.2 复杂装备异构数据作战场景实验

实验的目的是，对本文研究的两个关键技术点所实现的系统应用测试可行性，通过接入测试的异构数据源，实现复杂装备数据快速集成与共享、资源简便管理。最后生成相应的数据决策可视化界面，实现用户在不同场景中，可以快速做出决策判断。

在 4G 内存 4 核硬件的条件下，基于服务的复杂装备数据分析系统主要采用 Python 建立后端服务，异构数据集成服务部分则使用 Java 进行开发。作为消息队列的数据库只用 Redis 的 Docker 镜像，系统配置以及中间量结果存放使用 PostgreSQL 的 Docker 镜像。最后将后端服务以及依赖打包至 Docker 镜像中。采用 Docker 镜像部署的方式可解决系统环境复杂难以迁移的问题，同时虚拟化手段降低系统资源占用。

Docker 镜像配置为：

表 5-1 镜像配置信息

应用名称	IP:端口
前端平台	192.168.99.100:30091
复杂装备数据资源动态管理平台	192.168.99.100:30050
基于服务的复杂装备数据集成平台	192.168.99.100:8088
消息中间件数据库	192.168.99.100:6379
系统数据库	192.168.99.100:5432

5.2.1 作战场景数据分析平台

考虑到各装备部门信息化发展迅速，各部门、各军种类别之间权限管理复杂冗余，当前设计的数据管理方法不能较好解决问题。作战场景数据分析平台侧重于各个装备部门、各个军种之间复杂装备异构数据权限的划分访问。因此，采用本文研究方法所设计的复杂装备数据动态管理系统，对该场景进行验证。

5.2.1.1 权限分组实验

作战场景数据分析平台中，划分 4 组权限访问级别：电子对抗作战、电子对抗管理、电子对抗装备检测以及系统最高管理员，通过装备资源管理技术对权限进行分组设置，建立相应权限分组，如图 5-1。admin 分组将属性标记为系统最高管理员，该组成员可对数据源、可视化组件共享、仪表盘共享等功能权限设置。其余分组只能配置组内数据源所对应的数据集、可视化组件以及仪表盘的读写权限。

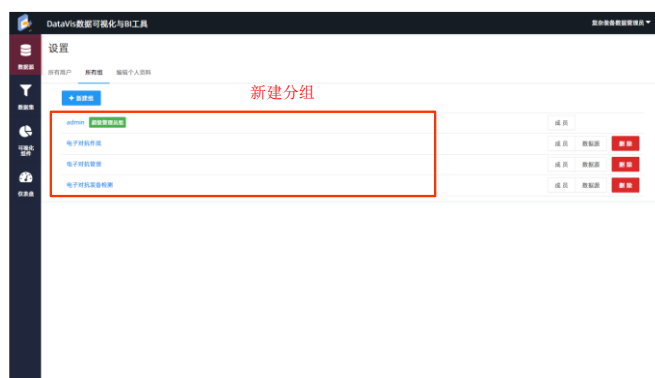


图 5-1 权限分组建立

根据作战环境中各个成员的属性不同，作战场景数据分析平台成员包括：电子对抗装备检测员 1、电子对抗作战员 1、电子对抗作战员 2、电子对抗管理员、复杂装备数据管理员，如图 5-2。

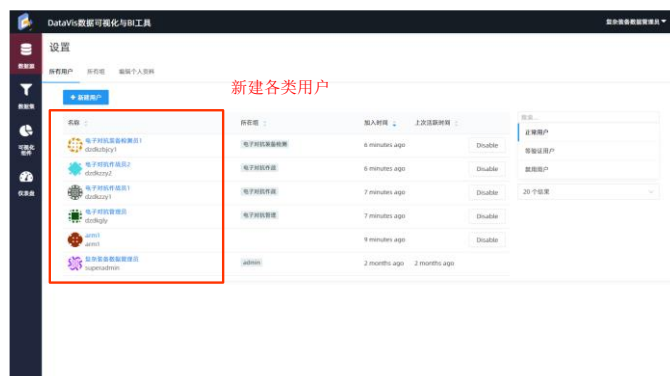


图 5-2 作战场景成员建立

复杂装备数据管理员作为系统权限最高管理，加入至 **admin** 分组。电子对抗管理加入电子对抗管理分组、电子对抗作战员 1 以及电子对抗作战员 2 加入至电子对抗作战分组中，其余的成员角色分配到各自的权限组中，使得成员继承分组的权限内容，如图 5-3，图 5-4。

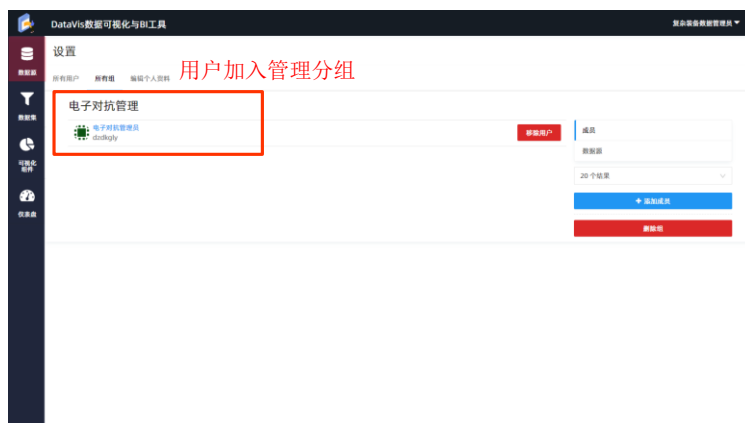


图 5-3 作战场景管理成员分配

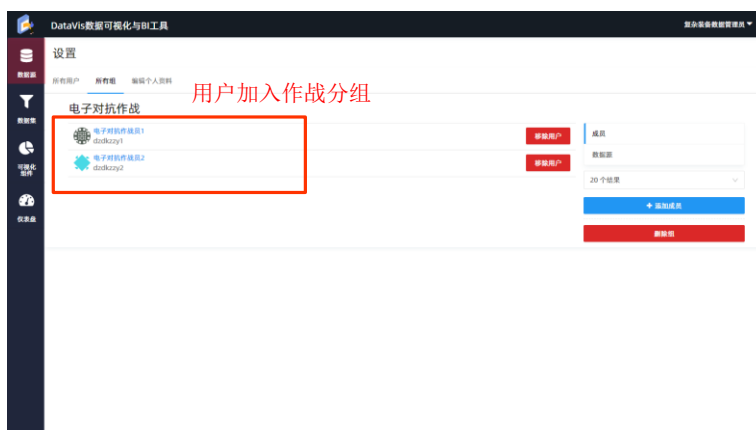


图 5-4 作战场景作战成员分配

根据复杂装备异构数据动态管理框架中的权限控制技术，生成成员角色-分组-权限的映射结构，将可视化分配的用户信息与分组、权限信息进行关联，可在数据库中查看到相应的表结构数据，如图 5-5。id 列作为关联的外键，permissions 包含该分组的权限序列信息。

```
postgres=# select * from groups;
```

id	org_id	type	name	created_at	permissions
1	1	builtin	admin	2020-11-23 07:00:34.595847+00	{admin,super_admin,create_dashboard,create_query,edit_dashboard,edit_query,view_query,view_source,execute_query,list_users,schedule_query,list_dashboards,list_alerts,list_data_sources}
2	1	regular	电子对抗管理	2021-01-12 12:16:53.058327+00	{user}
3	1	regular	电子对抗作战	2021-01-12 12:17:08.92276+00	{user}
4	1	regular	电子对抗装备检测	2021-01-12 12:17:21.319358+00	{admin,create_dashboard,create_query,edit_dashboard,edit_query,view_query,view_source,execute_query,list_users,schedule_query,list_dashboards,list_alerts,list_data_sources}
5	1	regular	电子对抗指挥	2021-01-12 12:28:47.947414+00	{admin,create_dashboard,create_query,edit_dashboard,edit_query,view_query,view_source,execute_query,list_users,schedule_query,list_dashboards,list_alerts,list_data_sources}

(5 rows)

数据库分组权限更新

图 5-5 生成权限表结构

不同成员所在不同权限分组，则各自所能访问的数据源、数据集以及可视化分析都有所不同，如图 5-6，左图为电子对抗作战成员，右图为电子对抗管理成员。制图作为作战成员分组，内部成员只能访问组件 1 和组件 5 的数据内容以及相关可视化组件，电子对抗管理作为当前场景的最高权限拥有分组，其内部成员能够访问所有数据集以及相关可视化组件，同时能够对数据源可读权限进行管理，因此能够获取到组件 1 至组件 6 的数据集。



图 5-6 不同权限查看资源对比

通过查看数据集与成员、权限分组的对应关系，检验数据权限分组的正确性，如表 5-1 所示，各类成员只能访问权限分组提供的数据集集合。

表 5-1 权限访问测试结果

成员	权限分组	数据集 ID	权限是否正确分配
电子对抗作战管理者	管理组	001,002,316,317,318,319,320,321	√
电子对抗作战装备维护员	装备维护组	001,002,316	√
电子对抗作战成员 1	作战成员组	001,002,326,320	√
电子对抗作战成员 2			√
电子对抗需求分析员	需求分析组	001,002,316,317,318,319	√
电子对抗装备制造员	装备制造组	320	√

5.2.1.2 平台验证

根据各成员的不同权限属性，各自设计可视化大屏进行数据分析，如图 5-7、图 5-8。图 5-7 表示电子对抗管理员所展示的可视化大屏，管理员成员拥有最高管理权限以及读写范围，因此，在该大屏中能展示所有可视化分析组件。图 5-8 为电子对抗作战成员所展示的可视化大屏，由于作战成员读写权限的限制，不能访问部分数据集，因此这些数据集对应的可视化组件将不显示在可视化大屏中。

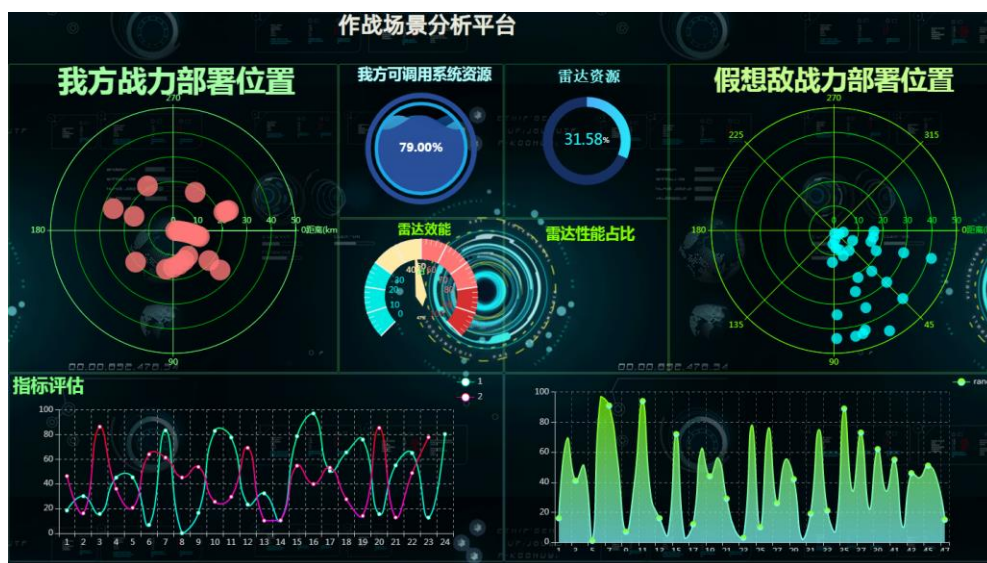


图 5-7 作战场景分析平台验证-管理员

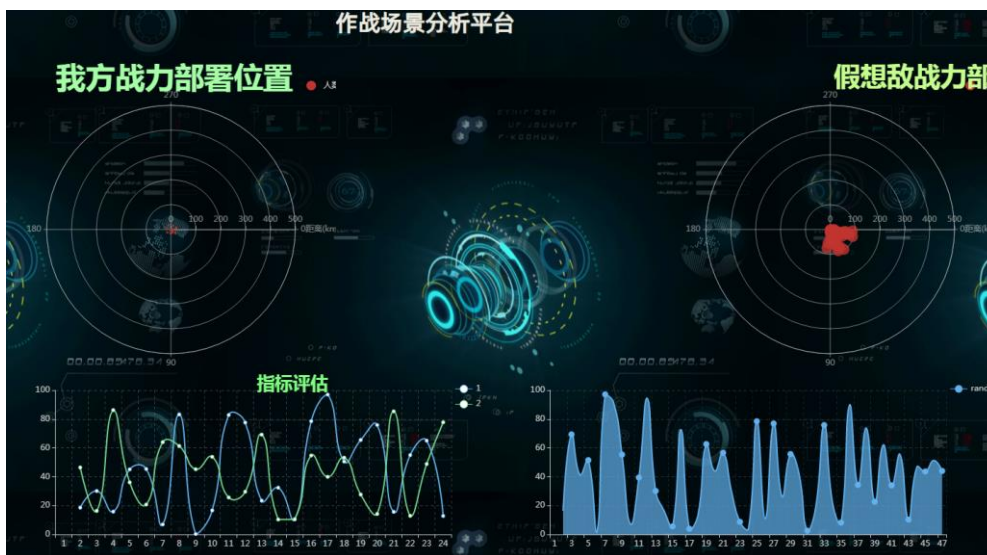


图 5-8 作战场景分析平台验证-作战成员

本小节使用本文研究的基于服务的复杂装备数据分析方法，设计并实现了作战场景数据分析平台。通过对分组以及成员的权限分配，验证了基于角色的数据权

限管理以及基于异步消息队列的资源动态管理的可行性。

5.2.2 复杂装备异构数据测试分析平台

复杂装备信息化发展快、种类多、涉及领域宽，传统复杂装备数据分析的集成异构数据模式粗放单一、接口标准等难以统一，与目标数据统一集成发布存在一定距离。采用本文研究的基于服务的复杂装备数据集成方法，解决许多的冗余、孤岛式的数据集成开发。复杂装备异构数据测试分析平台侧重于各类复杂装备领域中的异构数据集成，因此，设计并实现复杂装备异构数据测试分析平台以验证本文研究方法的可行性。

5.2.2.1 文本型数据链接

根据 OSLC 核心规范，首先系统将扫描出复杂装备异构数据源列表，即服务提供者列表，如图 5-9。由于 OSLC 的服务发现思想，修改 URI 各级名称即可返回上层或者找到其他相关服务 URI 信息。

通过服务发现，可查找到当前所有集成的异构数据源，下层服务提供者目录对异构数据类型进行划分，第一条 URI 代表文本数据类型的 excel 数据源、第二条 URI 代表关系型数据类型的 Mysql 数据源、第三条 URI 代表非关系型数据类型的 Redis 数据源。



图 5-9 文本型服务提供者列表查看

若点击代表文本型数据源的第一条 URI: <http://localhost:8088/catalogs/excel>,

即可进入相应数据源的服务提供者，即，该数据源所集成的文本数据列表，如图 5-10。服务提供者列表中，列出所集成的 5 条文本数据资源，资源名称通过 URI 表示，如：kmeans_data、T1、T20、T22_data 以及 tag1 数据资源。点击相应的 URI 即可查看该数据源的数据、数据领域定义、数据读写属性等信息。



图 5-10 文本型资源列表查看

点击服务提供者 Service Provider 'Excel service provider kmeans_data.xls', 进入查看资源创建类型、资源查询能力、资源类型对应的 URI 链接，如图 5-11。Service Provider 表示服务名称类型以及数据源名称，创建资源类型 URI 可获取数据字段的具体属性类型，资源查询能力 URI 表示用户可通过该 URI 链接获取该数据资源。

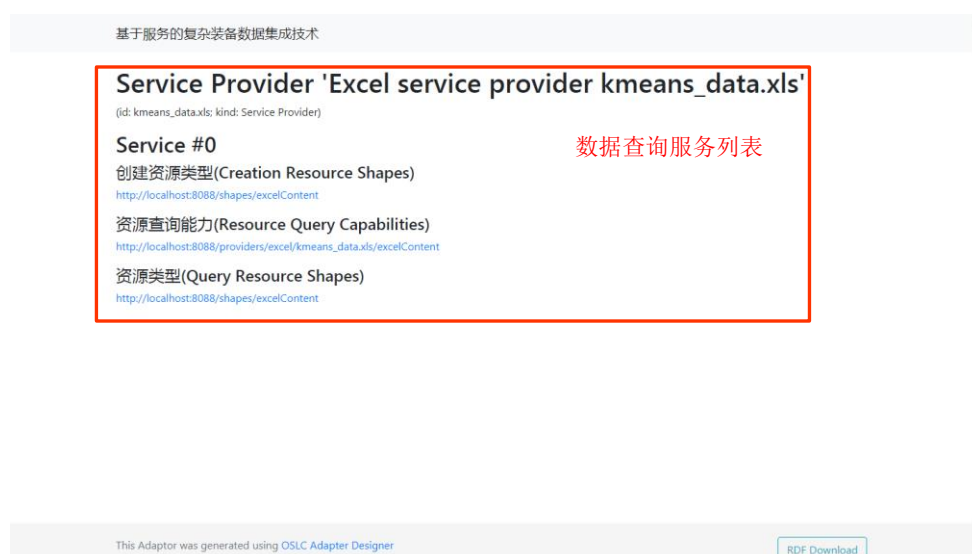
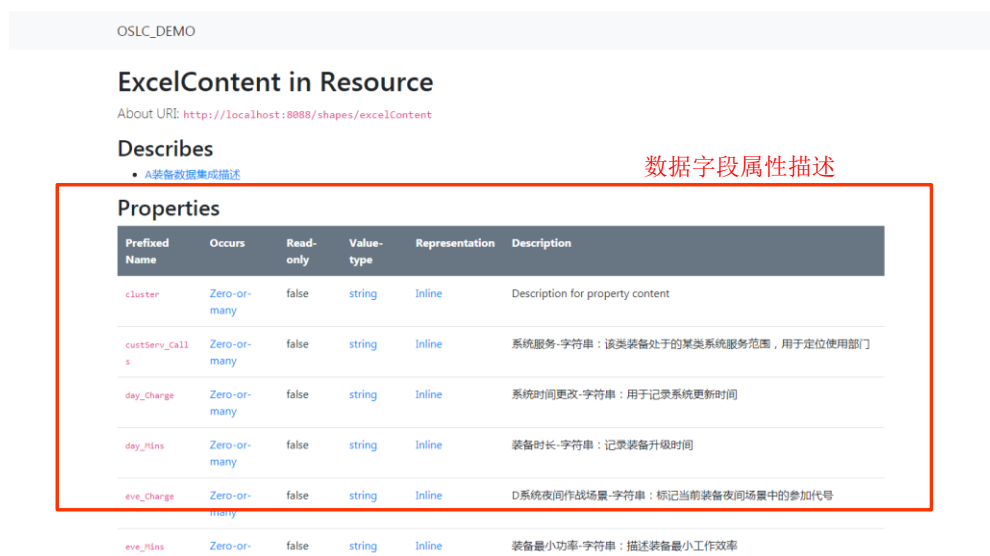


图 5-11 文本型资源属性 URI 查看

点击查看创建资源类型 URI:<http://localhost:8088/shapes/excelContent>, 可查看资源描述以及属性, 包括字段名、字段类型、只读、值类型、字段描述等, 可查看该领域异构数据的标准定义以及描述, 如图 5-12。由属性表格可知, 该数据源 cluster 字段: Read-only 表示改字段为可读可写类型数据、Value-type 表示字符串类型数据、Description 表示该字段的领域定义。



OSLC_DEMO

ExcelContent in Resource

About URI: <http://localhost:8088/shapes/excelContent>

Describes

- A装备数据集描述

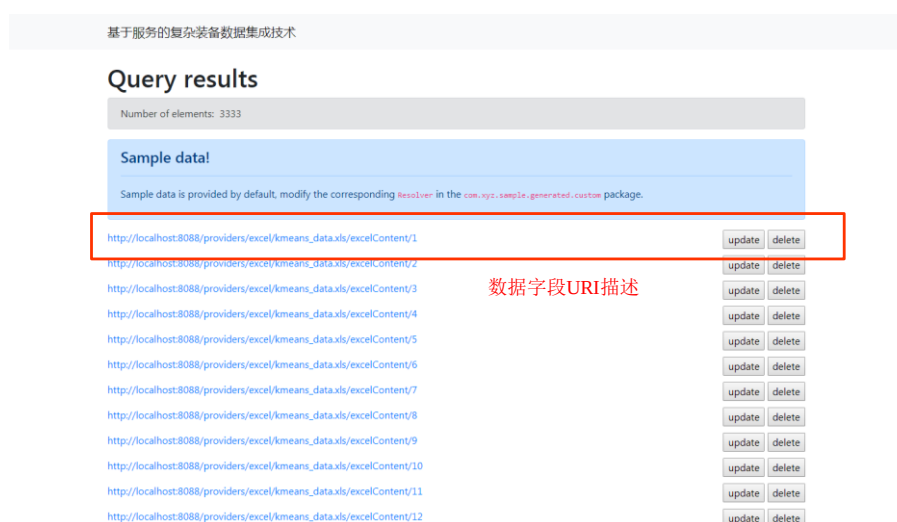
数据字段属性描述

Prefixed Name	Occurs	Read-only	Value-type	Representation	Description
cluster	Zero-or-many	false	string	Inline	Description for property content
custServ_calls	Zero-or-many	false	string	Inline	系统服务-字符串: 该类装备处于的某类系统服务范围, 用于定位使用部门
day_Change	Zero-or-many	false	string	Inline	系统时间更改-字符串: 用于记录系统更新时间
day_Hins	Zero-or-many	false	string	Inline	装备时长-字符串: 记录装备升级时间
eve_Change	Zero-or-many	false	string	Inline	D系统夜间作战场景-字符串: 标记当前装备夜间场景中的参加代号
eve_Hins	Zero-or-many	false	string	Inline	装备最小功率-字符串: 描述装备最小工作效率

图 5-12 文本型字段属性查看

点击查看资源查询能力 URI:

http://localhost:8088/providers/excel/kmeans_data.xls/excelContent。可查看各个字段单独的 URI 链接, 同时可查看各个字段各自的领域属性, 如图 5-13。通过数据 id 表示各行数据 URI, 可对该行数据进行更新或删除操作。



基于服务的复杂装备数据集集成技术

Query results

Number of elements: 3333

Sample data!

Sample data is provided by default, modify the corresponding Resolver in the `com.vyz.sample.generated.custom` package.

http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/1	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/2	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/3	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/4	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/5	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/6	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/7	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/8	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/9	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/10	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/11	update	delete
http://localhost:8088/providers/excel/kmeans_data.xls/excelContent/12	update	delete

数据字段URI描述

图 5-13 文本型资源查询能力 URI

通过点击数据资源 URI，即可查看该资源在领域中的数据字段、类型、定义等信息，如图 5-14。

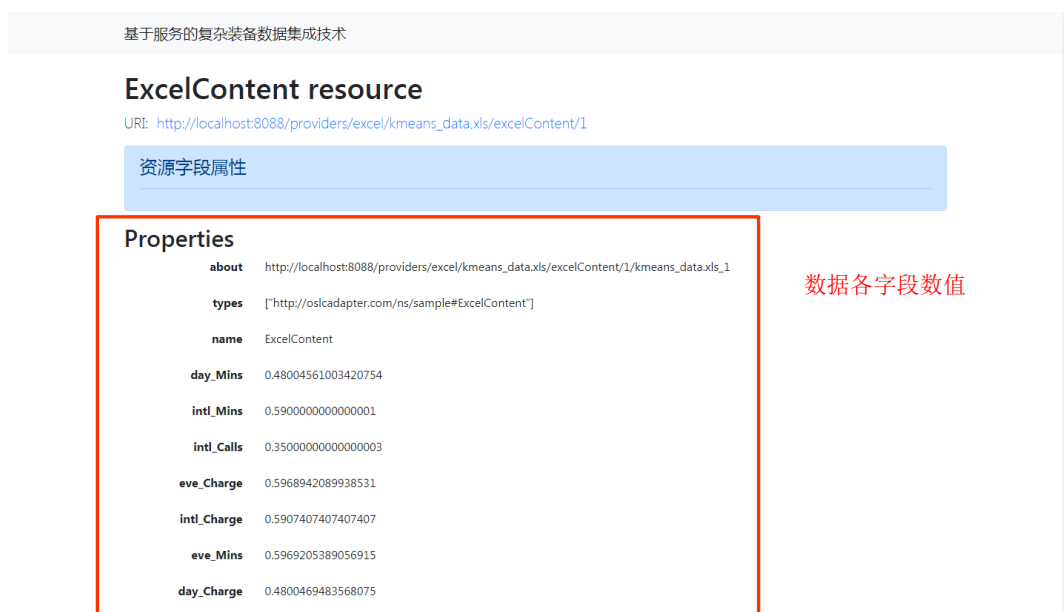


图 5-14 文本型数据属性查看

5.2.2.2 关系型数据库链接

同上一小节类似，通过 OSLC 核心规范的描述，生成服务提供者目录、服务提供、查询能力等 URI 链接，供用户使用，如图 5-15。该类集成数据服务有 2 条 URI，AIR_Foce 以及 army 数据源。



图 5-15 关系型数据库资源列表

5.2.2.3 非关系型数据库链接

针对非关系型数据,经过统一的服务化处理后,也同关系型、文本数据一样,可生成相应的 URI 链接对数据源进行访问、数据查询等,如图 5-16。通过对文本数据传入值 redis 中,在该类数据源即可扫描出 5 条数据服务。



图 5-16 关系型数据库资源列表

5.2.2.4 平台验证

通过资源查询能力 URI 链接,接入平台数据集查询,即可与复杂装备异构数据源连接上并获取数据,如图 5-17。通过资源查询能力页面生成的 URI: http://localhost:8088/providers/excel/kmeans_data.xls/excelContent,在平台中配置为 URI 数据源,在数据集查询页面写入该 URI,点击执行,即可通过 URI 获取到链接的资源数据,通过对比真实文本数据源内容,接入数据无误。

通过建立数据集,在可视化组件部分选用该数据集构建各类数据分析组件,在仪表盘页面对创建的组件进行布局规划,即可设计出数据分析平台,如图 5-18。图中可视化组件主要由字段 eve_Mins、day_Mins 以及 cluster 节点名称绘制,用于分析昼夜装备信号以及各项性能,在可视化大屏中进行展示。



图 5-17 URI 数据链接查询

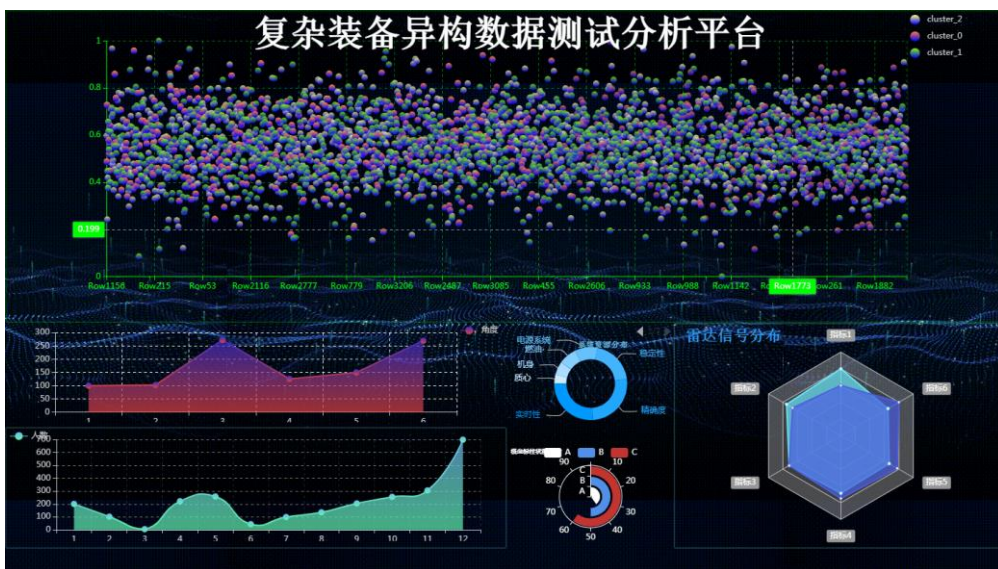


图 5-18 复杂装备异构数据测试平台验证

本小节使用本文研究的基于服务的复杂装备数据分析方法，设计并实现了复杂装备异构数据测试分析平台。通过链接文本异构数据、关系型异构数据以及非关系型异构数据进行服务化集成。

实现复杂装备领域异构数据的统一集成以及统一进行 URI 的服务化发布。然后，通过平台接收异构数据，形成数据集。最后，生成复杂装备异构数据测试分析平台进行展示。验证了基于服务的复杂装备数据集成优化方法针对复杂装备领域异构数据集成的可行性。

5.3 本章小结

本章主要针对复杂装备数据分析中的两大问题：异构数据动态管理以及异构数据集成，对两大问题进行场景实现。本章通过研究所提供的测试数据，使用本文的技术对装备数据进行集成，模拟实际的数据分析流程，设计可视化组件以及最后的数据可视化决策界面，通过该实验，以验证本文所研究的关键技术在实际可视化数据分析系统应用的可行性。分别对两个核心问题所对应的复杂装备数据场景进行实现验证，完成了对两个场景的创建。

第六章 全文总结与展望

6.1 全文总结

在信息化战争的倒逼下,军用复杂装备逐渐向智能化方向发展。智能化的军用复杂装备涉及多个工程领域,导致领域间无法理解数据定义。同时也存在各领域间的异构数据集成困难,出现“信息孤岛”现象。复杂装备场景变化快速,工具平台多样,当前资源管理以及集成方案无法应对新情况的出现。因此选用复杂装备作为场景,研究多领域间的资源管理以及异构数据集成技术。

(1) 介绍了复杂装备数据分析相关背景概念和涉及的关键技术,同时总结了近年来复杂装备多源异构数据管理与集成领域的主要研究成果。

(2) 阐述了复杂装备数据分析各个关键技术主流的设计方法,总结它们各自的优缺点。

(3) 提出了基于异步消息队列的装备资源管理优化技术,设计一种基于异步消息队列的负载均衡技术,结合基于角色的访问控制,实现复杂装备异构数据动态管理。根据复杂装备数据权限特点,设计分层权限访问方法,通过添加属性的方式拓展分组权限分配功能。使用异步任务发布、任务调度器、任务执行单元三部分,实现复杂装备数据消息分发,通过比较不同数量级的相对并发请求测试,得到单机服务器响应速度对比其他主流方法平均提高 7.6%。

(4) 提出了基于服务的复杂装备数据集成优化技术,设计一种以面向服务为基础架构,采用 OSLC 核心规范定义复杂装备数据属性,使用数据映射驱动模版和 REST 服务化发布形式的复杂装备数据集成技术。通过装备数据采集、数据结构转换、数据处理引擎各个模块处理,优化查询性能,实现数据与服务的解耦合。

(5) 针对基于异步消息队列的复杂装备资源管理优化技术以及基于服务的复杂装备数据集成优化技术,结合复杂装备数据分析场景实验,对以上两种技术结合实际数据集成平台进行验证。测试证明,该系统实现了复杂装备资源管理能力,实现装备部门分组,成员加入分组继承权限,多工作单元并行处理任务,优化系统性能。同时,该系统实现了基于 OSLC 规范设计的 URI 资源获取,展示了服务提供者列表、服务列表、资源查询能力以及资源属性等 URI 链接。

6.2 后续工作展望

本文基于异步消息队列的装备资源管理技术以及基于服务的复杂装备数据集成技术,仍有补充和完善的空间,内容大致包括一下几个方面:

(1) 本文仅在单机系统上做资源动态分配的优化处理, 可通过系统的核数匹配工作单元数量, 但在多服务节点上仍未做优化处理, 多服务节点是以多个单机服务作为集群化处理, 需要考虑到各个节点之间的网络匹配关系, 多方协同处理任务消息。同时需要考虑到多服务节点资源同步问题以及哈希一致性问题。

(2) 本文仅利用 RBAC 模型做了一部分改进处理, 将角色层抽象为分组层, 同时增加分组的属性进行拓展处理, 但仍有更复杂的装备权限情况未考虑到, 改进后的权限模型未能更好地适应各类场景需求。

(3) 本文的复杂装备数据集成依靠的是 OSLC 核心规范所对应的数据映射模板, 模板的属性配置需要与 OSLC 规范相对应, 若需要拓展更多的装备领域中的属性配置, 则需要改动数据容器类定义以及规范定义。因此, 这部分可以作为后续更加灵活配置加入数据源的解耦合处理。

(4) 本文结合以上关键技术所构建的复杂装备数据分析平台仍有更多的提升空间, 需要配合各个模块的优化, 改进本文的关键技术研究方法, 丰富平台功能。

致 谢

时间过的很快，三年的研究生学业就要完结，即将告别最后在校学习的时期。回看过去研究生学习生活期间，学到很多以前没接触过的东西、遇到了各种各样的事情、结交了天南地北的朋友，研究生期间既是高兴的也是幸运的。感谢在三年求学之路中，能学习、认知和感悟到很多受益匪浅的东西，庆幸自己在求学的过程中都会遇到一群可爱、阳光的良师益友，无论在生活还是学习中，都给与我很多真诚地帮助。

首先十分感谢电子科技大学在我研究生求学期间提供了丰富的学术科研资源，良好的学习环境，舒适的生活。

求学三载，最要感谢的是我的指导老师王坚教授以及导师杨鍊副教授，两位老师循循善诱，谆谆教诲，在讨论项目以及讨论学术问题中，怀着严谨的学术态度，遵循“推敲”二字，让我认识到自己该如何看待学习过程中遇到的问题，思考的方向，细节往往是最关键的突破点，让我在学习理解新事物的过程中锻炼了自己，认清自己不足并加以改进。合作项目中遇到的问题以及处理与人交流沟通的难题，任务的安排，老师们都在指导我看待问题、分析难点的思维角度和思考深度，锻炼了我有深度多角度地看问题，寻找突破瓶颈的能力。在老师们认真、负责、耐心地指导以及真切帮助下，科研能力不强的我才能完成这篇论文相关的研究和写作。

同窗情深，友谊地久天长，更要感谢 KB341 教研室的兄弟姐妹萌：徐博轩、杨文秀、彭玉婷、师雨、陈子昱、李东升、朱继飞，他们都很会整活，使得让人自闭的科研生活变得有趣。讨人喜欢的幽默的师弟师妹，顾旭辉纯纯的好人，有趣又认真磕盐的师兄师姐。三年来，他们在科研上给了我欢乐，师兄们在我刚入学的时候也提供无数帮助，在生活中给了我很多启发。

真心地感谢我的家人对我的不断支持鼓励，在家做好吃的，在外一直打电话关心，让我在大学、在考研和科研过程中一直往前冲，在低落的时候鼓励，给了我最大的帮助。

最后，感谢百忙中评审论文并且给予我宝贵修改建议的各位老师！

参考文献

- [1] 王加,吴迪,何嘉武,郭英.装备保障异构大数据统一访问与转换平台的构建研究[J].科技与创新,2016(03):6-9
- [2] 周红桥,陈兴玉,张红旗.军用电子装备全生命周期大数据标准体系研究[J].标准科学,2020(02):39-42
- [3] 郭文越,陈虹,刘万军.基于 SOA 的数据共享与交换平台[J].计算机工程,2010,36(19):280-282
- [4] K. Mikhail, E. Liubov, F. Vladimir, B. Marina. Intelligent Approach for Heterogeneous Data Integration: Information Processes Analysis Engine in Clinical Remote Monitoring Systems[J]. Procedia Computer Science, 2019, 156(C): 134-141
- [5] L. Q. Han, Q. S. Dou, H. J. Li. Visual model of heterogeneous data sources based on service-ontology[C]. 2010 Seventh International Conference on Fuzzy Systems and Knowledge Discovery, Yantai, 2010, 2945-2949
- [6] A. M. Tjoa, The Linked Data Enterprise as Enabler for both Intra- and Inter-organizational Business Data Integration and Usage[C]. 2020 43rd International Convention on Information, Communication and Electronic Technology (MIPRO), Croatia, 2020, 1260-1262.
- [7] 李瑞轩.异构信息集成中的查询处理与优化研究[D].湖北:华中科技大学,2004,10-19
- [8] 贺永鑫.异构数据集成平台研究[D].山西:中北大学,2010,3-7
- [9] 屈振新,唐胜群.信息集成中冲突的语义解决方案[J].计算机科学,2010,37(01):167-169
- [10] M. P. Papazoglou, W. Heuvel. Service oriented architectures: approaches, technologies and research issues[J]. Vldb Journal, 2007, 16(3): 389-415
- [11] S. J. Zheng, W. Ma, J. L. ZHANG. Analysis of Distributed Video-On-Demand System Based on Cluster[C]. In: Proc. 2007 International Symposium on Distributed Computing and Applications to Business, Engineering and Science, Huber Science and Technology Press, Wuhan China, 2007, 400-403
- [12] 姚兆凡.轻量级 Web 服务器 Nginx 的研究与优化[D].江苏:南京邮电大学,2017,7-18
- [13] A. Gao, D. H. Liu. Study on Heterogeneous Data Integration of Agricultural Products Based on XML[J]. Standard Science, 2016(02): 24-34
- [14] X. Zhang, H. Guo, H. Chen and J. Gong. Research on IETM Heterogeneous Data Integration of Quartermaster Equipment Based on XML[C]. Sixth International Symposium on Computational Intelligence and Design, Hangzhou, 2014, 124-127

- [15] M. Lenzerini. Data Integration: A Theoretical Perspective[C]. Proceedings of the Twenty first ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, USA, 2002, 134-152
- [16] M. Matyoqubov, A. Saidov, O. Kazakov and O. Rustamova, Enterprise Systems Data In tegration[C]. 2020 International Conference on Information Science and Communications Technologies (ICISCT), Uzbekistan, 2020, 1-3.
- [17] Y. G. Alley. White House Big Data Initiative [M]. Cham: Schintler, Laurie A, 2017, 1-5
- [18] H. J. Xiao, F. Wang, F. M. Wang, X. Z. Wang. Some key problems of data management in army data engineering based on big data[C]. IEEE International Conference on Big Data Analysis, Beijing, 2017, 149-152
- [19] 黄焕昭.基于 Nginx 的 Web 服务器集群负载均衡技术研究[D].广东:广东工业大学,2019,7-20
- [20] B. Jitendra, P. Tirth, T. Harshal, M. Vishrut. HTV Dynamic Load Balancing Algorithm for Virtual Machine Instances in Cloud[C]. International Symposium on Cloud and Services Computing, Yantai, 2012, 2945-2949
- [21] D. M. Dias, W. Kish, R. Mukherjee. A scalable and highly available Web server[C]. Technologies for the Information Superhighway Digest of Papers, Santa Clara, USA, 1996, 85-92
- [22] V. Cardellini, M. Colajanni. Dynamic load balancing on Web-server systems[J]. IEEE Internet Computing, 1999, 3(3): 28-39
- [23] E. Casalicchio, S. Tucci. Static and Dynamic Scheduling Algorithms for Scalable Web Server Farm[C]. 9th Euromicro Workshop on Parallel and Distributed Processing, Mantova, 2001, 369-376
- [24] Y. F. Zhang, Q. L. Wang, Y. Zhang. Load balancing algorithm based on load weights[J]. Application Research of Computers, 2012, 29(12): 4711-4713
- [25] Z. R. Deng, X. X. Tang, W. M. Huang, Y. W. Li. A load balancing scheduling algorithm for web server cluster[J], Computer Applications and Software, 2013, 30(10): 53-56
- [26] L. Rao, X. C. Tang, Z. J. Hou. Research on Load Balancing Strategy for Server Cluster[J], Computer and Modernization, 2013, 29(12): 29-32
- [27] X. Y. Zhang, Y. L. Zhao, X. Su. Load balancing algorithm based virtual machine dynamic migration scheme for datacenter application with optical networks[C]. International Conference on Communications and Networking in China. Kun Ming, 2013, 271-276

- [28] N. X. Xiong, K. H. Xu, L. L. Chen. An Effective Self-adaptive Load Balancing Algorithm for Peer-to-Peer Networks[C]. International Conference on Communications and Networking in China, 2012, 1425-1432
- [29] Y. Wu, S. Luo, Q. Li. An Adaptive Weighted Least-Load Balancing Algorithm Based on Server Cluster[C]. International Conference on Intelligent Human-Machine Systems and Cybernetics, Hangzhou, 2013, 224-227
- [30] 覃川. 基于 Nginx 的 Web 服务器负载均衡策略改进与实现[D]. 四川: 西南交通大学, 2017, 2-17
- [31] 王永辉. 基于 Nginx 高性能 Web 服务器性能优化与负载均衡的改进和实现[D]. 四川: 电子科技大学, 2015, 2-18
- [32] P. Florin, X. M. Zhu, L. T. Yang. MidHDC: Advanced Topics on Middleware Services for Heterogeneous Distributed Computing. Part 1[J]. Future Generation Computer Systems, 2016, 56: 734-735
- [33] P. Florin, X. M. Zhu, L. T. Yang. MidHDC: Advanced topics on middleware services for heterogeneous distributed computing. Part 2[J]. Future Generation Computer Systems, 2017, 74(SEP.): 86-89
- [34] 张阳. 大数据可视化统计分析通用平台的设计与实现[D]. 北京交通大学, 2019.
- [35] Q. Y. Jiang. Development and Present Situation of Heterogeneous Data Integration Technology[J]. Computer Engineering and Applications, 2002(11): 112-114
- [36] 翟妍伟. 基于联邦的异构数据集成与交换系统研究与实现[D]. 西南交通大学, 2015.
- [37] X. Fang, J. S. Liu. Research and Analysis of Integration Technology[J]. Ship Electronic Engineering, 2006, 12(5): 11-13
- [38] S. Alkis. Mapping conceptual to logical models for ETL processes[C]. Proceedings of the 8th ACM international workshop on Data warehousing and OLAP, Bremen, 2005, 67-76
- [39] Y. R. Cao, J. T. Bi, T. H. Chi, J. B. He. Research on data collection in science data sharing project[C]. Science of Surveying and Mapping, Fu Jian, 2005, 615-621
- [40] J. Han. OLAP mining: an integration of OLAP with data mining[M]. Germany: Springer, 1998, 3-20
- [41] P. Liu, C. Tian. Research on multi-source heterogeneous data collection for the Smart City public information platform[C]. Geoscience and Remote Sensing Symposium, Japan, 2016, 623-626
- [42] D. C. Marinescu. Parallel and Distributed Systems[M], California: Morgan Kaufmann, 2018, 113-150

- [43] G. S. Zhu, H. Huang, W. D. Yang. Keyword Search Based XML Data Source Selection[J]. Journal of Chinese Computer Systems, 2012, 33(06): 1183-1188
- [44] R. Y. Wang, D. M. Strong. Beyond Accuracy: What Data Quality Means to Data Consumers[J]. Journal of Management Information Systems, 1996, 12(4): 5-33
- [45] I. K. Pepper, R. Wolf. International Council on Systems Engineering[M]. San Diego: Ceed Publishing, 2012, 26-32
- [46] M. Alachew, P. Adrian, A. Adeel. Traceability Support in OpenModelica Using Open Services for Lifecycle Collaboration (OSLC)[C]. The 12th International Modelica Conference, Prague, 2017, 823-830
- [47] M. Saadatmand. OSLC Tool Integration and Systems Engineering -- The Relationship between the Two Worlds[C]. Euromicro Conference on Software Engineering and Advanced Applications, Verona, 2014, 93-101
- [48] S. Paschke, S. Softic. Open Services for Lifecycle Collaboration: Ein Ansatz zur Unterstützung der Zusammenarbeit in der Produktentwicklung[J]. Mensch & Computer Workshopband, 2014, 12(4): 313-320
- [49] I. K. Pepper, R. Wolf. The International Council on Systems Engineering (INCOSE)[M]. San Diego: Ceed Publishing, 2012, 45-48
- [50] J. Chen, Z. C. Hu, J. Z. Li. An Open Source Lifecycle Collaboration Approach Supporting Internet of Things System Development[C]. 14th Annual System of Systems Engineering Conference, Anchorage, 2019, 245-248
- [51] L. Y. Bao, D. F. Zhao, Y. F. Zhao. A priority-based polling scheduling algorithm in web cluster servers[C]. IEEE International Conference on Computer Science and Information Technology, Cheng Du, 2010, 501-505
- [52] J. Chang, S. Junde, Z. Q. Lin. Scheduling Algorithm of Load Balancing Based on Dynamic Policies[C]. International Conference on Networking and Services, Cancun, 2010, 363-367
- [53] L. Kang, X. Ting. Application of adaptive load balancing algorithm based on minimum traffic in cloud computing architecture[C]. International Conference on Logistics, Australia, 2016, 251-253
- [54] W. Ying, S. H. Luo, Q. Li. An Adaptive Weighted Least-Load Balancing Algorithm Based on Server Cluster[C]. International Conference on Intelligent Human-Machine Systems and Cybernetics, Hangzhou, 2013, 224-227

- [55] E. Qin, Y. Wang, L. Yuan and Y. Zhong, Research on Nginx Dynamic Load Balancing Algorithm[C]. 2020 12th International Conference on Measuring Technology and Mechatronics Automation (ICMTMA), Thailand, 2020, 620-624.
- [56] C. R. Yan, M. Zhu, Y. Q. Shi. A Response Time based Load Balancing Algorithm for Service Composition[C]. 2008 Third International Conference on Pervasive Computing and Applications, Alexandria, 2008, 13-16
- [57] F. Ebadifard, S. M. Babamir and S. Barani, A Dynamic Task Scheduling Algorithm Improved by Load Balancing in Cloud Computing[C]. 2020 6th International Conference on Web Research (ICWR), Iran, 2020, 177-183.
- [58] X. Yu and Q. Wu, Multi-source Heterogeneous Data Association Technology to Build Public Safety Big Data Integration Research[C]. 2020 International Conference on Big Data Economy and Information Management (BDEIM), Zhengzhou, 2020, 17-20.
- [59] B. M. H. Awadallah, S. C. Haw, L. K. Soon. A Review on Utilising XML as the Mediated Layer for Data Integration[J]. Journal of Computational and Theoretical Nanoscience, 2018, 24(2): 1191-1195
- [60] Y. Q. Cheng, Z. Y. Wu. Design Of Digital Campus Platform Data Integration System Based On Xml[C]. 2019 International Conference on Intelligent Transportation, Big Data and Smart City (ICITBS), Chang Sha, 2019, 276-281
- [61] G. Jayashree and C. Priya, Data Integration with XML ETL Processing[C]. 2020 International Conference on Computer Science, Engineering and Applications (ICCSEA), Gunupur, India, 2020, 1-8.
- [62] 孙书琼.基于 JSON 的异构数据集成的研究[D].云南:云南大学,2017,5-13
- [63] I. Glickstein, S. Ruberg. Database management for integrated avionics system[C]. Aerospace and Electronics Conference, USA, 2002, 617-622
- [64] X. H. Zhang, H. F. Guo, H. Y. Chen. Research on IETM Heterogeneous Data Integration of Quartermaster Equipment Based on XML[C]. International Symposium on Computational Intelligence and Design, Hangzhou, 2013, 124-127
- [65] S. P. Zhou, P. Wang. The Integration of Multi-Source Heterogeneous Data Based on Middleware[C]. International Conference on Information Science and Engineering, Nan jing, 2009, 2212-2215
- [66] 姚崇东.基于 XML 的多源异构数据集成的实现方法研究[D].哈尔滨:哈尔滨工程大学,2007,7-8

- [67] 苏琪,刘西林,王军.基于 WebService 的数据集成及研究[J].计算机技术与发展,2014,24(8):51-54
- [68] 李晓丽.异构数据集成技术在物联网中的研究与应用[D].北京:北京邮电大学,2012,4-7
- [69] W. Ling, M. F. Dong. Estimation of Missing Values Using a Weighted K-Nearest Neighbors Algorithm[C]. International Conference on Environmental Science and Information Application Technology, Wuhan, 2009, 660-663
- [70] J. Sreemathy, I. Joseph V., S. Nisha, C. Prabha I. and G. Priya R.M., Data Integration in ETL Using TALEND[C]. 2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 2020, 1444-1448.
- [71] E. S. Richard, S. Douglas. Middleware for Distributed Systems - Evolving the Common Structure for Network-centric Applications[C]. Chapter in the Encyclopedia of Software Engineering, USA, 2002, 801-813
- [72] Y. Cong, Research on Data Association Rules Mining Method Based on Improved Apriori Algorithm[C]. 2020 International Conference on Big Data & Artificial Intelligence & Software Engineering (ICBASE), Bangkok, Thailand, 2020, 373-376.

攻读硕士学位期间取得的成果

1. 参与项目

2019.10-2021.3 基于原生云架构的数据可视化设计与服务

2. 获得奖励

2018.10 研究生奖学金三等

2020.10 研究生奖学金三等